

Unit 1

Introduction to programming in C

Computer

- The word computer comes from the word “compute”, which means, “to calculate”.
- A computer is an electronic device that can perform arithmetic operations at high
- speed and it can process data, pictures, sound and graphics.
- It can solve highly complicated problems quickly and accurately.
- Advantages: Speed, storage, accuracy, reliability, multitasking, automation
- Disadvantages: lack of intelligence, unable to correct mistakes

Software

- A set of instruction in a logical order to perform a meaningful task is called program and a set of program is called software.
- It tell the hardware how to perform a task.

Types of software

- System software - Examples: Windows, Linux, Unix etc.
- Application software - Examples: Microsoft Word, Excel, PowerPoint etc.

Programming languages

- Language is a mode of communication that is used to **share ideas, opinions with each other**. For example, if we want to teach someone, we need a language that is understandable by both communicators.
- A programming language is a **computer language** that is used by **programmers (developers) to communicate with computers**. It is a set of instructions written in any specific language (C, C++, Java, Python) to perform a specific task.
- A programming language is mainly used to **develop desktop applications, websites, and mobile applications**.

Types of programming language

- Machine level language OR Low level language
 - It is language of 0's and 1's.
 - Computer directly understand this language.
- Assembly language
 - It uses short descriptive words (MNEMONIC) to represent each of the machine language instructions.
 - It requires a translator known as assembler to convert assembly language into machine language
 - so that it can be understood by the computer.
 - Examples: 8085 Instruction set

Types of programming language

- Higher level language
 - It is a machine independent language.
 - We can write programs in English like manner and therefore easier to learn and use.
 - Examples: C++, JAVA etc...

Types of programming language

- C language is one of the most popular computer languages today because it is a structured, high level and machine independent language.
- It allows software developers to develop software without worrying about the hardware platforms, where they will be implemented.
- It can be used to write high-performance code for both application and system software.
- Thus, C is best suited where speed, space, and portability are important.
- Most of the operating systems and gaming software are also written in C.
- C has the features of both assembly level languages i.e low-level languages and higher level languages. So that's why C is generally called as a middle-level Language.

Types of programming language

- C has the features of both assembly level languages i.e low-level languages and higher level languages. So that's why C is generally called as a middle-level Language.
- C programming supports Inline assembly language programs. We can directly access system registers with the help of inline assembly language feature in C. C programming is used to access memory directly using a pointer. C programming also supports high-level language features.

Flowchart

- Flowchart is a pictorial or graphical representation of a program.
- It is drawn using various symbols.
- Easy to understand.
- Easy to show branching and looping.
- Flowchart for big problem is impractical.

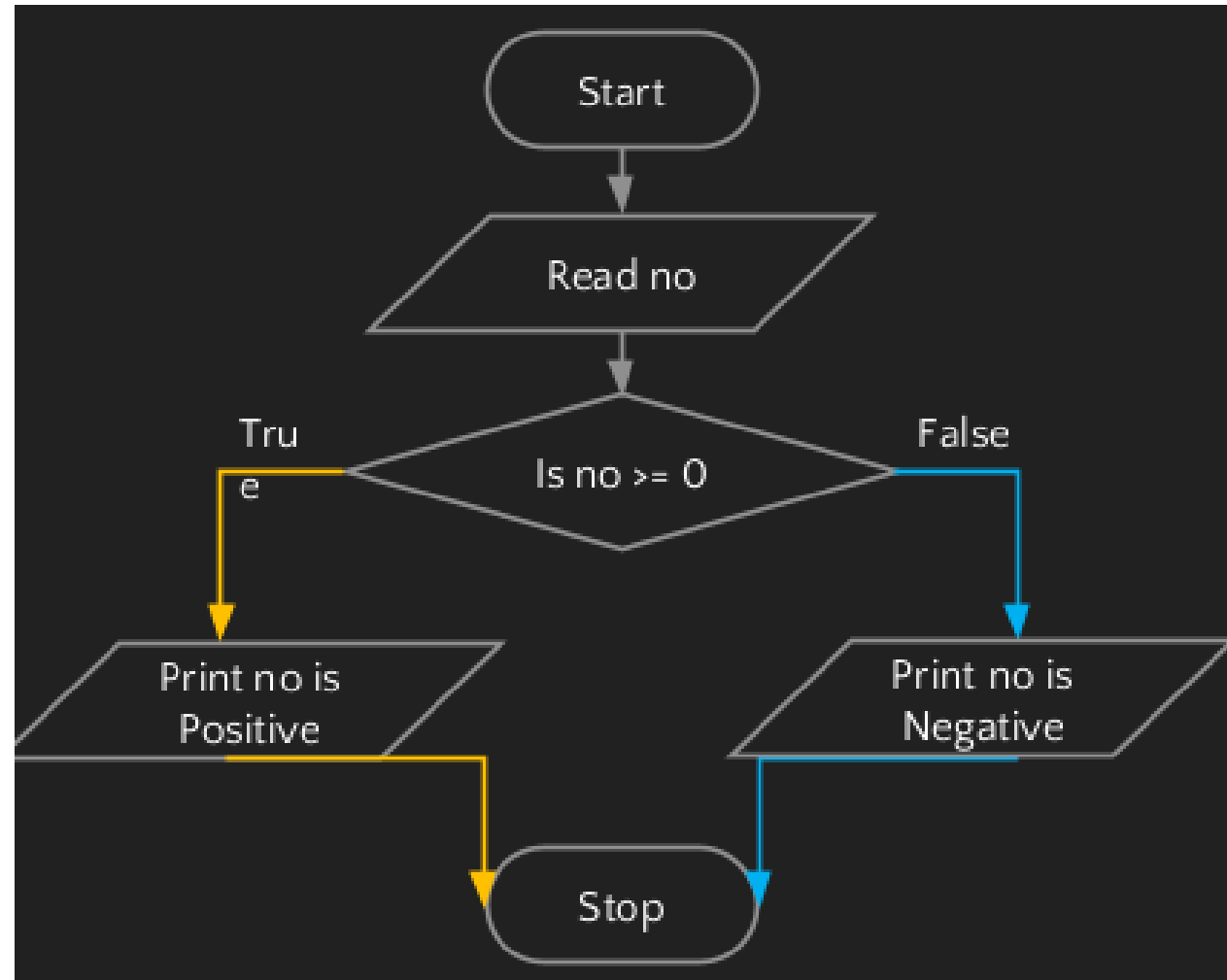
Symbols used in Flowchart

Symbol	Symbol Name	Description
	Flow Lines	Used to connect symbols
	Terminal	Used to start, pause or halt in the program logic
	Input/output	Represents the information entering or leaving the system
	Processing	Represents arithmetic and logical instructions
	Decision	Represents a decision to be made
	Connector	Used to Join different flow lines
	Sub function	used to call function

Algorithm

- Algorithm is a finite sequence of well defined steps for solving a problem.
- It is written in the natural language like English.
- Difficult to understand.
- Difficult to show branching and looping.
- Algorithm can be written for any problem.

Example: Number is positive or negative



Example: Number is positive or negative

- Algorithm

Step 1: Read no.

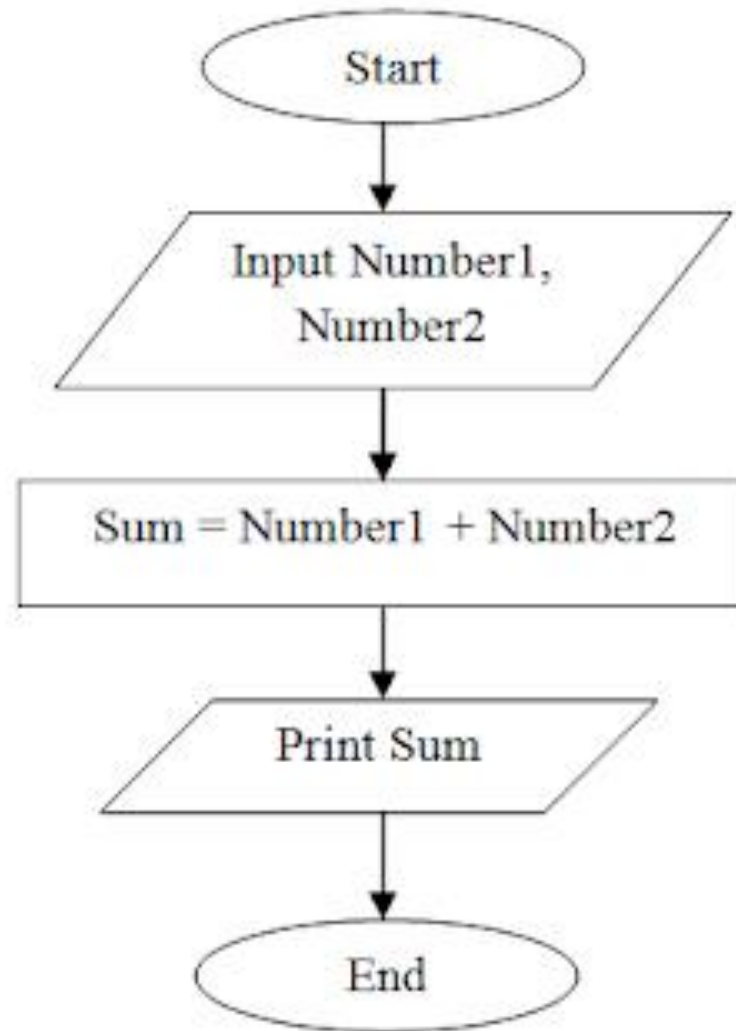
Step 2: If no is greater than equal zero, go to step 4.

Step 3: Print no is a negative number, go to step 5.

Step 4: Print no is a positive number.

Step 5: Stop.

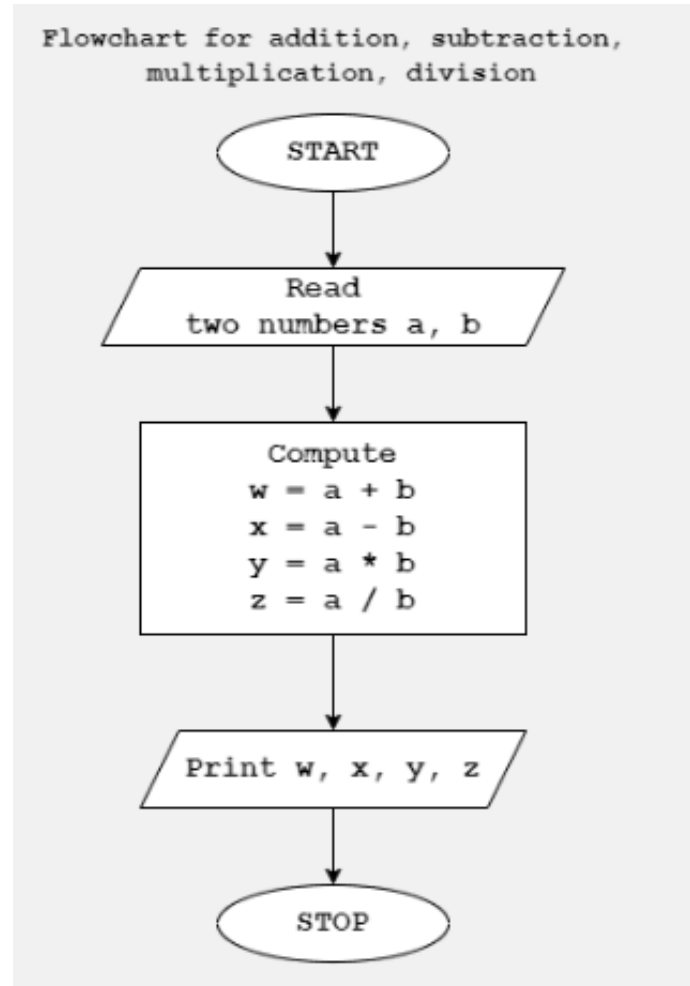
Example: addition of two input numbers



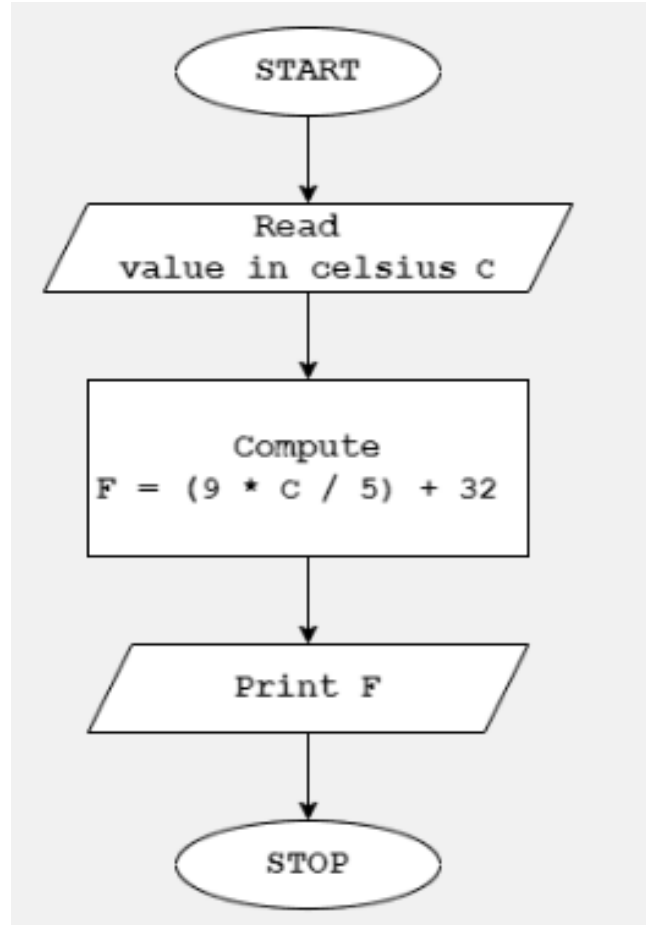
Example: addition of two input numbers

- Step 1: Start
- Step 2: Read values for num1, num2.
- Step 3: Add num1 and num2 and assign the result to a variable sum.
- Step 4: Display sum
- Step 5: Stop

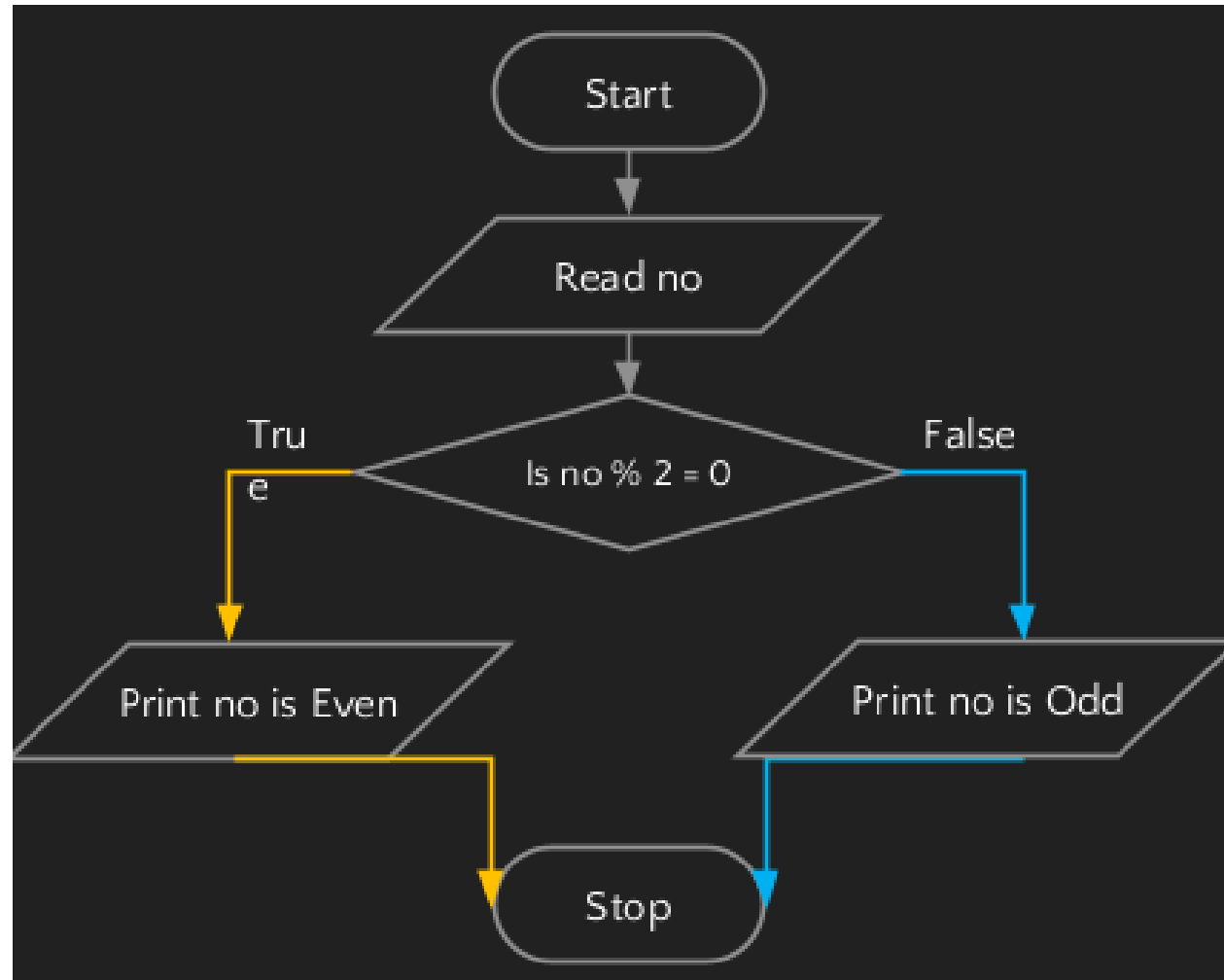
Example: multiplication, subtraction, addition of two input numbers



Example: convert Celsius into Fahrenheit . ($F = 1.8 * C + 32$)



Example: checks if an input number is odd or even



Example: checks if an input number is odd or even

- Step 1: Read no.
- Step 2: If $\text{no} \bmod 2 = 0$, go to step 4.
- Step 3: Print no is a odd, go to step 5.
- Step 4: Print no is a even.
- Step 5: Stop.

C program structure

Documentation section (Used for comments)
Link section
Definition section
Global declaration section (Variables used in more than one functions)
<pre>void main () { Declaration part Executable part }</pre>
Subprogram section (User defined functions)

```
// Documentation
/**
 * file: sum.c
 * author: you
 * description: program to find sum.
 */
// Link
#include <stdio.h>
// Definition
#define X 20

// Global Declaration
int sum(int y);

// Main() Function
int main(void)
{
    int y = 55;
    printf("Sum: %d", sum(y));
    return 0;
}
// Subprogram
int sum(int y)
{
    return y + X;
}
```

C program syntax rules

1. Statements end with semicolon (;)

- A semicolon ; is used to mark the end of a statement and the beginning of another statement in the C language.
- The absence of a semicolon at the end of any statement will mislead the compiler to think that this statement is not yet finished.
- And it will add the next consecutive statement after it, which may lead to a **compilation(syntax) error**.

C program syntax rules

2. Adding Comments to Code

1. Using `//`: This is used to write a **single-line comment**.
2. Using `/* */`: Anything enclosed within `/*` and `*/` , will treated as **multi-line comments**.

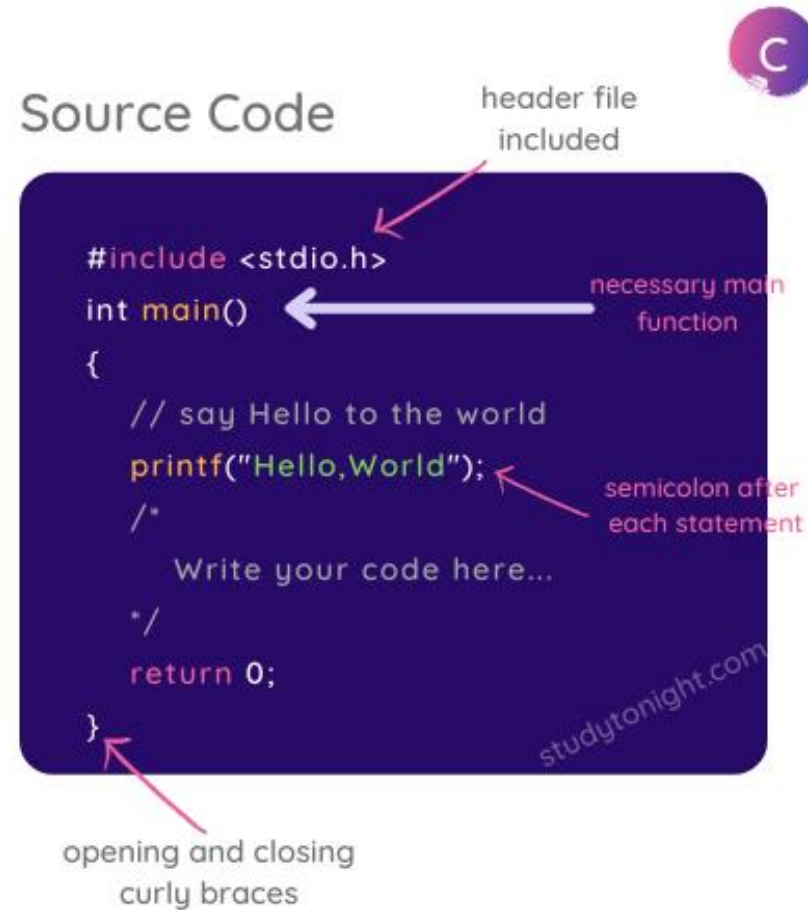
3. More Syntax rules for the C Language

C program syntax rules

3. More Syntax rules for the C Language

- C is a **case-sensitive language** so all C instructions must be written in lower case letters. **main** is not the same as **MAIN**.
- All C statements must end with a **semicolon**.
- **Whitespace** is used in C to add blank space and tabs.
- You do not have to worry about the indentation of the code.
- When we write a function, its body is enclosed in **curly braces**, like for the **main()** function.

C program syntax rules



Compiler, Interpreter and Assembler

- Compiler translates program of higher level language to machine language. It converts whole program at a time.
 - C, C++, C#, etc are programming languages that are compiler-based
- Interpreter translates program of higher level language to machine language. It converts program line by line.
 - Python, Ruby, Perl, SNOBOL, MATLAB, etc are programming languages that are interpreter-based.
- Assembler translates program of assembly language to machine language.

What is a Compilation?

- Before diving into the traditional definition of compilation, let us consider an example where there is a person A who speaks Hindi language and person A wants to talk to person B who only knows English language, so now either of them requires a *translator* to translate their words to communicate with each other.
- This process is known as *translation*, or in terms of programming, it is known as **compilation** process.
- The compilation process in C is converting an understandable human code into a Machine understandable code and checking the syntax and semantics of the code to determine any syntax errors or warnings present in our C program.
- Suppose we want to execute our C Program written in an IDE (Integrated Development Environment).
- In that case, it has to go through several phases of compilation (translation) to become an executable file that a machine can understand.

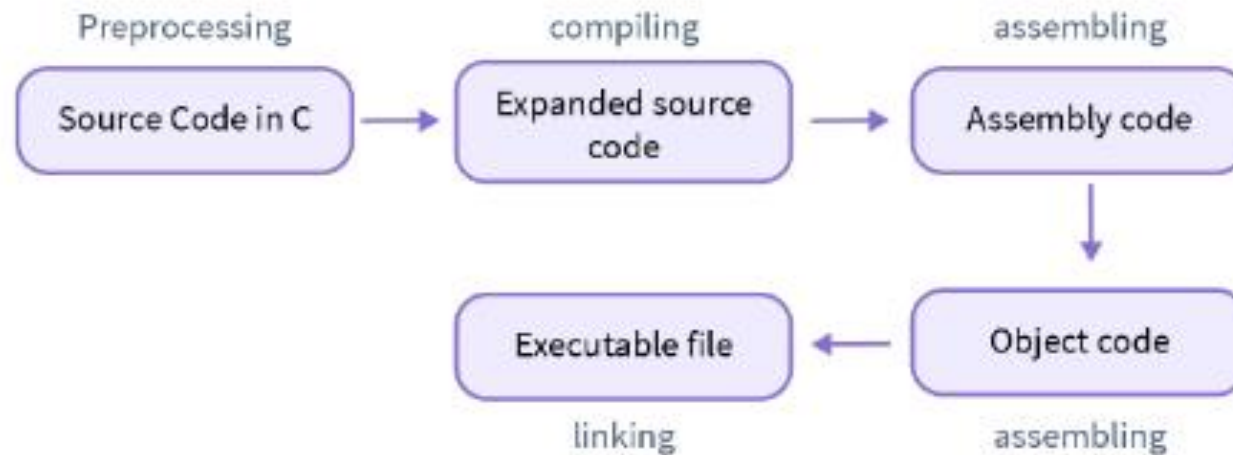
What is a Compilation?



What is a Compilation?

Compilation process in C involves four steps:

1. Preprocessing
2. Compiling
3. Assembling
4. Linking



What is a Compilation?

Compilation process in C involves four steps:

1. Preprocessing

- Pre-processing is the first step in the compilation process in C performed using the *pre-processor tool* (A pre-written program invoked by the system during the compilation).
- All the statements starting with the `#` symbol in a C program are processed by the pre-processor, and it converts our program file into an intermediate file with no `#` statements.
- Under following pre-processing tasks are performed :

a. Comments Removal

b. Macros Expansion (`#define` G 9.8, `#define SUM(a,b) (a+b)`)

- Macros are some constant values or expressions defined using the **`#define`** directives in C Language.
- A macro call leads to the macro expansion.
- The pre-processor creates an intermediate file where some pre-written assembly level instructions replace the defined expressions or constants (basically matching tokens).
- To differentiate between the original instructions and the assembly instructions resulting from the macros expansion, a '+' sign is added to every macros expanded statement.

What is a Compilation?

Compilation process in C involves four steps:

1. Preprocessing

C. File inclusion

- File inclusion in C language is the addition of another *file* containing some pre-written code into our C Program during the pre-processing. It is done using the **#include** directive.
- File inclusion during pre-processing causes the entire content of **filename** to be added to the source code, replacing the **#include<filename>** directive, creating a new intermediate file.
- **Example:** If we have to use basic input/output functions like `printf()` and `scanf()` in our C program, we have to include a pre-defined **standard input output header file** i.e. **stdio.h**.

What is a Compilation?

Compilation process in C involves four steps:

1. Preprocessing

iv. Conditional Compilation

- Conditional compilation is running or avoiding a block of code after checking if a **macro** is defined or not (a constant value or an expression defined using `#define`).
- The preprocessor replaces all the conditional compilation directives with some pre-defined assembly code and passes a newly expanded file to the compiler.
- Conditional compilation can be performed using commands like `#ifdef`, `#endif`, `#ifndef`, `#if`, `#else` and `#elif` in a C Program.
- Example :
- Printing the AGE macro, if AGE macro is defined, else printing Not Defined and ending the conditional compilation block with an `#endif` directive.

What is a Compilation?

Compilation process in C involves four steps:

1. Preprocessing

iv. Conditional Compilation

```
#include <stdio.h>

// if we uncomment the below line, then the program will print
// #define AGE 18

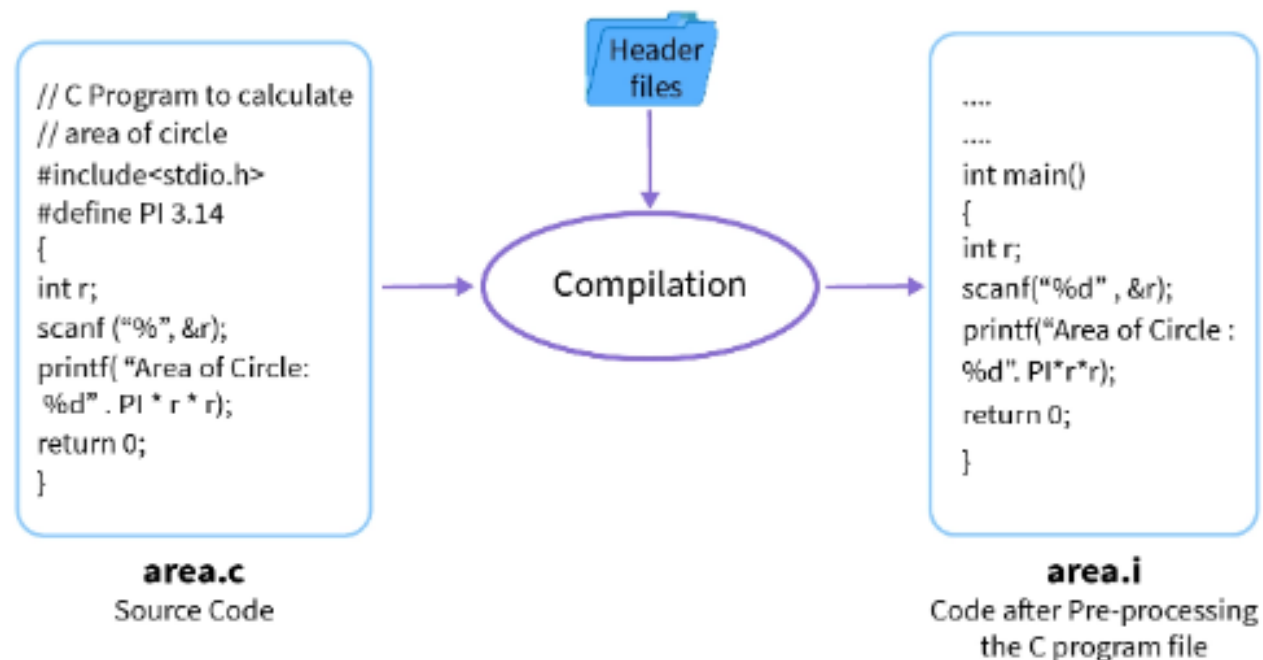
int main()
{
    // if `AGE` is defined then print the `AGE` else print
    #ifdef AGE
        printf("Age is %d", AGE);
    #else
        printf("Not Defined");
    #endif

    return 0;
}
```


What is a Compilation?

Compilation process in C involves four steps:

Now let's see the below figure that shows how a pre-processor converts our source code file into an intermediate file. **Intermediate file** has an extension of .i, and it is the expanded form of our C program containing all the content of header files, macros expansion, and conditional compilation.



What is a Compilation?

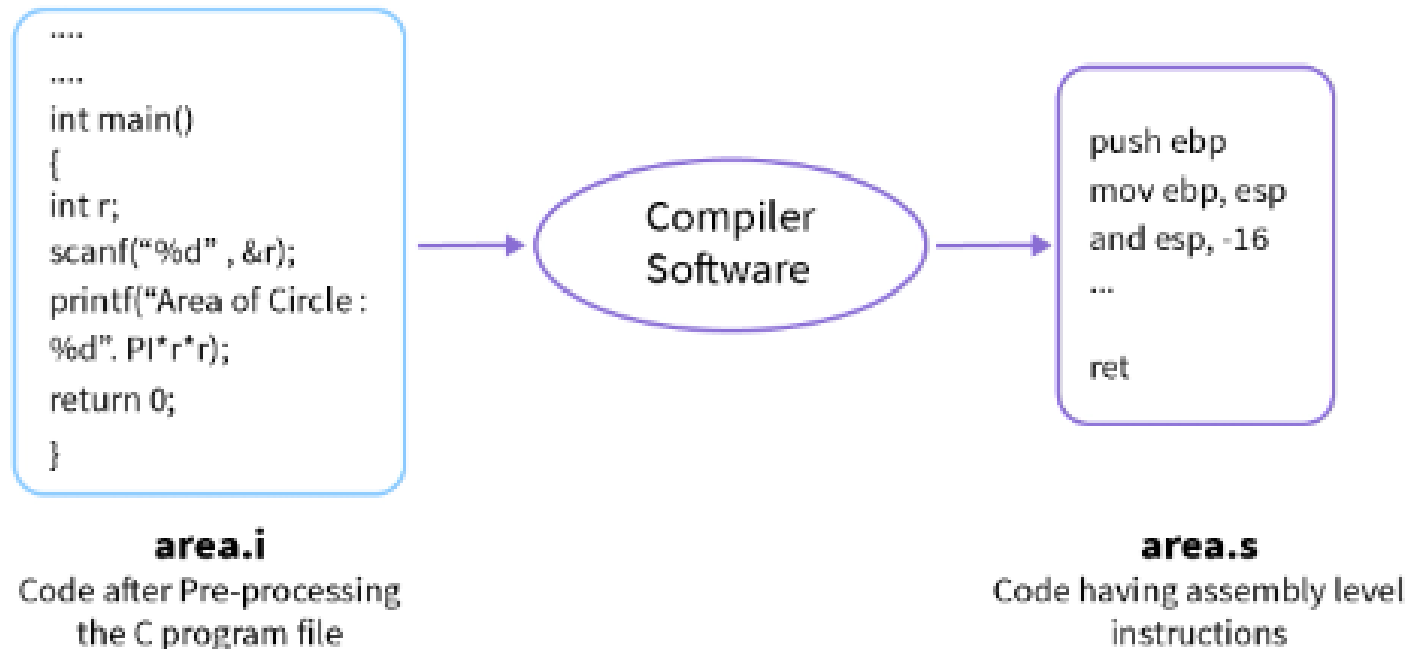
2. Compiling

- Compiling phase in C uses an inbuilt *compiler software* to convert the intermediate (.i) file into an **Assembly file** (.s) having assembly level instructions (low-level code).
- To boost the performance of the program C compiler translates the intermediate file to make an assembly file.
- Assembly code is a simple English-type language used to write low-level instructions (in micro-controller programs, we use assembly language).
- The whole program code is parsed (syntax analysis) by the compiler software in one go, and it tells us about any **syntax errors** or **warnings** present in the source code through the terminal window.

What is a Compilation?

2. Compiling

The below image shows an example of how the compiling phase works.



What is a Compilation?

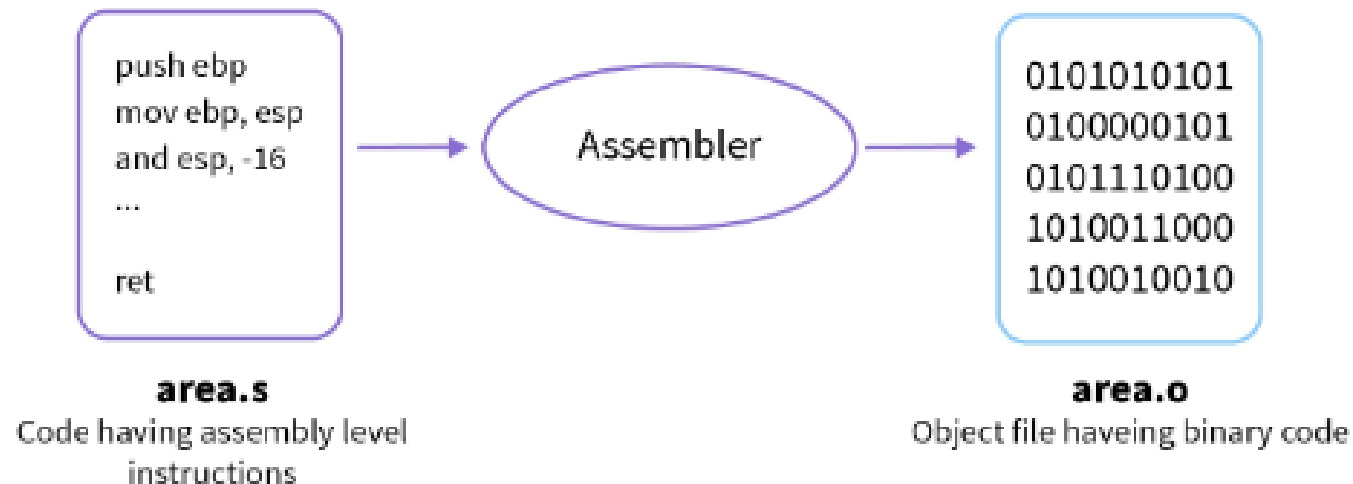
3. Assembling

- Assembly level code (.s file) is converted into a machine-understandable code (in binary/hexadecimal form) using an *assembler*.
- Assembler is a pre-written program that translates assembly code into machine code. It takes basic instructions from an assembly code file and converts them into binary/hexadecimal code specific to the machine type known as the object code.
- The file generated has the same name as the assembly file and is known as an **object file** with an extension of **.obj** in DOS and **.o** in UNIX OS.

What is a Compilation?

3. Assembling

The below image shows an example of how the assembly phase works. An assembly file **area.s** is translated to an object file **area.o** having the same name but a different extension.



What is a Compilation?

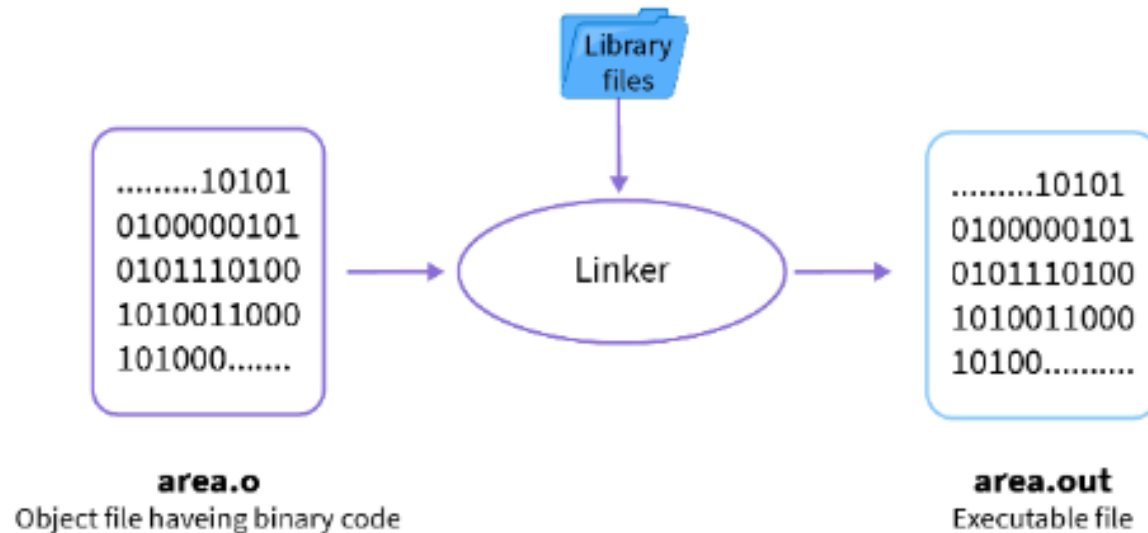
3. Linking

- Linking is a process of including the library files into our program.
- *Library Files* are some predefined files that contain the definition of the functions in the machine language and these files have an extension of .lib.
- Some unknown statements are written in the object (.o/.obj) file that our operating system can't understand.
- You can understand this as a book having some words that you don't know, and you will use a dictionary to find the meaning of those words.
- Similarly, we use *Library Files* to give meaning to some unknown statements from our object file. The linking process generates an **executable file** with an extension of **.exe** in DOS and **.out** in UNIX OS.

What is a Compilation?

3. Linking

The below image shows an example of how the linking phase works, and we have an object file having machine-level code, it is passed through the linker which links the library files with the object file to generate an executable file.



What is C?

- C is a general-purpose programming language created by Dennis Ritchie at the Bell Laboratories in 1972.
- It is a very popular language, despite being old.
- C is strongly associated with UNIX, as it was developed to write the UNIX operating system.

The main features of the C language include:

General Purpose and Portable

Memory management

Fast Speed

Clean Syntax

- These features make the C language suitable for system programming like an operating system or compiler development.
- Many later languages have borrowed syntax/features directly or indirectly from the C language. Like syntax of Java, PHP, JavaScript, and many other languages are mainly based on the C language.

What is procedural language ?

- A procedural language is a sort of computer programming language that has a set of functions, instructions, and statements that must be executed in a certain order to accomplish a job or program.
- In general, procedural language is used to specify the steps that the computer takes to solve a problem.
- In procedural programming, we break down a task into variables and routines.
- Procedural programming is a programming language derived from structure programming and based on the concept of calling procedure. The procedures are the functions, routines, or subroutines that consist series of steps to be carried out.
- Computer procedural languages include BASIC, C, FORTRAN, Java, and Pascal, to name a few.

Linker Vs. Loader: Explore the difference between Linker and Loader

- When it comes to the execution of any programs, linker and loader play a very important role.
- They are the utility programs that support a program while executing. The purpose of a linker is to produce executable files whereas the major aim of a loader is to load executable files to the memory.
- A linker is an important utility program that takes the object files, produced by the assembler and compiler, and other code to join them into a single executable file. There are two types of linkers, dynamic and linkage.
- In the world of computer science, a loader is a vital component of an operating system that is accountable for loading programs and libraries. Absolute, Direct Linking, Bootstrap and Relocating are the types of loaders.

Difference between Linker and Loader

S.No.	Linker	Loader
1	A linker is an important utility program that takes the object files, produced by the assembler and compiler, and other code to join them into a single executable file.	A loader is a vital component of an operating system that is accountable for loading programs and libraries.
2	It uses an input of object code produced by the assembler and compiler.	It uses an input of executable files produced by the linker.
3	The foremost purpose of a linker is to produce executable files.	The foremost purpose of a loader is to load executable files to memory.
4	Linker is used to join all the modules.	Loader is used to allocate the address to executable files.
5	It is accountable for managing objects in the program's space.	It is accountable for setting up references that are utilized in the program.

Useful Escape Sequences

Escape sequence	Description	Example	Output
<code>\n</code>	New line	<code>printf("Hello \n World");</code>	Hello World
<code>\t</code>	Horizontal tab	<code>printf("Hello \t World");</code>	Hello World
<code>\'</code>	Single quote	<code>printf("Hello \'World\' ");</code>	Hello 'World'
<code>\"</code>	Double quote	<code>printf("Hello \"World\" ");</code>	Hello "World"
<code>\\</code>	Backslash	<code>printf("Hello \\World");</code>	Hello \World

Header files in C

- A header file is a file with extension .h which contains the set of predefined standard library functions.
- The “#include” preprocessing directive is used to include the header files with extension in the program.

Header file	Description
stdio.h	Input/Output functions (printf and scanf)
conio.h	Console Input/Output functions (getch and clrscr)
math.h	Mathematics functions (pow, exp, sqrt etc...)
string.h	String functions (strlen, strcmp, strcat etc...)

Variables

- Variable is a symbolic name given to some value which can be changed.
- x , y , a , *count*, *etc.* can be variable names.
- **Rules for Naming a Variable in C**
- We give a variable a meaningful name when we create it. Here are the rules that we must follow when naming it:
 1. The name of the variable must not begin with a digit.
 2. A variable name can consist of digits, alphabets, and even special symbols such as an underscore (`_`).
 3. A variable name must not have any keywords, for instance, `float`, `int`, *etc.*
 4. There must be no spaces or blanks in the variable name.
 5. The C language treats lowercase and uppercase very differently, as it is case sensitive. Usually, we keep the name of the variable in the lower case.

Variables

- *Let us look at some of the examples,*
- `int var1; //` it is correct
- `int 1var; //` it is incorrect – the name of the variable should not start using a number
- `int my_var1; //` it is correct
- `int my$var //` it is incorrect – no special characters should be in the name of the variable
- `char else; //` there must be no keywords in the name of the variable
- `int my var; //` it is incorrect – there must be no spaces in the name of the variable
- `int COUNT; //` it is a new variable
- `int Count; //` it is a new variable
- `int count; //` it is a valid variable name

Variables

- Declaring a variable provides the compiler with an assurance that there is a variable that exists with that very given name.
- *type variableName = value; //* exa : int no=5;
- This way, the compiler will get a signal to proceed with the further compilation without needing the complete details regarding that variable.

In C, there are different **types** of variables (defined with different keywords), for example:

- **int** - stores integers (whole numbers), without decimals, such as **123** or **-123**
- **float** - stores floating point numbers, with decimals, such as **19.99** or **-19.99**
- **char** - stores single characters, such as **'a'** or **'B'**. Char values are surrounded by **single quotes**

Format Specifiers

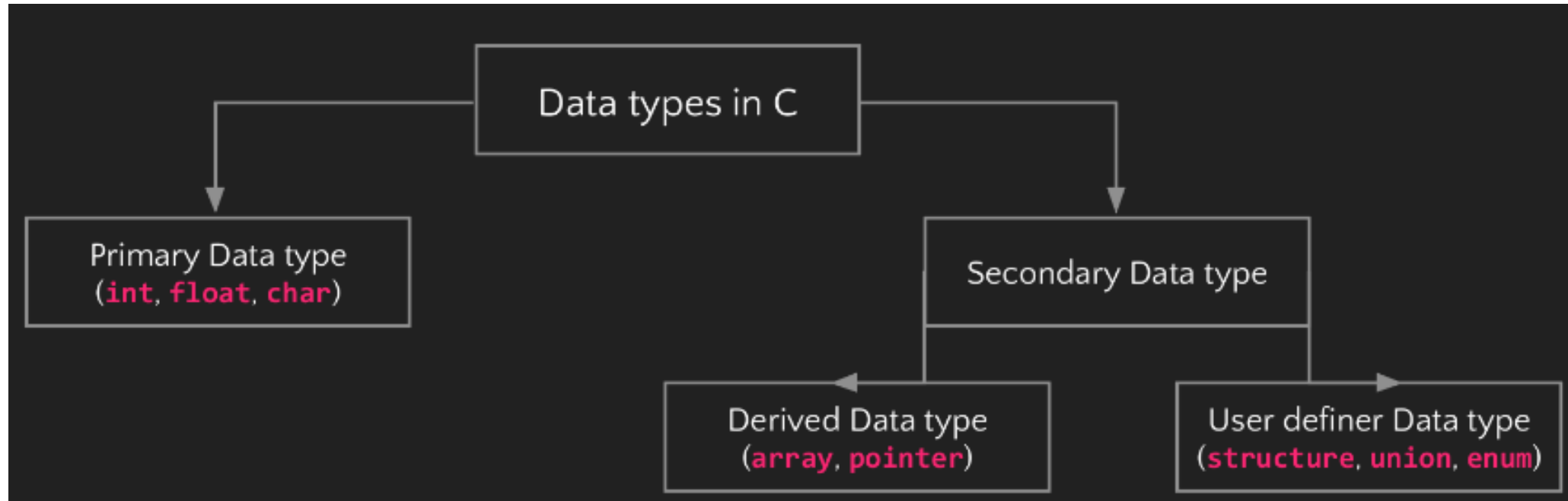
- Format specifiers are used together with the `printf()` function to tell the compiler
- what type of data the variable is storing. It is basically a placeholder for the variable value.
- A format specifier starts with a percentage sign `%`, followed by a character.
- For example, to output the value of an `int` variable,
- you must use the format specifier `%d` or `%i` surrounded by double quotes, inside the `printf()` function:

- ```
int myNum = 15;
printf("%d", myNum); // Outputs 15
```

To print other types, use `%c` for `char` and `%f` for `float`:

# Data types in C

- Data types are defined as the data storage format that a variable can store a data.
- It determines the type and size of data associated with variables.



# Primary Data Type

- Primary data types are built in data types which are directly supported by machine.
- They are also known as fundamental data types.
- **int:**
- int datatype can store integer number which is whole number without fraction part such as 10, 105 etc.
- Example: `int a=10;`

# Primary Data Type

- **int:**
- An integer type variable can store zero, positive, and negative values without any decimal.
- In C language, the integer data type is represented by the '**int**' keyword, and it can be both signed or unsigned.
- By default, the value assigned to an integer variable is considered positive if it is unsigned.
- The integer data type is further divided into **short**, **int**, and **long** data types. The short data type takes 2 bytes of storage space; int takes 2 or 4 bytes, and long takes 8 bytes in 64-bit and 4 bytes in the 32-bit operating system.

# Primary Data Type

- **int:**
- If you try to assign a decimal value to the integer variable, the value after the decimal will be truncated, and only the whole number gets assigned to the variable.

# Primary Data Type

- **int:**
- The memory size of these data types can change depending on the operating system (32-bit or 64-bit).
- Here is the table showing the data types commonly used in **C programming** with their storage size and value range, according to the 32-bit architecture.

| Type                           | Storage Size | Value Range       |
|--------------------------------|--------------|-------------------|
| Int (or signed int)            | 2 bytes      | -32,768 to 32,767 |
| unsigned int                   | 2 bytes      | 0 to 65,535       |
| Short int(or signed short int) | 2 bytes      | -32,768 to 32,767 |

# Primary Data Type

- **int:**

|                           |          |                                            |
|---------------------------|----------|--------------------------------------------|
| Long(or signed short int) | 4 bytes  | -2,147,483,648 to 2,147,483,647            |
| unsigned long             | 4 bytes  | 0 to 4,294,967,295                         |
| float                     | 4 bytes  | 1.2E-38 to 3.4E+38 (6 decimal places)      |
| double                    | 8 bytes  | 2.3E-308 to 1.7E+308 (15 decimal places)   |
| Long double               | 10 bytes | 3.4E-4932 to 1.1E+4932 (19 decimal places) |
| char(or signed char)      | 1 byte   | -128 to 127                                |
| unsigned char             | 1 byte   | 0 to 255                                   |

# Primary Data Type

- float:
- The floating-point data type allows a user to store decimal values in a variable. It is of two types:
- Float
- Double
- The float data type can hold approximately 7 digits. The storage size of the float variable is 4 bytes, but the size may vary for different processors, the same as the 'int' data type.
- In C language, the **float** values are represented by the '%f' format specifier. A variable containing integer value will also be printed in the floating type with redundant zeroes.



# Primary Data Type

- float:

- **Double**

The double data type is similar to float, but here, the double data type can hold approximately 15-17 digits. It is represented by the 'double' keyword and is mainly used in scientific programs where extreme precision is required.

- Exa: `double average=3499999999.454;`

```
printf("Average is %lf",average);
```

# Primary Data Type

- **char:**
  - Char data type can store single character of alphabet or digit or special symbol such as 'a', '5' etc.
  - Each character is assigned some integer value which is known as ASCII values.
  - Example: `char a='a';`
- **void:**
  - The void type has no value therefore we cannot declare it as variable as we did in case of int or float or char.
  - The void data type is used to indicate that function is not returning anything.

# Primary Data Type

| Data Type | Size         | Description                                                                                           |
|-----------|--------------|-------------------------------------------------------------------------------------------------------|
| int       | 2 or 4 bytes | Stores whole numbers, without decimals                                                                |
| float     | 4 bytes      | Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits |
| double    | 8 bytes      | Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits  |
| char      | 1 byte       | Stores a single character/letter/number, or ASCII values                                              |

# Secondary Data Type

- Secondary data types are not directly supported by the machine.
- It is combination of primary data types to handle real life data in more convenient way.
- It can be further divided in two categories,
- **Derived data types:** Derived data type is extension of primary data type. It is built-in system and its structure cannot be changed. Examples: **Array** and **Pointer**.
- Array: An array is a fixed-size sequenced collection of elements of the same data type.
- Pointer: Pointer is a special variable which contains memory address of another variable.
- **User defined data types:** User defined data type can be created by programmer using combination of primary data type and/or derived data type. Examples: **Structure, Union, Enum**.
- Structure: Structure is a collection of logically related data items of different data types grouped together under a single name.
- Union: Union is like a structure, except that each element shares the common memory.
- Enum: Enum is used to assign names to integral constants, the names make a program easy to read and maintain.

# printf()

- printf() is a function defined in stdio.h file
- It displays output on standard output, mostly monitor
- Message and value of variable can be printed
- Let's see few examples of printf
- `printf(" ");`
- `printf("Hello World"); // Hello World`
- `printf("%d", c); // 15`
- `printf("Sum = %d", c); // Sum = 15`
- `printf("%d+%d=%d", a, b, c); // 10+5=15`

# scanf()

- scanf() is a function defined in stdio.h file
- scanf() function is used to read character, string, numeric data from keyboard
- Syntax of scanf
- scanf("%X", &variable);
- where %X is the format specifier which tells the compiler what type of data is in a variable.
- & refers to address of “variable” which is directing the input value to a address returned by &variable.

# scanf()

| Format specifier | Supported data types | Example         | Description                                      |
|------------------|----------------------|-----------------|--------------------------------------------------|
| %d               | Integer              | scanf("%d", &a) | Accept integer value such as 1, 5, 25, 105 etc   |
| %f               | Float                | scanf("%f", &b) | Accept floating value such as 1.5, 15.20 etc     |
| %c               | Character            | scanf("%c", &c) | Accept character value such as a, f, j, W, Z etc |
| %s               | String               | scanf("%s", &d) | Accept string value such as diet, india etc      |

# Arithmetic operators

| Operator | Meaning                    | Example  | Description               |
|----------|----------------------------|----------|---------------------------|
| +        | Addition                   | $a + b$  | Addition of a and b       |
| -        | Subtraction                | $a - b$  | Subtraction of b from a   |
| *        | Multiplication             | $a * b$  | Multiplication of a and b |
| /        | Division                   | $a / b$  | Division of a by b        |
| %        | Modulo division- remainder | $a \% b$ | Modulo of a by b          |



# Token

The smallest individual unit of a program is known as token.

C has the following tokens:

## Keywords

- C reserves a set of 32 words for its own use. These words are called keywords (or reserved words), and each of these keywords has a special meaning within the C language.

## Identifiers

- Identifiers are names that are given to various user defined program elements, such as variable, function and arrays.

## Constants

- Constants refer to fixed values that do not change during execution of program.

## Strings

- A string is a sequence of characters terminated with a null character \0.

## Special Symbols

- Symbols such as #, &, =, \* are used in C for some specific function are called as special symbols.

## Operators

- An operator is a symbol that tells the compiler to perform certain mathematical or logical operation.

# Relational Operators

- Relational operators are used to compare two numbers and taking decisions based on their relation.
- Relational expressions are used in decision statements such as if, for, while, etc...

| Operator | Meaning                     | Example    | Description                     |
|----------|-----------------------------|------------|---------------------------------|
| <        | Is less than                | $a < b$    | a is less than b                |
| <=       | Is less than or equal to    | $a \leq b$ | a is less than or equal to b    |
| >        | Is greater than             | $a > b$    | a is greater than b             |
| >=       | Is greater than or equal to | $a \geq b$ | a is greater than or equal to b |
| ==       | Is equal to                 | $a = b$    | a is equal to b                 |
| !=       | Is not equal to             | $a \neq b$ | a is not equal to b             |

# Logical Operators

- Logical operators are used to test more than one condition and make decisions.

| Operator | Meaning                                                          |
|----------|------------------------------------------------------------------|
| &&       | logical AND (Both non zero then true, either is zero then false) |
|          | logical OR (Both zero then false, either is non zero then true)  |

| a | b | a&&b | a    b |
|---|---|------|--------|
| 0 | 0 | 0    | 0      |
| 0 | 1 | 0    | 1      |
| 1 | 0 | 0    | 1      |
| 1 | 1 | 1    | 1      |

# Assignment Operators

- Assignment operators (=) is used to assign the result of an expression to a variable.
- Assignment operator stores a value in memory.
- C also supports shorthand assignment operators which simplify operation with assignment.

| Operator | Meaning                                  |
|----------|------------------------------------------|
| =        | Assigns value of right side to left side |
| +=       | a += 1 is same as a = a + 1              |
| -=       | a -= 1 is same as a = a - 1              |
| *=       | a *= 1 is same as a = a * 1              |
| /=       | a /= 1 is same as a = a / 1              |
| %=       | a %= 1 is same as a = a % 1              |

# Increment and Decrement Operators

- Increment (++) operator used to increase the value of the variable by one.
- Decrement (--) operator used to decrease the value of the variable by one.
- Ex: `x=100; x++;` // After the execution the value of x will be 101.
- Ex: `x=100; x--;` // After the execution the value of x will be 99.

# Increment and Decrement Operators

| Operator                                       | Description                                                                                                                         |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Pre increment operator (++x)                   | value of x is incremented before assigning it to the variable on the left                                                           |
| <div>Example</div> <pre>x=10;<br/>p=++x;</pre> | <div>Explanation</div> <p>First increment value of x by one then assign.</p> <div>Output</div> <p>x will be 11<br/>p will be 11</p> |
| Operator                                       | Description                                                                                                                         |
| Post increment operator (x++)                  | value of x is incremented after assigning it to the variable on the left                                                            |
| <div>Example</div> <pre>x=10;<br/>p=x++;</pre> | <div>Explanation</div> <p>First assign value of x then increment value.</p> <div>Output</div> <p>x will be 11<br/>p will be 10</p>  |

# Conditional Operators

- A ternary operator is known as conditional operator.
- Syntax: `exp1 ? exp2 : exp3`

Explanation:

`exp1` is evaluated first

if `exp1` is true(nonzero) then

`exp2` is evaluated and its value becomes the value of the expression

if `exp1` is false(zero) then

`exp3` is evaluated and its value becomes the value of the expression

Example

```
m=2, n=3;
r=(m>n) ? m : n;
```

Explanation

Value of `r` will be 3

# Bitwise Operators

- Bitwise operators are used to perform operation bit by bit.
- Bitwise operators may not be applied to float or double.

| Operator | Meaning                                     |
|----------|---------------------------------------------|
| &        | bitwise AND                                 |
|          | bitwise OR                                  |
| ^        | bitwise exclusive OR                        |
| <<       | shift left (shift left means multiply by 2) |
| >>       | shift right (shift right means divide by 2) |



# Bitwise Operators

The result of bitwise XOR operator is **1** if the corresponding bits of two operands are opposite. It is denoted by  $\oplus$ .

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bitwise XOR Operation of 12 and 25

00001100

$\wedge$  00011001

00010101 = 21 (In decimal)

```
#include <stdio.h>

int main() {

 int a = 12, b = 25;
 printf("Output = %d", a ^ b);

 return 0;
}
```

Output

Output = 21

# Bitwise Operators

8 = 1000 (In Binary) and 6 = 0110 (In Binary)

Example: Bitwise & (AND)

```
int a=8, b=6, c;
c = a & b;
printf("Output = %d", c);
```

Output

0

Example: Bitwise | (OR)

```
int a=8, b=6, c;
c = a | b;
printf("Output = %d", c);
```

Output

14

Example: Bitwise << (Shift Left)

```
int a=8, b;
b = a << 1;
printf("Output = %d", b);
```

Output

16 (multiplying a by a power of two)

Example: Bitwise >> (Shift Right)

```
int a=8, b;
b = a >> 1;
printf("Output = %d", b);
```

Output

4 (dividing a by a power of two)

# Special Operators

| Operator | Meaning                                                                            |
|----------|------------------------------------------------------------------------------------|
| &        | Address operator, it is used to determine address of the variable.                 |
| *        | Pointer operator, it is used to declare pointer variable and to get value from it. |
| ,        | Comma operator. It is used to link the related expressions together.               |
| sizeof   | It returns the number of bytes the operand occupies.                               |
| .        | member selection operator, used in structure.                                      |
| ->       | member selection operator, used in pointer to structure.                           |

# Expressions

- An expression is a combination of operators, constants and variables.
- An expression may consist of one or more operands, and zero or more operators to produce a value.
- Example:  $\text{final} = a + b * c$
- A, b and c are operands
- + and \* are operators
- Final stores the result of the expression.

# Type conversion

## Types of type conversion

There are two types of type conversion in the C language.

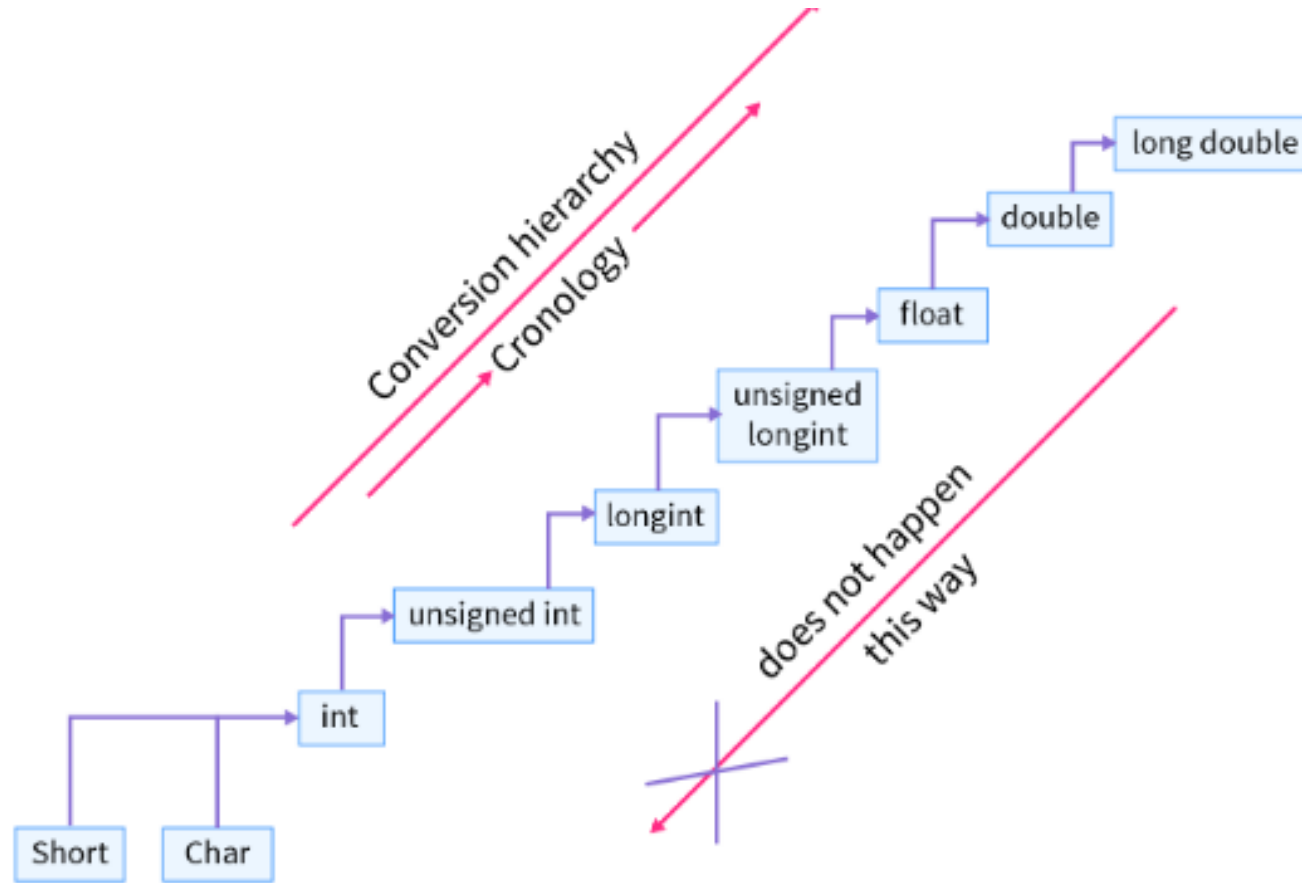
1. Implicit type conversion.
2. Explicit type conversion.

# Implicit type conversion

The implicit type conversion takes place when more than one data type is present in an expression. It is done by the compiler itself it is also called automatic type conversion. Here the automatic type conversion takes place in order to prevent data loss, as the datatypes are upgraded to the variable with datatype having the largest value.

**Eg.** If we add one integer and float so one of them has to become float because there is a conversion hierarchy according to which conversion happens.

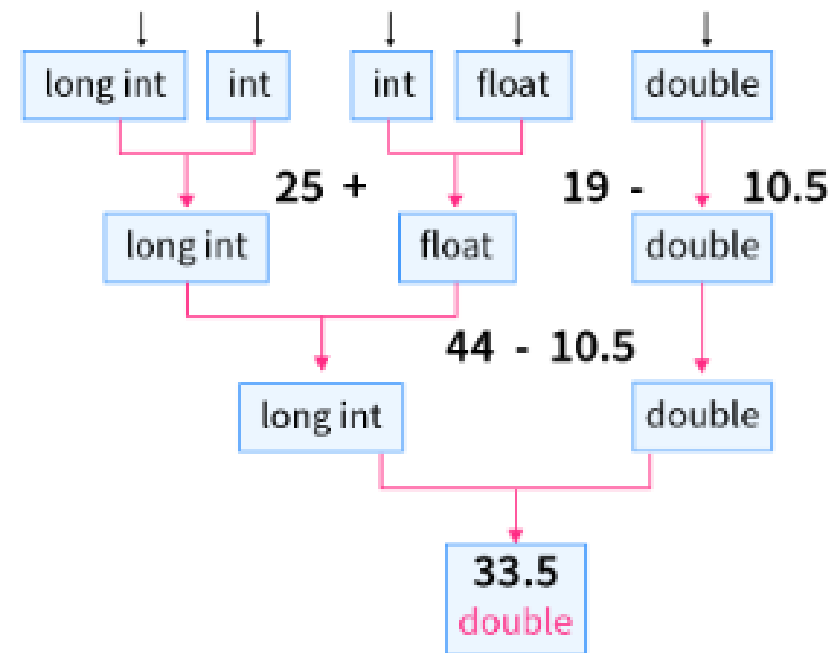
# Implicit type conversion



# Implicit type conversion

$a = z / b + b * x - y$

$a = 50 / 2 + 2 * 9.5 - 10.5$





# Explicit type conversion

## 2. Explicit Type Conversion

Let's start with an example. If we perform an arithmetic operation on two same types of datatype variables so the output will be in the same data type. But there are some operations like the division that can give us output in float or double.

Eg.

```
a = 3; (int)
b = 20; (int)
c = b/a = 6
```



Here the expected output was 6.66 but **a** and **b** were integers so the output came as 6 integer. But if we need 6.66 as output we need explicit type conversion.

# Explicit type conversion

Explicit type conversion refers to the type conversion performed by a programmer by modifying the data type of an expression using the type cast operator.

The explicit type conversion is also called type casting in other languages. It is done by the programmer, unlike implicit type conversion which is done by the compiler.

Syntax:

```
(datatype) expression
```



# If else statement

## C if Statement

The syntax of the `if` statement in C programming is:

```
if (test expression)
{
 // code
}
```

Expression is true.

```
int test = 5;
```

```
if (test < 10)
{
 // codes
}
```

```
// codes after if
```

Expression is false.

```
int test = 5;
```

```
if (test > 10)
{
 // codes
}
```

```
// codes after if
```

Working of if Statement

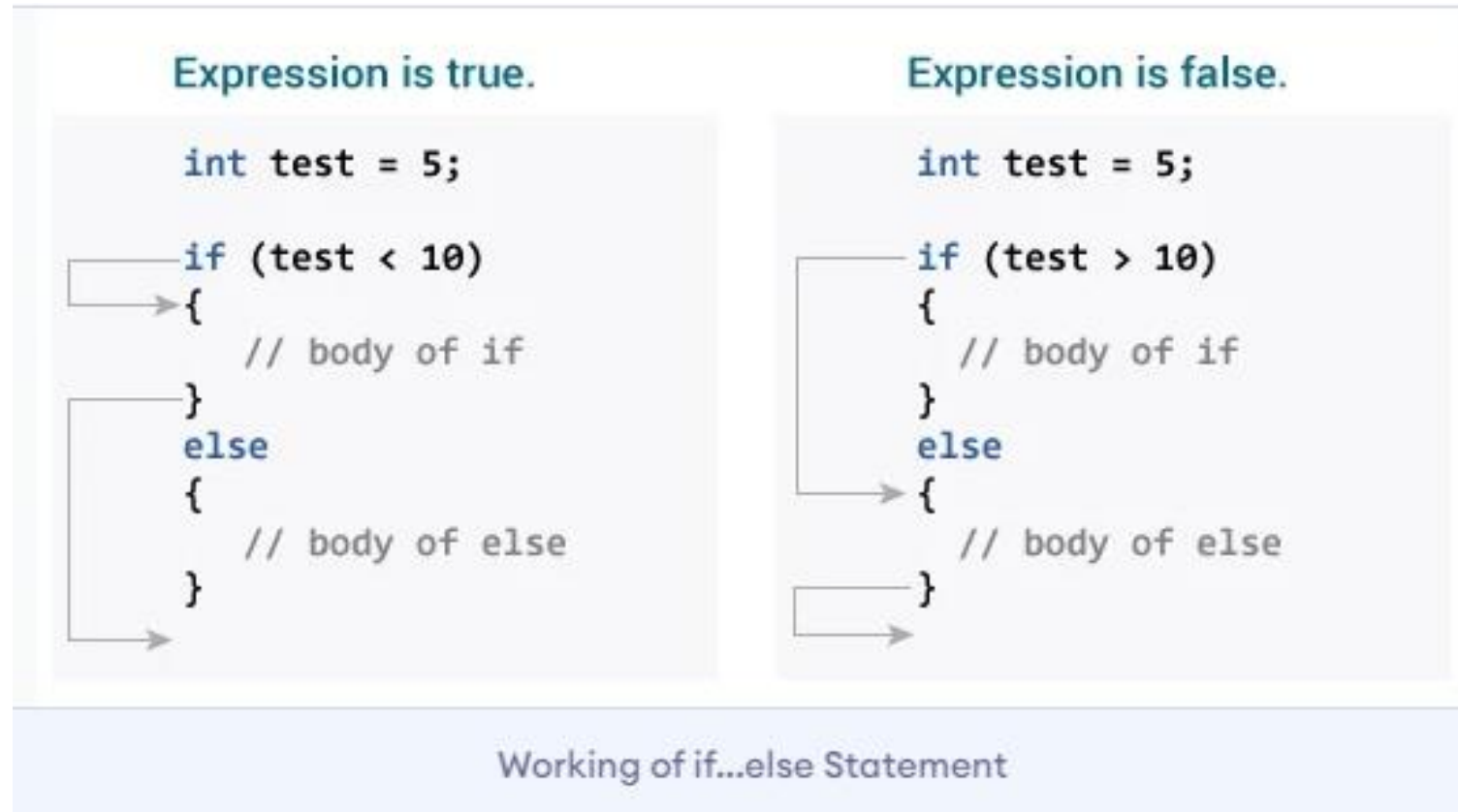
# If else statement

## C if...else Statement

The `if` statement may have an optional `else` block. The syntax of the `if..else` statement is:

```
if (test expression) {
 // run code if test expression is true
}
else {
 // run code if test expression is false
}
```

# If else statement



# The else if Statement

Use the **else if** statement to specify a new condition if the first condition is **false**.

## Syntax

```
if (condition1) {
 // block of code to be executed if condition1 is true
} else if (condition2) {
 // block of code to be executed if the condition1 is false and condition2 is true
} else {
 // block of code to be executed if the condition1 is false and condition2 is false
}
```

# The else if Statement

## Example

```
int time = 22;
if (time < 10) {
 printf("Good morning.");
} else if (time < 20) {
 printf("Good day.");
} else {
 printf("Good evening.");
}
// Outputs "Good evening."
```

## Example

```
int myNum = 10; // Is this a positive or negative number?

if (myNum > 0) {
 printf("The value is a positive number.");
} else if (myNum < 0) {
 printf("The value is a negative number.");
} else {
 printf("The value is 0.");
}
```

# Nested if Statement

## Syntax

The syntax for a **nested if** statement is as follows –

```
if(boolean_expression 1) {

 /* Executes when the boolean expression 1 is true */
 if(boolean_expression 2) {
 /* Executes when the boolean expression 2 is true */
 }
}
```



# Nested if Statement

```
int main () {

 /* local variable definition */
 int a = 100;
 int b = 200;

 /* check the boolean condition */
 if(a == 100) {

 /* if condition is true then check the following */
 if(b == 200) {
 /* if condition is true then print the following */
 printf("Value of a is 100 and b is 200\n");
 }
 }

 printf("Exact value of a is : %d\n", a);
 printf("Exact value of b is : %d\n", b);

 return 0;
}
```

# Dangling else problem

- The dangling else problem is syntactic ambiguity.
- It occurs when we use nested if. When there are multiple “if” statements, the “else” part doesn’t get a clear view with which “if” it should combine.
- Dangling else can lead to serious problems. It can lead to wrong interpretations by the compiler and ultimately lead to wrong results.

*For example:*

```
Initialize k=0 and o=0
if(ch>=3)
if(ch<=10)
k++;
else
o++;
```

- In this case, we don’t know when the variable “o” will get incremented.
- Either the first “if” condition might not get satisfied or the second “if” condition might not get satisfied.
- Even the first “if” condition gets satisfied, the second “if” condition might fail which can lead to the execution of the “else” part. Thus it leads to wrong results.

# Resolving Dangling-else Problem

we can resolve the dangling-else problems in programming languages by using braces and indentation.

*For example:*

```
if (condition) {
 if (condition 1) {
 if (condition 2) {}
 }
}
else {
}
```

In the above example, we are using braces and indentation so as to avoid confusion.

# Resolving Dangling-else Problem

we can also use *the “if – else if – else”* format so as to specifically indicate which “e/se” belongs to which “if”.

*For example:*

```
if(condition) {
}
else if(condition-1) {
}
else if(condition-2){
}
else{
}
```

# Compiler not reading the string after integer in C programming? How can we solve this problem?

## Solution

- you are entering space or enter button after integer input, So b is assigned that space/enter.
- When you enter an integer number and press enter to read next value, compiler stores null into the string's first char and string input terminates. Because scanf will terminate whenever it reads a null character.

A simple solution is to discard a single delimiting white space character after the numeric data is read. C provides a suppression conversion specifier that will read a data item of any type and discard it. Here are some examples:

```
scanf("%*c"); /* read and discard a character */
scanf("%*d"); /* read and discard an integer */
scanf("%d%*c", &n); /* read an integer and store it in n, */ /* then read and discard a character */
scanf("%*c%c", &ch); /* read and discard a character, */ /* and read another, store it in ch, */
```

# Compiler not reading the string after integer in C programming? How can we solve this problem?

```
int a;
char b;
clrscr();
printf("enter no");
scanf("%d",&a);
printf("enter ch");
//scanf("%c",&b);
//use single scanf
//scanf("%d%c",&a,&b);
//or use following lines, it will first discard
//the white space character after interger and then read character
//scanf("%*c");
//scanf("%c",&b);
printf("no is %d char is %c",a,b);
```

# L value & R value

Lvalue is also known as locator value which always refers to an object. Here the object is **named region of storage**. In simple words, an Lvalue is an identifier that has a type and storage class that yields some address location. In the case of Rvalue, they cannot as they simply have a value that can be assigned to an lvalue.

## **Examples of Lvalues:**

Variables, Arrays, pointer variables, structure or union e.t.c

## **Example of Rvalues:**

For all **constants**, the function returning a non-reference value is referred to as R-value.

Remember, R-values don't have any storage space associated with them.

# L value & R value example

```
#include <stdio.h>

int main(){
 int a;
 a = 10;
 printf("%d", a);
}
```

In the above example, **a** is an Lvalue because it has the valid type **int** and also a storage class “**auto**” associated with it. 10 is referred to as an Rvalue because it is an integer constant & no address/storage space is associated with it.



# C switch Statement

The switch statement allows us to execute one code block among many alternatives.

You can do the same thing with the `if...else..if` ladder. However, the syntax of the `switch` statement is much easier to read and write.

## Syntax of switch...case

```
switch (expression)
{
 case constant1:
 // statements
 break;

 case constant2:
 // statements
 break;
 .
 .
 .
 default:
 // default statements
}
```

# C switch Statement

## How does the switch statement work?

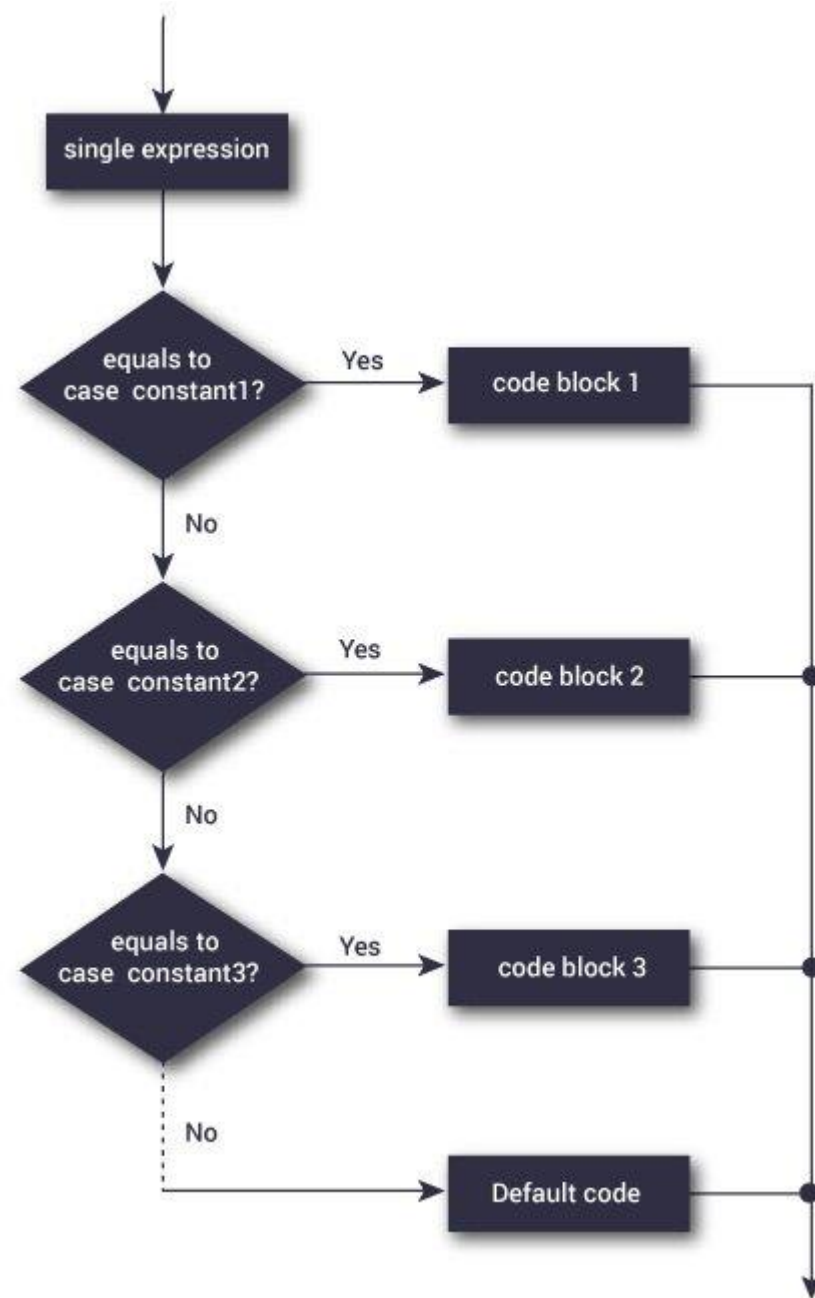
The `expression` is evaluated once and compared with the values of each `case` label.

- If there is a match, the corresponding statements after the matching label are executed. For example, if the value of the expression is equal to `constant2`, statements after `case constant2:` are executed until `break` is encountered.
- If there is no match, the default statements are executed.

### Notes:

- If we do not use the `break` statement, all statements after the matching label are also executed.
- The `default` clause inside the `switch` statement is optional.

# C switch Statement



# C while and do...while Loop

In programming, loops are used to repeat a block of code until a specified condition is met.

C programming has three types of loops.

- 1.for loop

- 2.while loop

- 3.do...while loop

## for Loop

The syntax of the `for` loop is:

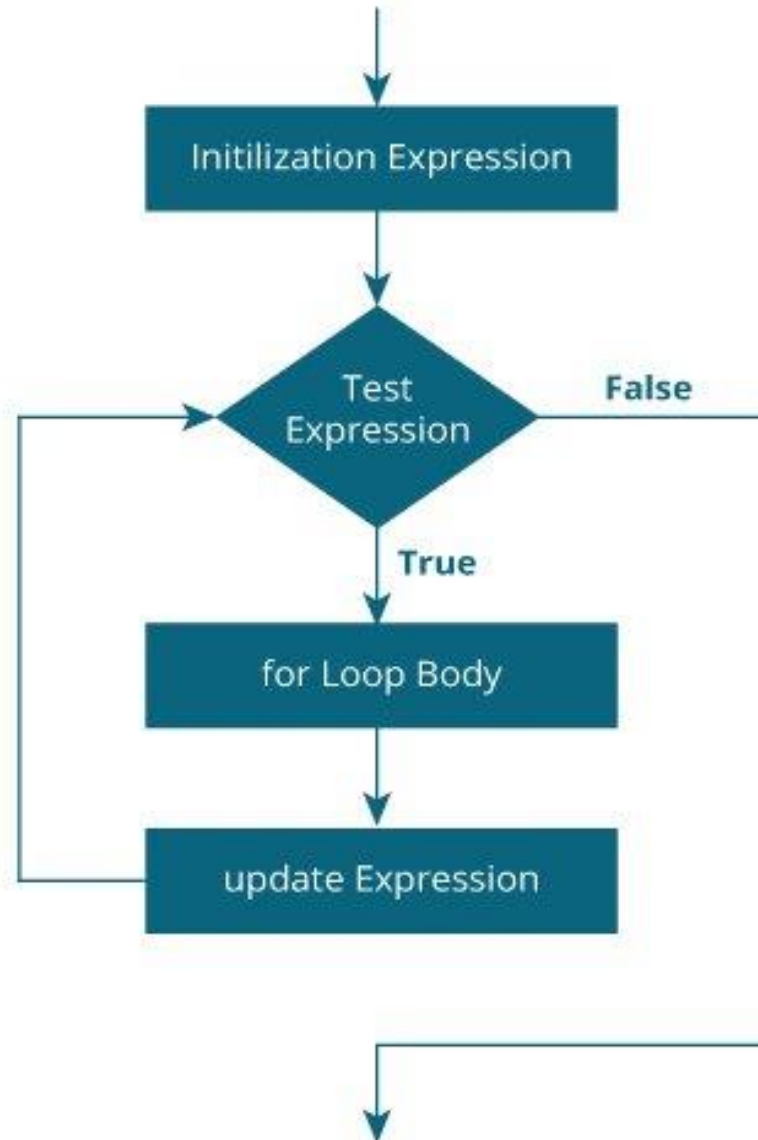
```
for (initializationStatement; testExpression; updateStatement)
{
 // statements inside the body of loop
}
```

### How for loop works?

- The initialization statement is executed only once.
- Then, the test expression is evaluated. If the test expression is evaluated to false, the `for` loop is terminated.
- However, if the test expression is evaluated to true, statements inside the body of the `for` loop are executed, and the update expression is updated.
- Again the test expression is evaluated.

This process goes on until the test expression is false. When the test expression is false, the loop terminates.

# For loop



# For loop

```
// Print numbers from 1 to 10
#include <stdio.h>

int main() {
 int i;

 for (i = 1; i < 11; ++i)
 {
 printf("%d ", i);
 }
 return 0;
}
```

## while loop

The syntax of the `while` loop is:

```
while (testExpression) {
 // the body of the loop
}
```

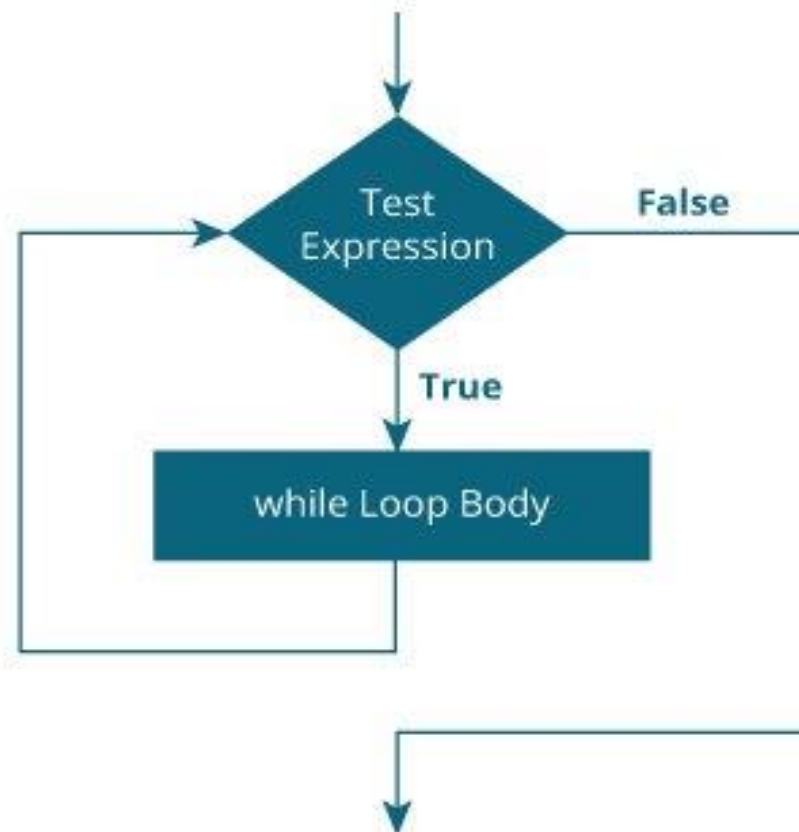
---

### How while loop works?

- The `while` loop evaluates the `testExpression` inside the parentheses `()`.
- If `testExpression` is **true**, statements inside the body of `while` loop are executed. Then, `testExpression` is evaluated again.
- The process goes on until `testExpression` is evaluated to **false**.
- If `testExpression` is **false**, the loop terminates (ends).



## Flowchart of while loop



## while loop

```
// Print numbers from 1 to 5

#include <stdio.h>
int main() {
 int i = 1;

 while (i <= 5) {
 printf("%d\n", i);
 ++i;
 }

 return 0;
}
```

## do...while loop

The `do...while` loop is similar to the `while` loop with one important difference. The body of `do...while` loop is executed at least once. Only then, the test expression is evaluated.

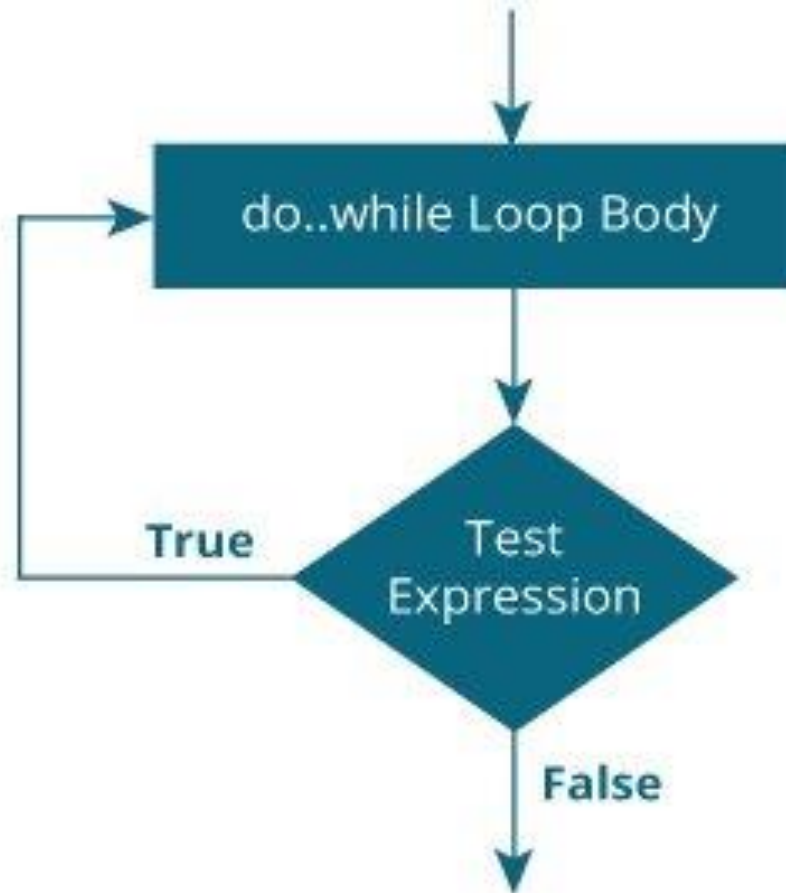
The syntax of the `do...while` loop is:

```
do {
 // the body of the loop
}
while (testExpression);
```

### How do...while loop works?

- The body of `do...while` loop is executed once. Only then, the `testExpression` is evaluated.
- If `testExpression` is **true**, the body of the loop is executed again and `testExpression` is evaluated once more.
- This process goes on until `testExpression` becomes **false**.
- If `testExpression` is **false**, the loop ends.

## Flowchart of do...while Loop



## Which loop should be used?

### **while Loops ( Condition-Controlled Loops )**

- Both while loops and do-while loops ( see below ) are ***condition-controlled***, meaning that they continue to loop until some condition is met.
- Both while and do-while loops alternate between performing actions and testing for the stopping condition.
- While loops check for the stopping condition first, and may not execute the body of the loop at all if the condition is initially false.
- do-while loops are exactly like while loops, except that the test is performed at the end of the loop rather than the beginning.
- This guarantees that the loop will be performed at least once, which is useful for checking user input among other things
- Used when we don't know how many times we have to iterate.

### **for Loops**

- for-loops are ***counter-controlled***, meaning that they are normally used whenever the number of iterations is known in advance.
- When we know how many times we have to loop over a particular piece of code.

So problems like print “Hello World five times” are better to be done with for loop instead of while loop.

## Goto statement

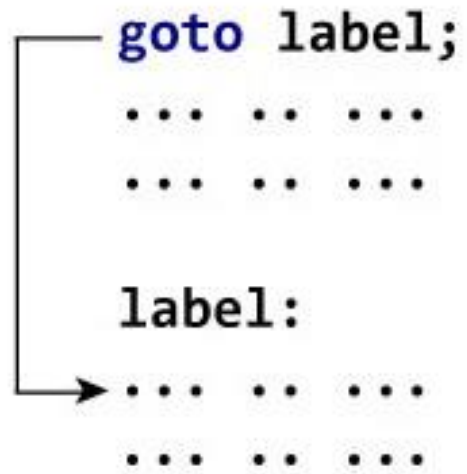
The `goto` statement allows us to transfer control of the program to the specified `label`.

### Syntax of goto Statement

```
goto label;
... ..
... ..
label:
statement;
```

The `label` is an identifier. When the `goto` statement is encountered, the control of the program jumps to `label:` and starts executing the code.

## Goto statement



```
goto label;
... ..
... ..

label:
... ..
... ..
```

Working of goto Statement

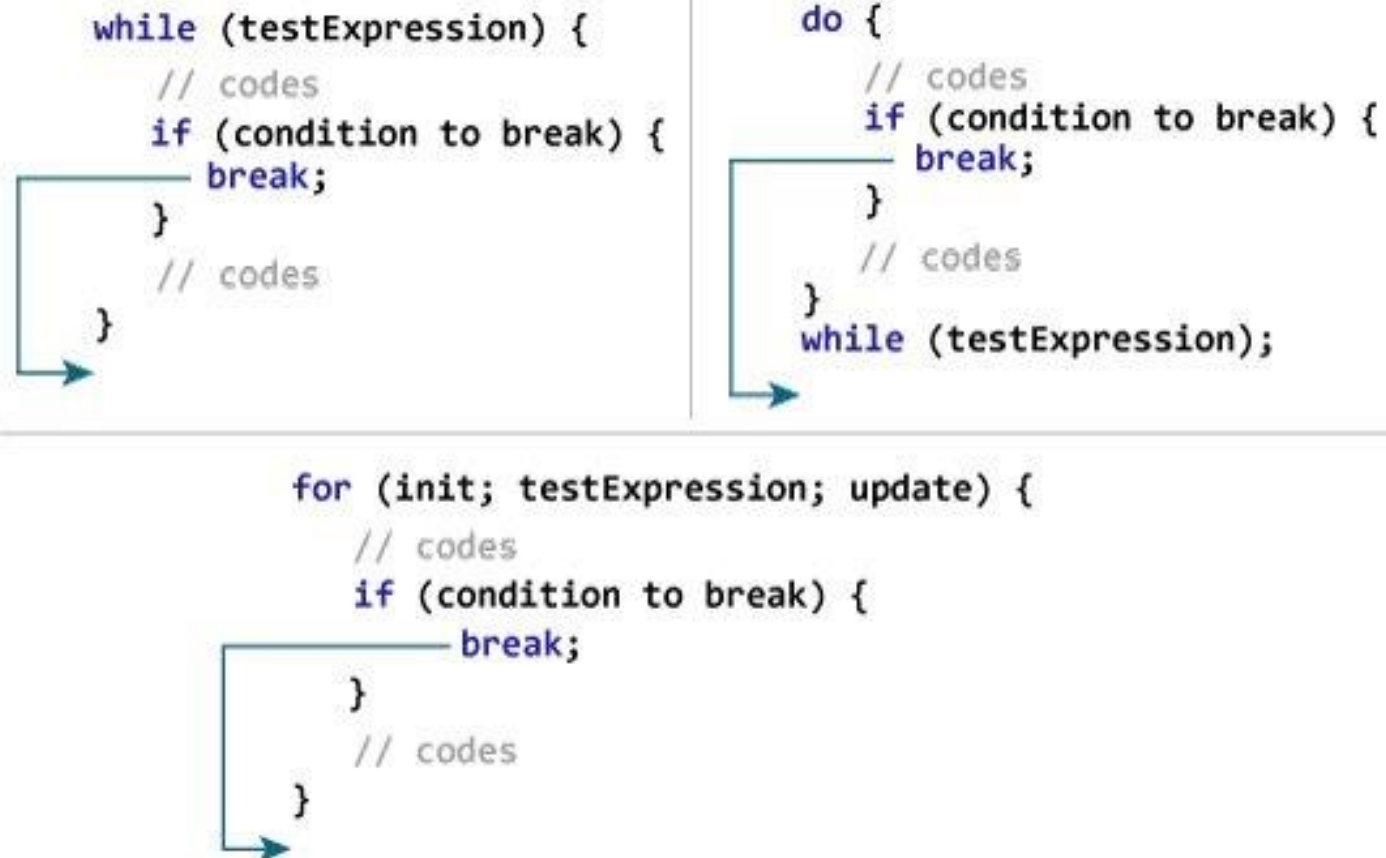
```
clrscr();
printf("1\n");
printf("2\n");
goto jump;
printf("3\n");
printf("4\n");
jump:
printf("jump");
getch();
```

## C break

The break statement ends the loop immediately when it is encountered. Its syntax is:

```
break;
```

The break statement is almost always used with `if...else` statement inside the loop.



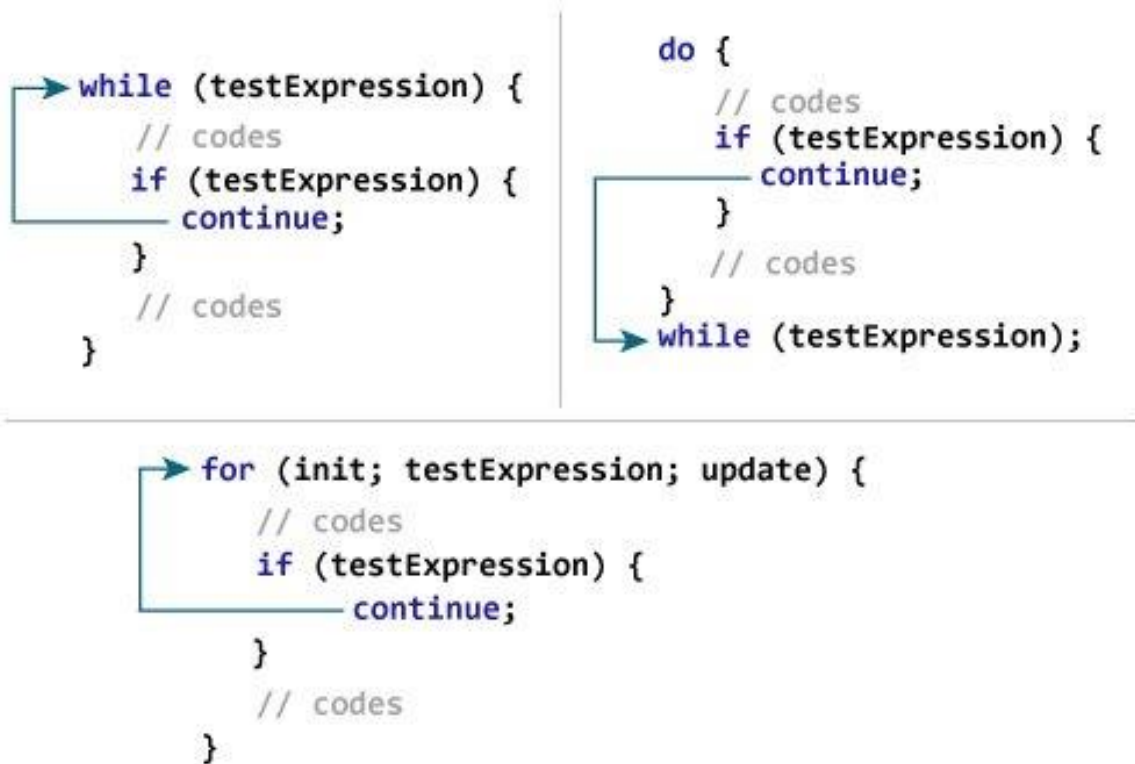


# C continue

The `continue` statement skips the current iteration of the loop and continues with the next iteration. Its syntax is:

```
continue;
```

The `continue` statement is almost always used with the `if...else` statement.



```
void main()
{
 int i;
 for(i=1;i<=10;i++)
 {
 if(i==5)
 {
 break;
 }
 printf("%d",i);
 }
 getch();
}
```

```
int i;
clrscr();
for(i=1;i<=10;i++)
{
 if(i==5)
 {
 continue;
 }
 printf("%d",i);
}
```

## Formatted I/O Functions.

- Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers.
- These I/O supports all data types like int, float, char, and many more.

### **Why they are called formatted I/O?**

- These functions are called formatted I/O functions because we can use format specifiers in these functions and hence, we can format these functions according to our needs.
- Examples of format specifiers are %d,%f,%c etc.

The following are formatted I/O functions

printf()

scanf()

sprintf()

sscanf()

## Formatted I/O Functions.

### **sprintf():**

- sprintf stands for “**string print**”.
- This function is similar to printf() function but this function prints the string into a character array instead of printing it on the console screen.

#### Syntax:

```
sprintf(array_name, "format specifier", variable_name);
```

```
void main()
{
 char str[50];
 clrscr();
 //string is now stored into str
 sprintf(str, "hello world");

 //display string
 printf("%s", str);
 getch();
}
```

## Formatted I/O Functions.

### **sscanf():**

- sscanf stands for “**string scanf**”.
- This function is similar to scanf() function but this function reads data from the string or character array instead of the console screen.

### **Syntax:**

```
sscanf(array_name, "format specifier", &variable_name);
```

```
char str[50];
int a=2,b=8,c,d;
clrscr();
//string "a=2 and b=8" is stored into str
sprintf(str,"a=%d and b=%d",a,b);

//now value of a and b are in c and d
sscanf(str,"a=%d and b=%d",&c,&d);

//display c and d
printf("c=%d and d=%d",c,d);_
getch();
```

## Unformatted Input/Output functions

- Unformatted I/O functions are used only for character data type or character array/string and cannot be used for any other datatype.
- These functions are used to read single input from the user at the console and it allows to display the value at the console.

These functions are called unformatted I/O functions because we cannot use format specifiers in these functions and hence, cannot format these functions according to our needs.

The following unformatted I/O functions will be discussed in this section-

- 1.getch()
- 2.getche()
- 3.getchar()
- 4.putchar()
- 5.gets()
- 6.puts()
- 7.putch()

## Unformatted Input/Output functions

`getch()`:

`getch()` function reads a single character from the keyboard by the user but doesn't display that character on the console screen and immediately returned without pressing enter key. This function is declared in `conio.h`(header file). `getch()` is also used for hold the screen.

Syntax:

```
getch();
```

or

```
variable-name = getch();
```

```
// C program to implement
```

```
// getch() function
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
// Driver code
```

```
void main()
```

```
{
```

```
 printf("Enter any character: ");
```

```
 // Reads a character but
```

```
 // not displays
```

```
 getch();
```

```
}
```

## Unformatted Input/Output functions

`getche()`:

`getche()` function reads a single character from the keyboard by the user and displays it on the console screen and immediately returns without pressing the enter key. This function is declared in `conio.h`(header file).

Syntax:

```
getche();
```

*or*

```
variable_name = getche();
```

```
void main()
{
 clrscr();
 printf("enter char");
 getche();
 getch();
}
```



## Unformatted Input/Output functions

`getchar()`:

The `getchar()` function is used to read only a first single character from the keyboard whether multiple characters are typed by the user and this function reads one character at one time until and unless the enter key is pressed. This function is declared in `stdio.h` (header file)

Syntax:

```
Variable-name = getchar();
```

```
void main()
{
 char ch;
 clrscr();
 printf("enter char");
 ch=getchar();
 printf("%c",ch);
 getch();
}
```

## Unformatted Input/Output functions

### putchar():

The putchar() function is used to display a single character at a time by passing that character directly to it or by passing a variable that has already stored a character. This function is declared in `stdio.h`(header file)

### Syntax:

```
putchar(variable_name);
```

```
void main()
{
 char ch;
 clrscr();
 printf("enter char");
 ch=getchar();
 putchar(ch);
 getch();
}
```

## Unformatted Input/Output functions

`gets()`:

`gets()` function reads a group of characters or strings from the keyboard by the user and these characters get stored in a character array. This function allows us to write space-separated texts or strings. This function is declared in `stdio.h`(header file).

Syntax:

```
char str[length of string in number]; //Declare a char type variable of any length
```

```
gets(str);
```

```
void main()
{
 char name[50];
 clrscr();
 printf("enter some char");
 gets(name);
 printf("your name is: %s",name);_
 getch();
}
```

## Unformatted Input/Output functions

**puts():**

In C programming puts() function is used to display a group of characters or strings which is already stored in a character array. This function is declared in `stdio.h`(header file).

**Syntax:**

```
puts(identifier_name);
```

```
void main()
{
 char name[50];
 clrscr();
 printf("enter some char");
 gets(name);
 printf("your name is:");
 puts(name);_
 getch();
}
```

## Unformatted Input/Output functions

putch():

putch() function is used to display a single character which is given by the user and that character prints at the current cursor location. This function is declared in conio.h(header file)

Syntax:

```
putch(variable_name);
```

```
void main()
{
 char ch;
 clrscr();
 printf("enter char");
 ch=getch();
 printf("char is:");
 putch(ch);
 getch();
}
```

## Printf formatted output

```
{
 int a,b;
 float c,d;
 clrscr();
 a=15;
 b=a/2;
 printf("%d\n",b);
 printf("%3d\n",b);
 printf("%03d\n",b);
 c=15.3;
 d=c/3;
 printf("%3.2f\n",d);
 getch();
}
```

Output of the source above:



```
7
 7
007
5.10
```

As you can see in the first printf statement we print a decimal. In the second printf statement we print the same decimal, but we use a width (%3d) to say that we want three digits (positions) reserved for the output.

The result is that two "space characters" are placed before printing the character. In the third printf statement we say almost the same as the previous one. Print the output with a width of three digits, but fill the space with 0.

In the fourth printf statement we want to print a float. In this printf statement we want to print three position before the decimal point (called width) and two positions behind the decimal point (called precision).

## Printf formatted output

- %d (print as a decimal integer)
- %6d (print as a decimal integer with a width of at least 6 wide)
- %f (print as a floating point)
- %4f (print as a floating point with a width of at least 4 wide)
- %.4f (print as a floating point with a precision of four characters after the decimal point)
- %3.2f (print as a floating point at least 3 wide and a precision of 2)

```
clrscr();
printf("the color is %s\n","pink");
printf("first no is %d\n",1234);
printf("float no is %3.2f\n",3.14159);
getch();
```

```
the color is pink
first no is 1234
float no is 3.14
```

## Printf formatted output

```
clrscr();
printf(":%s:\n","hello, world!");
printf(":%15s:\n","hello, world!");
printf(":%.10s:\n","hello, world!");
printf(":%-10s:\n","hello, world!");
printf(":%-15s:\n","hello, world!");
printf(":%.15s:\n","hello, world!");
printf(":%15.10s:\n","hello, world!");
printf(":%-15.10s:\n","hello, world!");_
getch();
```

```
:hello, world!:
: hello, world!:
:hello, wor:
:hello, world!:
:hello, world! :
:hello, world!:
: hello, wor:
:hello, wor :
```

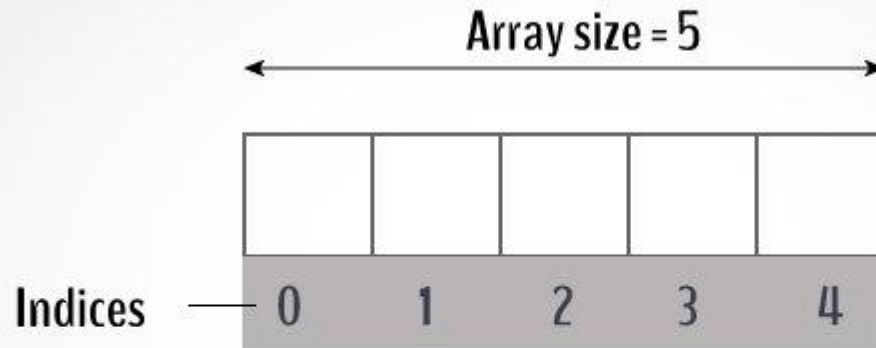


## Printf formatted output

- The `printf(":%s:\n", "Hello, world!");` statement prints the string (nothing special happens.)
- The `printf(":%15s:\n", "Hello, world!");` statement prints the string, but print 15 characters. If the string is smaller the "empty" positions will be filled with "whitespace."
- The `printf(":%.10s:\n", "Hello, world!");` statement prints the string, but print only 10 characters of the string.
- The `printf(":%-10s:\n", "Hello, world!");` statement prints the string, but prints at least 10 characters. If the string is smaller "whitespace" is added at the end. (See next example.)
- The `printf(":%-15s:\n", "Hello, world!");` statement prints the string, but prints at least 15 characters. The string in this case is shorter than the defined 15 character, thus "whitespace" is added at the end (defined by the minus sign.)
- The `printf(":%.15s:\n", "Hello, world!");` statement prints the string, but print only 15 characters of the string. In this case the string is shorter than 15, thus the whole string is printed.
- The `printf(":%15.10s:\n", "Hello, world!");` statement prints the string, but print 15 characters. If the string is smaller the "empty" positions will be filled with "whitespace." But it will only print a maximum of 10 characters, thus only part of new string (old string plus the whitespace positions) is printed.
- The `printf(":%-15.10s:\n", "Hello, world!");` statement prints the string, but it does the exact same thing as the previous statement, accept the "whitespace" is added at the end.

## Arrays

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- To create an array, define the data type (like `int`) and specify the name of the array followed by **square brackets []**.
- For example, if you want to store 100 integers, you can create an array for it :  
`int data[100];`



# C Arrays

## How to declare an array?

```
dataType arrayName[arraySize];
```

For example,

```
float mark[5];
```

Here, we declared an array, `mark`, of floating-point type. And its size is 5. Meaning, it can hold 5 floating-point values.

For example,

```
float mark[5];
```

## Access Array Elements

You can access elements of an array by indices.

Suppose you declared an array `mark` as above. The first element is `mark[0]`, the second element is `mark[1]` and so on.

`mark[0]` `mark[1]` `mark[2]` `mark[3]` `mark[4]`

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|--|--|--|--|--|

Declare an Array

### Few keynotes:

- Arrays have 0 as the first index, not 1. In this example, `mark[0]` is the first element.
- If the size of an array is `n`, to access the last element, the `n-1` index is used. In this example, `mark[4]`
- Suppose the starting address of `mark[0]` is **2120d**. Then, the address of the `mark[1]` will be **2124d**. Similarly, the address of `mark[2]` will be **2128d** and so on. This is because the size of a `float` is 4 bytes.

## How to initialize an array?

It is possible to initialize an array during declaration. For example,

```
int mark[5] = {19, 10, 8, 17, 9};
```

You can also initialize an array like this.

```
int mark[] = {19, 10, 8, 17, 9};
```

Here, we haven't specified the size. However, the compiler knows its size is 5 as we are initializing it with 5 elements.

| mark[0] | mark[1] | mark[2] | mark[3] | mark[4] |
|---------|---------|---------|---------|---------|
| 19      | 10      | 8       | 17      | 9       |

|    |    |   |    |   |
|----|----|---|----|---|
| 19 | 10 | 8 | 17 | 9 |
|----|----|---|----|---|

```
mark[0] is equal to 19
mark[1] is equal to 10
mark[2] is equal to 8
mark[3] is equal to 17
mark[4] is equal to 9
```

## Change Value of Array elements

```
int mark[5] = {19, 10, 8, 17, 9}

// make the value of the third element to -1
mark[2] = -1;

// make the value of the fifth element to 0
mark[4] = 0;
```

## Input and Output Array Elements

Here's how you can take input from the user and store it in an array element.

```
// take input and store it in the 3rd element
scanf("%d", &mark[2]);

// take input and store it in the ith element
scanf("%d", &mark[i-1]);
```

Here's how you can print an individual element of an array.

```
// print the first element of the array
printf("%d", mark[0]);

// print the third element of the array
printf("%d", mark[2]);

// print ith element of the array
printf("%d", mark[i-1]);
```



```
// Program to take 5 values from the user and store them in an array
// Print the elements stored in the array

#include <stdio.h>

int main() {

 int values[5];

 printf("Enter 5 integers: ");

 // taking input and storing it in an array
 for(int i = 0; i < 5; ++i) {
 scanf("%d", &values[i]);
 }

 printf("Displaying integers: ");

 // printing elements of an array
 for(int i = 0; i < 5; ++i) {
 printf("%d\n", values[i]);
 }
 return 0;
}
```

```
Enter 5 integers: 1
-3
34
0
3
Displaying integers: 1
-3
34
0
3
```

# Bubble Sort

**Bubble sort** is a sorting algorithm that compares two adjacent elements and swaps them until they are in the intended order.

Just like the movement of air bubbles in the water that rise up to the surface, each element of the array move to the end in each iteration. Therefore, it is called a bubble sort.

## Working of Bubble Sort

Suppose we are trying to sort the elements in **ascending order**.

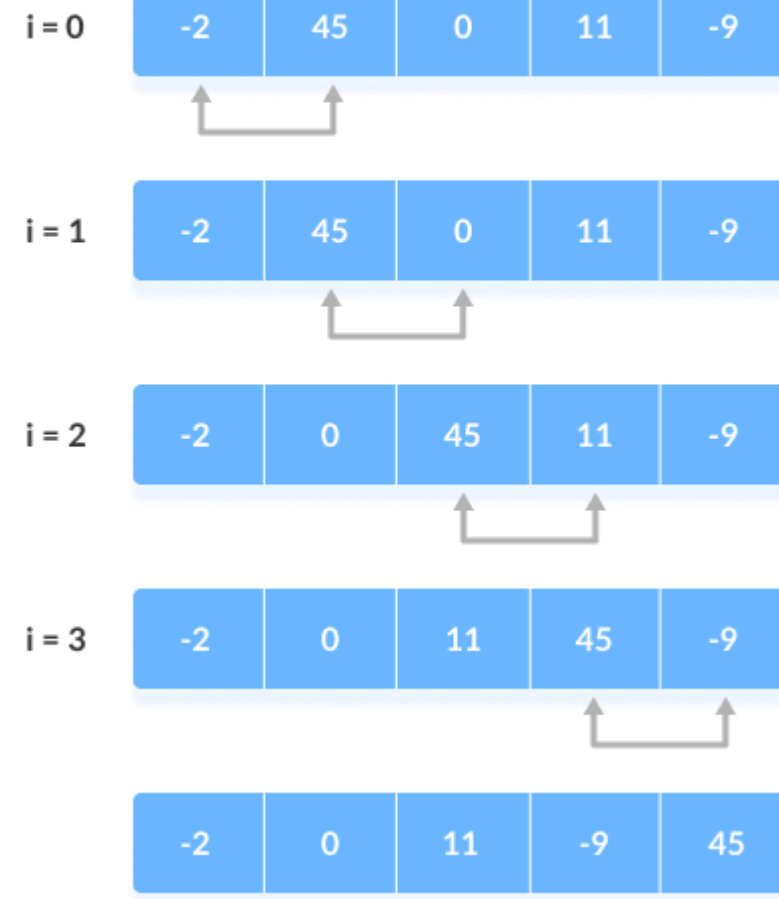
### 1. First Iteration (Compare and Swap)

1. Starting from the first index, compare the first and the second elements.
2. If the first element is greater than the second element, they are swapped.
3. Now, compare the second and the third elements. Swap them if they are not in order.
4. The above process goes on until the last element.

## 1. First Iteration (Compare and Swap)

1. Starting from the first index, compare the first and the second elements.
2. If the first element is greater than the second element, they are swapped.
3. Now, compare the second and the third elements. Swap them if they are not in order.
4. The above process goes on until the last element.

step = 0



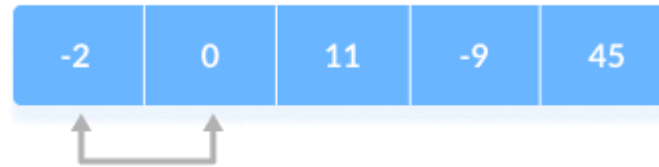
## 2. Remaining Iteration

The same process goes on for the remaining iterations.

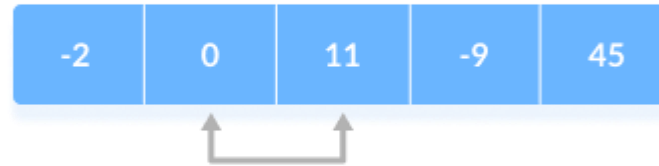
After each iteration, the largest element among the unsorted elements is placed at the end.

step = 1

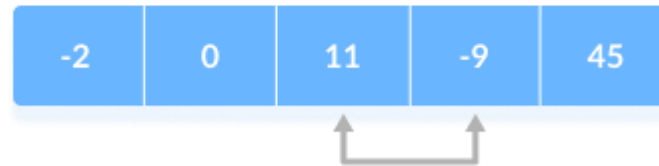
i = 0



i = 1



i = 2



In each iteration, the comparison takes place up to the last unsorted element.

step = 2

i = 0



i = 1

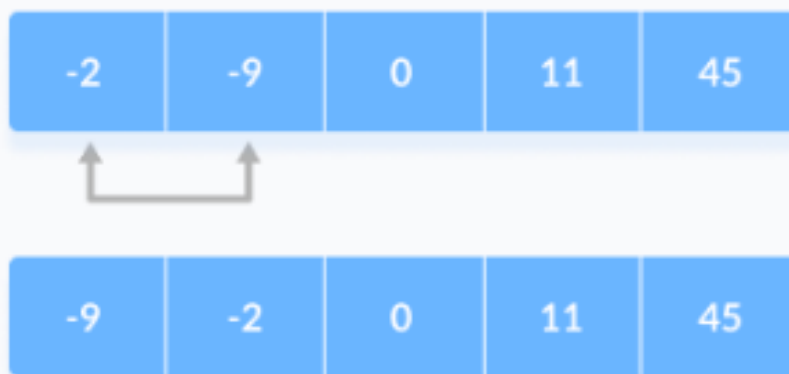


Compare the adjacent elements

The array is sorted when all the unsorted elements are placed at their correct positions.

step = 3

i = 0



The array is sorted if all elements are kept in the right order

# C Multidimensional Arrays

In C programming, you can create an array of arrays. These arrays are known as multidimensional arrays. For example,

```
float x[3][4];
```

Here, `x` is a two-dimensional (2d) array. The array can hold 12 elements. You can think the array as a table with 3 rows and each row has 4 columns.

|       | Column 1             | Column 2             | Column 3             | Column 4             |
|-------|----------------------|----------------------|----------------------|----------------------|
| Row 1 | <code>x[0][0]</code> | <code>x[0][1]</code> | <code>x[0][2]</code> | <code>x[0][3]</code> |
| Row 2 | <code>x[1][0]</code> | <code>x[1][1]</code> | <code>x[1][2]</code> | <code>x[1][3]</code> |
| Row 3 | <code>x[2][0]</code> | <code>x[2][1]</code> | <code>x[2][2]</code> | <code>x[2][3]</code> |

Two dimensional Array

Similarly, you can declare a three-dimensional (3d) array. For example,

```
float y[2][4][3];
```

Here, the array `y` can hold 24 elements.



## Initializing a multidimensional array

Here is how you can initialize two-dimensional and three-dimensional arrays:

---

### Initialization of a 2d array

```
// Different ways to initialize two-dimensional array

int c[2][3] = {{1, 3, 0}, {-1, 5, 9}};

int c[][3] = {{1, 3, 0}, {-1, 5, 9}};

int c[2][3] = {1, 3, 0, -1, 5, 9};
```

Program to input and print 2D array

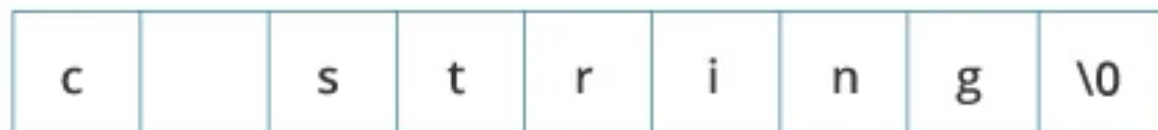
```
clrscr();
int a[2][2],i,j;
for(i=0;i<2;i++)
{
 for(j=0;j<2;j++)
 {
 printf("enter a[%d][%d]:",i,j);
 scanf("%d",&a[i][j]);
 }
}
for(i=0;i<2;i++)
{
 for(j=0;j<2;j++)
 {
 printf("elements are a[%d][%d]:%d\n",i,j,a[i][j]);
 }
}
getch();
```

# C Programming Strings

In C programming, a string is a sequence of characters terminated with a null character `\0`. For example:

```
char c[] = "c string";
```

When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character `\0` at the end by default.



Memory Diagram

Here's how you can declare strings:

```
char s[5];
```

| s[0] | s[1] | s[2] | s[3] | s[4] |
|------|------|------|------|------|
|      |      |      |      |      |

String Declaration in C

Here, we have declared a string of 5 characters.

## How to initialize strings?

You can initialize strings in a number of ways.

```
char c[] = "abcd";
```

```
char c[50] = "abcd";
```

```
char c[] = {'a', 'b', 'c', 'd', '\0'};
```

```
char c[5] = {'a', 'b', 'c', 'd', '\0'};
```

| c[0] | c[1] | c[2] | c[3] | c[4] |
|------|------|------|------|------|
| a    | b    | c    | d    | \0   |

String Initialization in C

```
void main()
{
 char c[10]="hello";
 clrscr();
 printf("%s",c);

 getch();
}
```

```
void main()
{
 char name[20];
 clrscr();
 printf("enter name");
 scanf("%s",name);
 printf("your name is: %s",name);
 getch();
}
```

```
enter name hello world
your name is: hello_
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
 char name[20];
 clrscr();
 _printf("enter name");
 gets(name);
 printf("your name is: %s",name);
 getch();
}
```

## Example

```
char greetings[] = "Hello World!";
printf("%c", greetings[0]);
```

## Modify Strings

```
char greetings[] = "Hello World!";
greetings[0] = 'J';
printf("%s", greetings);
// Outputs Jello World! instead of Hello World!
```

## Loop Through a String

```
char carName[] = "Volvo";
int i;

for (i = 0; i < 5; ++i) {
 printf("%c\n", carName[i]);
}
```

# String Manipulations In C Programming

## Using Library Functions

You need to often manipulate [strings](#) according to the need of a problem. Most, if not all, of the time string manipulation can be done manually but, this makes programming complex and large.

To solve this, C supports a large number of string handling functions in the [standard library](#) `"string.h"`.

Few commonly used string handling functions are discussed below:



Few commonly used string handling functions are discussed below:

| Function              | Work of Function                |
|-----------------------|---------------------------------|
| <code>strlen()</code> | computes string's length        |
| <code>strcpy()</code> | copies a string to another      |
| <code>strcat()</code> | concatenates(joins) two strings |
| <code>strcmp()</code> | compares two strings            |
| <code>strlwr()</code> | converts string to lowercase    |
| <code>strupr()</code> | converts string to uppercase    |

Functions gets() and puts() are two string functions to take string input from the user and display it respectively

```
#include<stdio.h>

int main()
{
 char name[30];
 printf("Enter name: ");
 gets(name); //Function to read string from user.
 printf("Name: ");
 puts(name); //Function to display string.
 return 0;
}
```

Strlen()

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
 char name[]="hello world";
 clrscr();
 printf("%d",strlen(name));
 getch();
}
```

## Concatenate Strings – strcat()

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
 char str1[]="hello";
 char str2[]=" world";
 clrscr();
 strcat(str1,str2);
 printf("%s",str1);
 getch();
}
```

Copy Strings– strcpy()

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
 char str1[20]="hello world";
 char str2[20];
 clrscr();
 //copy str1 to str2
 strcpy(str2,str1);
 printf("%s",str2);
 getch();
}
```

## Compare strings – strcmp()

To compare two strings, you can use the `strcmp()` function.  
It returns `0` if the two strings are equal, otherwise a value that is not 0:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
 char str1[]="hello";
 char str2[]="hello";
 char str3[]="hi";
 clrscr();
 printf("%d\n",strcmp(str1,str2));
 printf("%d\n",strcmp(str1,str3));
 getch();
}
```

Upper case and lower case

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
 char str1[]="hello";_
 char str3[]="HI";
 clrscr();
 printf("%s",strlwr(str3));
 printf("%s",strupr(str1));
 getch();
}
```

## Array of strings

In C, an array represents a linear collection of data elements of a similar type, thus an array of strings means a collection of several strings. The string is a one-dimensional array of characters terminated with the null character. Since strings are arrays of characters, the array of strings will evolve into a two-dimensional array of characters.

### Syntax

It is possible to define an array of strings in various ways, but generally, we can do so as follows:

```
char variableName[numberOfStrings][maxSizeOfString];
```



### Initialization of Array of Strings

Initialization in C means to provide the data to the variable at the time of declaration. It is pretty straightforward to initialize an array of strings. We can just create an array of some size and then we can place comma-separated strings inside that. See the example below to get a good idea about it.



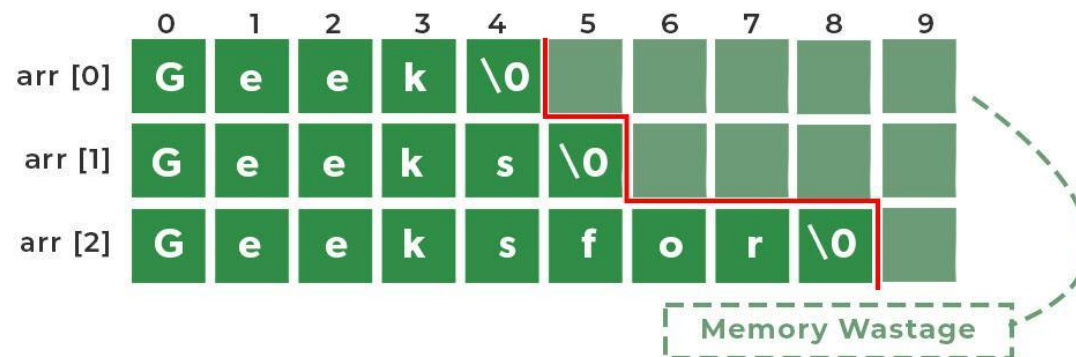
## Array of strings

```
char car[5][20] = {"Mahindra", "Audi", "Lamborghini", "Hyundai", "Tata Motors"};
```

### Explanation:

In this example, we have initialized a two-dimensional character array **car** which can have a maximum of 5 strings of size 20.

### Memory Representation of an Array of Strings



## Array of strings

```
int itemno,charpos;
char car[5][20]={"mahindra","audi","maruti","volvo","hyundai"};
clrscr();
for(itemno=0;itemno<5;itemno++)
{
 for(charpos=0;charpos<strlen(car[itemno]);charpos++)
 {
 printf("%c",car[itemno][charpos]);
 }
 printf("\n");
}
getch();
```

```
mahindra
audi
maruti
volvo
hyundai
```

## Functions

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

For example, `main()` is a function, which is used to execute code, and `printf()` is a function; used to output/print text to the screen:

Suppose, you need to create a program to create a circle and color it. You can create two functions to solve this problem:

- create a circle function
- create a color function

Dividing a complex problem into smaller chunks makes our program easy to understand and reuse.

## Types of function

There are two types of function in C programming:

- Standard library functions
- User-defined functions

## Standard library functions

The standard library functions are built-in functions in C programming.

These functions are defined in header files. For example,

- The `printf()` is a standard library function to send formatted output to the screen (display output on the screen). This function is defined in the `stdio.h` header file. Hence, to use the `printf()` function, we need to include the `stdio.h` header file using `#include <stdio.h>`.
- The `sqrt()` function calculates the square root of a number. The function is defined in the `math.h` header file.

# Create a Function

To create (often referred to as *declare*) your own function, specify the name of the function, followed by parentheses `()` and curly brackets `{}`:

## Syntax

```
void myFunction() {
 // code to be executed
}
```

## Example Explained

- `myFunction()` is the name of the function
- `void` means that the function does not have a return value. You will learn more about return values later in the next chapter
- Inside the function (the body), add code that defines what the function should do

# Call a Function

Declared functions are not executed immediately. They are "saved for later use", and will be executed when they are called.

To call a function, write the function's name followed by two parentheses `()` and a semicolon `;`

You can also create functions as per your need. Such functions created by the user are known as user-defined functions.

## How user-defined function works?

```
#include <stdio.h>
void functionName()
{

}

int main()
{

 functionName();

}
```

The execution of a C program begins from the `main()` function.

When the compiler encounters `functionName();`, control of the program jumps to

```
void functionName()
```

And, the compiler starts executing the codes inside `functionName()`.

The control of the program jumps back to the `main()` function once code inside the function definition is executed.



Note, function names are identifiers and should be unique.

## How function works in C programming?

```
#include <stdio.h>
```

```
void functionName()
{
```

```


```

```
}
```

```
int main()
{
```

```

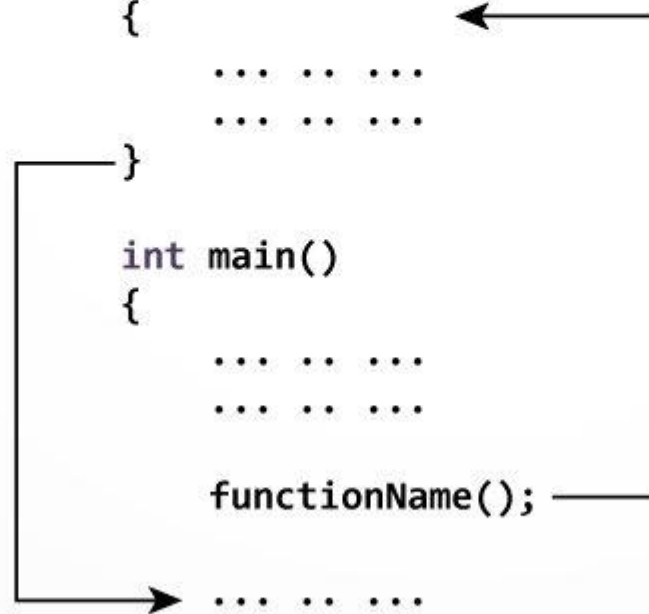

```

```
 functionName();
```

```


```

```
}
```



## **Advantages of user-defined function**

1. The program will be easier to understand, maintain and debug.
2. Reusable codes that can be used in other programs
3. A large program can be divided into smaller modules. Hence, a large project can be divided among many programmers.

```
#include<conio.h>
void fun();
void main()
{
 clrscr();
 printf("This is starting of main function\n");
 fun();
 printf("This is ending of main function\n");
 getch();
}
void fun()
{
 printf("this is UDF\n");
}
```

```
#include<stdio.h>
#include<conio.h>
void addno(int a,int b);
void main()
{
 int n1,n2,sum;
 clrscr();
 printf("enter two numbers:");
 scanf("%d%d",&n1,&n2);

 addno(n1,n2);

 getch();
}
void addno(int a,int b)
{
 int result;
 result=a+b;
 printf("sum is:%d",result);
}
```

## Types of User Defined Functions in C

A user-defined function is one that is defined by the user when writing any program, as we do not have library functions that have predefined definitions. To meet the specific requirements of the user, the user has to develop his or her own functions. Such functions must be defined properly by the user. There is no such kind of requirement to add any particular library to the program.

### Different Types of User-defined Functions in C

There are four types of user-defined functions divided on the basis of arguments they accept and the value they return:

1. Function with no arguments and no return value
2. Function with no arguments and a return value
3. Function with arguments and no return value
4. Function with arguments and with return value

## Types of User Defined Functions in C

### d) Functions Without Arguments and Without Return Values

In this type of user-defined function, we do not pass any arguments when we define, declare or call a function. Also, this function does not return any values when it is called from the `main()` function or any submethod inside the program. This type of function is used when we do not expect any return value but we need some statement to print the result as output. Since this function has no arguments and returns a value, it does not receive any data when the function is called.

```
void add();
void main()
{
 add();
 getch();
}
void add()
{
 int a=5,b=3,sum;
 sum=a+b;
 printf("sum is:%d",sum);_
}
```

## Types of User Defined Functions in C

### c) Functions Without Arguments and With Return Values

In this type of user-defined function, we do not pass any arguments when we define, declare or call a function. But, this type of function returns some value when it is called from the `main()` function or any submethod in the program. In it, the data type of return value is dependent on the return type of the declaration function. Suppose, if the return type is of 'int' then the return value will be also of `int` type. Let us see an

```
int add();
void main()
{
 int result;
 result=add();
 printf("result is:%d",result);
 getch();
}
int add()
{
 int a=5,b=3,sum;
 sum=a+b;
 return sum;
}
```

## Types of User Defined Functions in C

### b) Functions With Arguments and Without Return Values

In this type of function, the arguments are also passed to the function while calling it. But it will not return any value when the function is called from the main function or any submethod. This function contains arguments but does not return any value. In this method, we allow the user to enter the values as input rather than fixed values. Since we are allowing the user to enter the values as input, we do not expect any return type value. This type of function can be used in real-life problems. Let us take an example where the user will enter two values as input and these values will be passed to the user-defined function, which will do addition.

```
void add(int,int);
void main()
{
 int a,b;
 printf("enter a,b");
 scanf("%d%d",&a,&b);
 add(a,b);
 getch();
}
void add(int x,int y)
{
 int sum;
 sum=x+y;
 printf("sum is:%d",sum);
}
```

## Types of User Defined Functions in C

### a) Functions With Arguments and Return Values

This type of function has arguments and always returns a value. In this method, the arguments are passed to the function while calling it. The function will return some value when it is called from `main()` or any subfunction in the program. In it, data types are a must to define. If the return is of 'int' type, then the return value will also be 'int' type. This type of user-defined function is also called a **fully dynamic function** because the total control is in the hand of the end-user.

```
int add(int,int);
void main()
{
 int a,b,result;
 printf("enter a,b");
 scanf("%d%d",&a,&b);
 result=add(a,b);
 printf("result is: %d",result);
 getch();
}

int add(int x,int y)
{
 int sum;
 sum=x+y;
 return sum;
}
```



## Local and global variable in c

A scope in any programming is a region of the program where a defined variable can have its existence and beyond that variable it cannot be accessed. There are three places where variables can be declared in C programming language –

- Inside a function or a block which is called **local** variables.
- Outside of all functions which is called **global** variables.
- In the definition of function parameters which are called **formal** parameters.

## Local and global variable in c

# Local Variables

Variables that are declared inside a function or block are called local variables. They can be used only by statements that are inside that function or block of code. Local variables are not known to functions outside their own. The following example shows how local variables are used. Here all the variables a, b, and c are local to main() function.

```
#include <stdio.h>

int main () {

 /* local variable declaration */
 int a, b;
 int c;

 /* actual initialization */
 a = 10;
 b = 20;
 c = a + b;

 printf ("value of a = %d, b = %d and c = %d\n", a, b, c);

 return 0;
}
```

## Local and global variable in c

### Global Variables

Global variables are defined outside a function, usually on top of the program. Global variables hold their values throughout the lifetime of your program and they can be accessed inside any of the functions defined for the program.

A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration. The following program show how global variables are used in a program.

```
#include<stdio.h>
#include<conio.h>
int g=10;
void main()
{
 int a=5,b=2;
 g=a+b;
 printf("sum is:%d",g);
 getch();
}
```

## Local and global variable in c

A program can have same name for local and global variables but the value of local variable inside a function will take preference. Here is an example –

```
#include<stdio.h>
#include<conio.h>
int g=20;//global variable
void main()
{
 int g=10; //local variable
 printf("G is: %d",g);
 getch();
}
```

Output:  
G is 10

## Actual and Formal Parameters in C

# What are Actual Parameters in C?

Actual parameters are the values that are passed to the function during a function call. They are also known as arguments. Actual parameters are used to provide the values to the formal parameters of the function. Actual parameters can be of any data type such as int, float, char, etc.

## Syntax of Actual Parameters in C

```
function_name(actual_parameter1, actual_parameter2, ...);
```

## Actual and Formal Parameters in C

### Example of Actual Parameters in C

```
int main()
{
 int sum= add(10, 20);
 return 0;
}
```

## Actual and Formal Parameters in C

# What are Formal Parameters in C?

Formal parameters are also called function parameters or function arguments. They are used in the function definition to accept the values from the caller of the function. Formal parameters are declared in the function definition and are used to represent the data that will be passed to the function at the time of the function call. Formal parameters can be of any data type such as int, float, char, etc.

## Syntax of Formal Parameters in C

```
return_type function_name(parameter1_type parameter1_name, parameter2_ty
{
 // function body
}
```

## Actual and Formal Parameters in C

### Example of Formal Parameters in C

```
int add(int a, int b)
{
 return a + b;
}
```



## Actual and Formal Parameters in C

```
#include<stdio.h>
int add(int a, int b) {
 int sum = a + b;
 return sum;
}
int main()
{
 int sum = add(10, 20);
 return 0;
}
```

### Explanation of Actual and Formal Parameters in C

In the actual and formal parameters example, a and b are formal parameters or formal arguments. When this function is called with actual parameters, these parameters are passed to the function as arguments. Here, 10 and 20 are actual parameters that are passed to the add() function.

## Pass arrays to a function in C

In C programming, you can pass an entire array to functions. Before we learn that, let's see how you can pass individual elements of an array to functions.

### Pass Individual Array Elements

Passing array elements to a function is similar to [passing variables to a function](#).

#### Example 1: Pass Individual Array Elements

```
#include<stdio.h>
#include<conio.h>
void display(int,int);
void main()
{
 int a[5]={1,2,3,4,5};
 display(a[1],a[2]);
 getch();
}
void display(int x1,int x2)
{
 printf("x1:%d",x1);
 printf("x2:%d",x2);
}
```

## Pass arrays to a function in C

### Example 2: Pass Arrays to Functions

```
#include<stdio.h>
#include<conio.h>
int sum(int num[]);
void main()
{
 int a[5]={1,2,3,4,5};
 int result;
 clrscr();
 result=sum(a);
 printf("sum is:%d",result);
 getch();
}
int sum(int x[])
{
 int sum=0,i;
 for(i=0;i<5;i++)
 {
 sum=sum+x[i];
 }
 return sum;
}
```

# C Storage Class

Every variable in C programming has two properties: type and storage class.

Type refers to the data type of a variable. And, storage class determines the scope, visibility and lifetime of a variable.

There are 4 types of storage class:

1. automatic
2. external
3. static
4. register

## Lifetime of a variable in C

### What is the Lifetime of a variable in C?

Lifetime of a variable is defined as for how much time period a variable occupies a valid space in the system's memory or lifetime is the period between when memory is allocated to hold the variable and when it is freed. Once the variable is out of scope its lifetime ends. Lifetime is also known as the range/extent of a variable.

A variable in the C programming language can have a

- *static*,
- *automatic*, or
- *dynamic lifetime*.

## Lifetime of a variable in C

### a. Static Lifetime

Objects/Variables having static lifetime will remain in the memory until the execution of the program finishes. These types of variables can be declared using the **static** keyword, global variables also have a static lifetime: they survive as long as the program runs.

Example :

```
static int count = 0;
```

The **count** variable will stay in the memory until the execution of the program finishes.

```
#include<stdio.h>
#include<conio.h>
void fun();
void main()
{
 printf("first call\n");
 fun();
 printf("second call\n");
 fun();
 getch();
}

void fun()
{
 int a=10;
 static int b=20;
 a=a+1;
 b=b+1;
 printf("a is:%d\n",a);
 printf("b is:%d\n",b);
}
```

## Lifetime of a variable in C

### b. Automatic Lifetime

Objects/Variables declared inside a block have automatic lifetime. Local variables (those defined within a function) have an automatic lifetime by default: they arise when the function is invoked and are deleted (together with their values) once the function execution finishes.

Example :

```
{ // block of code
 int auto_lifetime_var = 0;
}
```

auto\_lifetime\_var will be deleted from the memory once the execution comes out of the block.

```
void fun();
void main()
{
 printf("first call\n");
 fun();
 getch();
}
void fun()
{
 int a=10;
 auto int b=20;
 printf("a is:%d\n",a);
 printf("b is:%d\n",b);
}
```

## Lifetime of a variable in C

### c. Dynamic Lifetime

Objects/Variables which are made during the run-time of a C program using the **Dynamic Memory Allocation** concept using the `malloc()`, `calloc()` functions in C or `new` operator in C++ are stored in the memory until they are explicitly removed from the memory using the `free()` function in C or `delete` operator in C++ and these variables are said to have a dynamic lifetime. These variables are stored in the *heap* section of our system's memory, which is also known as the *dynamic memory*.

Example :

```
int *ptr = (int *)malloc(sizeof(int));
```



Memory block (variable) pointed by `ptr` will remain in the memory until it is explicitly freed/removed from the memory or the program execution ends.



# C Recursion

A function that calls itself is known as a recursive function. And, this technique is known as recursion.

## How recursion works?

```
void recurse()
{

 recurse();

}

int main()
{

 recurse();

}
```

## How does recursion work?

The diagram shows the execution flow of a recursive function. It starts with the `main()` function, which calls `recurse()`. Inside `recurse()`, there is another call to `recurse()`. A bracket labeled "recursive call" connects the inner `recurse()` call back to the `recurse()` function definition, indicating a self-call. The `main()` function also has a bracket connecting its call to `recurse()` back to the `recurse()` function definition.

```
void recurse()
{

 recurse();

}

int main()
{

 recurse();

}
```

Working of Recursion

The recursion continues until some condition is met to prevent it.

To prevent infinite recursion, [if...else statement](#) (or similar approach) can be used where one branch makes the recursive call, and other doesn't.

# How Recursion Works?

The steps involved in a recursive function are as follows:

- The function checks whether a base condition has been met. If it has, the function returns a base value that terminates the recursion.
- If the base condition has not been met, the function calls itself with modified parameters.
- The function continues to call itself recursively until the base condition is met.
- Once the base condition is met, the function returns the final result of the recursion.

Note that the base case is essential to prevent infinite recursion, which could result in a stack overflow errors. It is also important to make sure that the modified parameters passed to the recursive call bring the function closer to the base case with each iteration.

```
#include<stdio.h>
#include<conio.h>
int add(int);
void main()
{
 int a=0,result;
 clrscr();
 result=add(a);
 printf("result:%d",result);
 getch();
}

int add(int x)
{
 if (x==10)
 {
 return x;
 }
 else
 {
 x++;
 return add(x);
 }
}
```

## Types of Recursion in C

There are two types of recursion in the C language.

1. Direct Recursion
2. Indirect Recursion

### 1. Direct Recursion

Direct recursion in C occurs when a function calls itself directly from inside. Such functions are also called direct recursive functions.

Following is the structure of direct recursion.

```
function_01()
{
 //some code
 function_01();
 //some code
}
```



In the direct recursion structure, the `function_01()` executes, and from inside, it calls itself recursively.

*Here is an example of a direct recursive function in C that calculates the factorial of number 5:*

```
#include <stdio.h>

int factorial(int n) {
 // base case
 if (n == 0 || n == 1) {
 return 1;
 }
 // recursive case
 else {
 return n * factorial(n - 1);
 }
}

int main() {
 int n = 5;
 int result = factorial(n);
 printf("Factorial of %d is %d\n", n, result);
 return 0;
}
```

In this example, the `factorial()` function is a direct recursive function that calculates the factorial of a number. If the input number is 0 or 1, the base case is met, and the function returns 1. If the input number is greater than 1, the function calls itself with a modified parameter ( $n - 1$ ), which eventually reaches the base case. Finally, the function returns the result of the recursion, which is the factorial of the input number.

When the `main()` function is executed, it calls the `factorial()` function with an input of 5, which is calculated recursively until the base case is met. The final result of the recursion is returned and printed to the console.

## 2. Indirect Recursion

Indirect recursion in C occurs when a function calls another function and if this function calls the first function again. Such functions are also called indirect recursive functions.

Following is the structure of indirect recursion.

```
function_01()
{
 //some code
 function_02();
}

function_02()
{
 //some code
 function_01();
}
```

In the indirect recursion structure the `function_01()` executes and calls `function_02()`. After calling now, `function_02` executes where inside it there is a call for `function_01`, which is the first calling function.

## C Program Function to show indirect recursion

Here is a C program to print numbers from 1 to 10 in such a manner that when an odd no is encountered, we will print that number plus 1. When an even number is encountered, we would print that number minus 1 and will increment the current number at every step.

```
#include<stdio.h>
#include<conio.h>
void even();
void odd();
int n=1;
void main()
{
 clrscr();
 odd();
 getch();
}
void odd()
{
 if(n<=10)
 {
 printf("%d ",n+1);
 n++;
 even();
 }
 return;
}
```

```
void even()
{
 if(n<=10)
 {
 printf("%d ",n-1);
 n++;
 odd();
 }
 return;
}
```

Output:

2 1 4 3 6 5 8 7 10 9

## Difference Between Recursion and Iteration

**Recursion and Iteration** both repeatedly execute the set of instructions. Recursion occurs when a statement in a function calls itself repeatedly. The iteration occurs when a loop repeatedly executes until the controlling condition becomes false. The basic difference between recursion and iteration is that recursion is a process always applied to a function and iteration is applied to the set of instructions which we want to be executed repeatedly.

| Recursion                                                                                                                                                                                                                                | Iteration                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Recursion uses the selection structure.                                                                                                                                                                                                  | Iteration uses the repetition structure.                                          |
| Infinite recursion occurs if the step in recursion doesn't reduce the problem to a smaller problem. It also becomes infinite recursion if it doesn't convert on a specific condition. This specific condition is known as the base case. | An infinite loop occurs when the condition in the loop doesn't become False ever. |



## Difference Between Recursion and Iteration

| Recursion                                                                                                                                                                                                                                | Iteration                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Recursion uses the selection structure.                                                                                                                                                                                                  | Iteration uses the repetition structure.                                          |
| Infinite recursion occurs if the step in recursion doesn't reduce the problem to a smaller problem. It also becomes infinite recursion if it doesn't convert on a specific condition. This specific condition is known as the base case. | An infinite loop occurs when the condition in the loop doesn't become False ever. |
| Recursion terminates when the base case is met.                                                                                                                                                                                          | Iteration terminates when the condition in the loop fails.                        |
| Recursion is slower than iteration since it has the overhead of maintaining and updating the stack.                                                                                                                                      | Iteration is quick in comparison to recursion. It doesn't utilize the stack.      |
| Recursion uses more memory in comparison to iteration.                                                                                                                                                                                   | Iteration uses less memory in comparison to recursion.                            |
| Recursion reduces the size of the code.                                                                                                                                                                                                  | Iteration increases the size of the code.                                         |

# C Pointers

## Address in C

If you have a variable `var` in your program, `&var` will give you its address in the memory.

We have used address numerous times while using the `scanf()` function.

```
scanf("%d", &var);
```

Here, the value entered by the user is stored in the address of `var` variable. Let's take a working example.

# C Pointers

Here, the value entered by the user is stored in the address of `var` variable. Let's take a working example.

```
#include <stdio.h>
int main()
{
 int var = 5;
 printf("var: %d\n", var);

 // Notice the use of & before var
 printf("address of var: %p", &var);
 return 0;
}
```

## Output

```
var: 5
address of var: 2686778
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int num=6;
 clrscr();
 printf("num is:%d",num);
 //address using &
 printf("address of num is:%p",&num);
 getch();
}
```

# C Pointers

## C Pointers

Pointers (pointer variables) are special variables that are used to store addresses rather than values.

### Pointer Syntax

Here is how we can declare pointers.

```
int* p;
```

Here, we have declared a pointer `p` of `int` type.

You can also declare pointers in these ways.

```
int *p1;
int * p2;
```

Let's take another example of declaring pointers.

```
int* p1, p2;
```

Here, we have declared a pointer `p1` and a normal variable `p2`.

# C Pointers

## Assigning addresses to Pointers

Let's take an example.

```
int* pc, c;
c = 5;
pc = &c;
```

Here, 5 is assigned to the `c` variable. And, the address of `c` is assigned to the `pc` pointer.

# C Pointers

## Get Value of Thing Pointed by Pointers

To get the value of the thing pointed by the pointers, we use the `*` operator. For example:

```
int* pc, c;
c = 5;
pc = &c;
printf("%d", *pc); // Output: 5
```

Here, the address of `c` is assigned to the `pc` pointer. To get the value stored in that address, we used `*pc`.

**Note:** In the above example, `pc` is a pointer, not `*pc`. You cannot and should not do something like `*pc = &c;`

By the way, `*` is called the dereference operator (when working with pointers). It operates on a pointer and gives the value stored in that pointer.

# C Pointers

## Changing Value Pointed by Pointers

Let's take an example.

```
int* pc, c;
c = 5;
pc = &c;
c = 1;
printf("%d", c); // Output: 1
printf("%d", *pc); // Output: 1
```

We have assigned the address of `c` to the `pc` pointer.

Then, we changed the value of `c` to 1. Since `pc` and the address of `c` is the same, `*pc` gives us 1.

## C Pointers

Let's take another example.

```
int* pc, c;
c = 5;
pc = &c;
*pc = 1;
printf("%d", *pc); // Output: 1
printf("%d", c); // Output: 1
```

We have assigned the address of `c` to the `pc` pointer.

Then, we changed `*pc` to 1 using `*pc = 1;`. Since `pc` and the address of `c` is the same, `c` will be equal to 1.

Let's take one more example.



# C Pointers

```
int* pc, c, d;
c = 5;
d = -15;

pc = &c; printf("%d", *pc); // Output: 5
pc = &d; printf("%d", *pc); // Output: -15
```

Initially, the address of `c` is assigned to the `pc` pointer using `pc = &c;`. Since `c` is 5, `*pc` gives us 5.

Then, the address of `d` is assigned to the `pc` pointer using `pc = &d;`. Since `d` is -15, `*pc` gives us -15.

# C Pointers

```
#include <stdio.h>
int main()
{
 int* pc, c;

 c = 22;
 printf("Address of c: %p\n", &c);
 printf("Value of c: %d\n\n", c); // 22

 pc = &c;
 printf("Address of pointer pc: %p\n", pc);
 printf("Content of pointer pc: %d\n\n", *pc); // 22

 c = 11;
 printf("Address of pointer pc: %p\n", pc);
 printf("Content of pointer pc: %d\n\n", *pc); // 11

 *pc = 2;
 printf("Address of c: %p\n", &c);
 printf("Value of c: %d\n\n", c); // 2
 return 0;
}
```

Address of c: 2686784  
Value of c: 22

Address of pointer pc: 2686784  
Content of pointer pc: 22

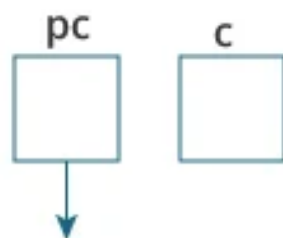
Address of pointer pc: 2686784  
Content of pointer pc: 11

Address of c: 2686784  
Value of c: 2

# C Pointers

## Explanation of the program

1. `int* pc, c;`

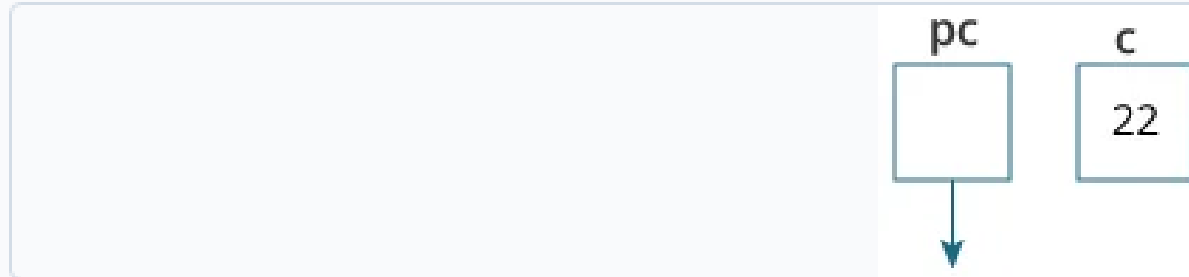


Here, a pointer `pc` and a normal variable `c`, both of type `int`, is created.

Since `pc` and `c` are not initialized at initially, pointer `pc` points to either no address or a random address. And, variable `c` has an address but contains random garbage value.

# C Pointers

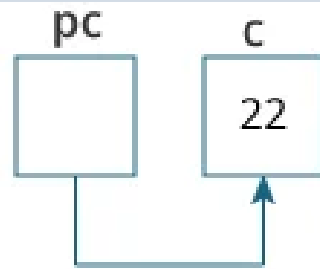
2. `c = 22;`



This assigns 22 to the variable `c`. That is, 22 is stored in the memory location of variable `c`.

# C Pointers

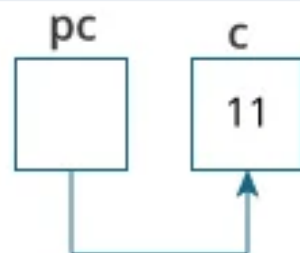
3. `pc = &c;`



This assigns the address of variable `c` to the pointer `pc`.

# C Pointers

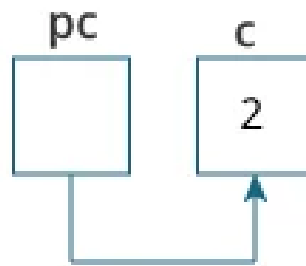
4. `c = 11;`



This assigns 11 to variable `c`.

# C Pointers

5. `*pc = 2;`



This change the value at the memory location pointed by the pointer `pc` to 2.

# Initializing Pointers

- ▶ Like other variables, always initialize pointers before using them!!!
- ▶ For example:

```
int main(){
 int x;
 int *p;
 scanf("%d",p); /* incorrect */
 p = &x;
 scanf("%d",p); /* Correct */
}
```



# Using Pointers

- ▶ You can use pointers to access the values of other variables, *i.e.* the contents of the memory for other variables.
- ▶ To do this, use the **\*** operator (dereferencing operator).
  - Depending on different context, **\*** has different meanings.
- ▶ For example:

```
int n, m=3, *p;
```

```
p=&m;
```

```
n=*p;
```

```
printf("%d\n", n); // 3
```

```
printf("%d\n", *p); //3
```

# Example

```
#include <stdio.h>
int main()
{
 int a=5, b=10;
 int *p;
 p = &a;
 printf("\na=%d b=%d *p=%d", a, b,*p);
 p=&b;
 printf("\na=%d b=%d *p=%d", a, b,*p);
 return 0;
}
```

## Output:

a=5 b=10 \*p=5

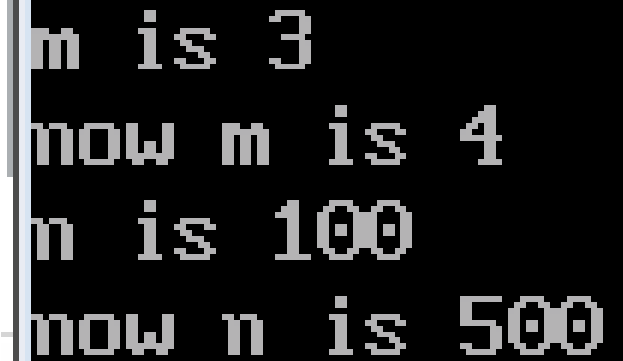
a=5 b=10 \*p=10

- The following statements are also valid.
- `*ptr = *ptr + 10;`
- increments `*ptr` by 10.

```
void main()
{
 int i=10;
 int *p;
 clrscr();
 p=&i;
 printf("value of i:%d\n",i);
 printf("value of i:%d\n",*(&i));
 getch();
}
```

```
value of i:10
value of i:10
```

```
int m=3, n=100, *p;
p=&m;
printf("m is %d\n",*p);
m++;
printf("now m is %d\n",*p);
p=&n;
printf("n is %d\n",*p);
p=500; / *p is at the left of "=" */
printf("now n is %d\n", n);
```



```
m is 3
now m is 4
n is 100
now n is 500
```

# Void or Generic pointer-

- A void pointer is a special type of pointer. It can point to any data type, from an integer value or a float to a string of characters.
- Its sole limitation is that the pointed data cannot be referenced directly (the asterisk \* operator cannot be used on them) since its length is always undetermined.
- Therefore, *type casting or assignment* must be used to turn the void pointer to a pointer of a concrete data type to which we can refer

# Example-

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int a=5;
 double b=3.1415;
 void *vp;
 vp=&a;
 printf("a=%d\n",*((int *)vp));
 vp=&b;
 printf("b=%f",*((double *)vp));
 getch();
}
```

# Output-

**Output:**

a= 5

b= 3.141500



# NULL Pointer-

- **NULL pointer in C-A null pointer is a pointer which points nothing.)**
- *A null pointer is a special pointer that points nowhere.*
- That is, no other valid pointer to any other variable or array cell or anything else will ever be equal to a null pointer.
- The most straightforward way to get a null pointer in the program is by using the predefined constant NULL,

# Continue..

- If it is too early in the code to know which address to assign to the pointer,
- then the pointer can be assigned to NULL, which is a constant with a value of zero defined in several standard libraries, including `stdio.h`.

# Example-

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int *p;
 p=NULL;
 void *vp;
 clrscr();
 printf("value of p is %p",p);
 getch();
}
```

## Output:

The value of p is 0

# Use of Pointers-

- Call by address-One of the typical applications of pointers is to support call by reference.
- C makes use of only one mechanism to communicate arguments to a function: call by value.
- This means that when a function is called, a copy of the values of arguments is created and given to the function.

## Call by Value and Call by Reference in C

### Introduction

Suppose you have a file and someone wants the information present in the file. So to protect from alteration in the original file, you give a copy of your file to them and if you want the changes done by someone else in your file then you have to give them your original file. In C also, if we want the changes done by function to reflect in the original **parameters** also, then we pass the parameter by reference, and if we don't want the changes in the original parameter, then we pass the parameters by value.

### Call by Value in C

Calling a function by value will cause the program to copy the contents of an object passed into a function. To implement this in C, a function declaration has the following form: **[return type] functionName([type][parameter name],...)**.

## Call by Value Example: Swapping the values of the two variables

```
#include<stdio.h>
#include<conio.h>
void swap(int,int);
void main()
{
 int x=10,y=11;
 clrscr();
 printf("values before swap: x=%d and y=%d\n",x,y);
 swap(x,y);
 printf("values after swap: x=%d and y=%d\n",x,y);
 getch();
}
```

```
void swap(int x,int y)
{
 int temp;
 temp=x;
 x=y;
 y=temp;
}
```

```
values before swap: x=10 and y=11
values after swap: x=10 and y=11
```

### Call by Value Example: Swapping the values of the two variables

We can observe that even when we change the content of `x` and `y` in the scope of the `swap` function, these changes do not reflect on `x` and `y` variables defined in the scope of `main`. This is because we call `swap()` by value and it will get separate memory for `x` and `y` so the changes made in `swap()` will not reflect in `main()`.

## Call by Reference in C

Calling a function by reference will give function parameter the address of original parameter due to which they will point to same memory location and any changes made in the function parameter will also reflect in original parameters. To implement this in C, a function declaration has the following form: `[return type] functionName([type]* [parameter name],...).`



```
#include<stdio.h>
#include<conio.h>
void swap(int *x,int *y);
void main()
{
int x=10,y=11;
clrscr();
printf("values before swap: x=%d and y=%d\n",x,y);
swap(&x,&y);
printf("values after swap: x=%d and y=%d\n",x,y);
getch();
}
```

```
void swap(int *x,int *y)
{
 int temp;
 temp=*x;
 *x=*y;
 *y=temp;
}
```

values before swap: x=10 and y=11  
values after swap: x=11 and y=10

We can observe in function parameters instead of using `int x, int y` we used `int *x, int *y` and in function call instead of giving `x, y`, we give `&x, &y` this methodology is called by reference as we used pointers as function parameter which will get original parameters' address instead of their value. `&` operator is used to give address of the variables and `*` is used to access the memory location that pointer is pointing. As the function variable is pointing to the same memory location as the original parameters, the changes made in `swap()` reflect in `main()` which we can see in the above output.

## Difference Between Call by Value and Call by Reference in C

| Calling by Value                                                                                                                         | Calling by Reference                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Copies the value of an object.                                                                                                           | Pass a pointer that contains the memory address of an object that gives access to its contents.      |
| Guarantees that changes that alter the state of the parameter will only affect the named parameter bounded by the scope of the function. | Changes that alter the state of the parameter will reflect to the contents of the passed object.     |
| Simpler to implement and simpler to reason with.                                                                                         | More difficult to keep track of changing values that happens for each time a function may be called. |

## Returning pointer from a function-

- It is also possible to return a pointer from a function. When a pointer is returned from a function, it must point to data in the calling function or in the global variable.

# Example-

```
#include<stdio.h>
#include<conio.h>
int *max(int *,int *);
void main()
{
 int a,b,*p;
 clrscr();
 printf("\nenter a:");
 scanf("%d",&a);
 printf("\nenter b:");
 scanf("%d",&b);
 p=max(&a,&b);
 printf("\np is:%d",*p);
 getch();
}
int *max(int *x,int *y)
{
 if(*x>*y)
 return x;
 else
 return y;
}
```

# Continue..

**Output:**

a = 5

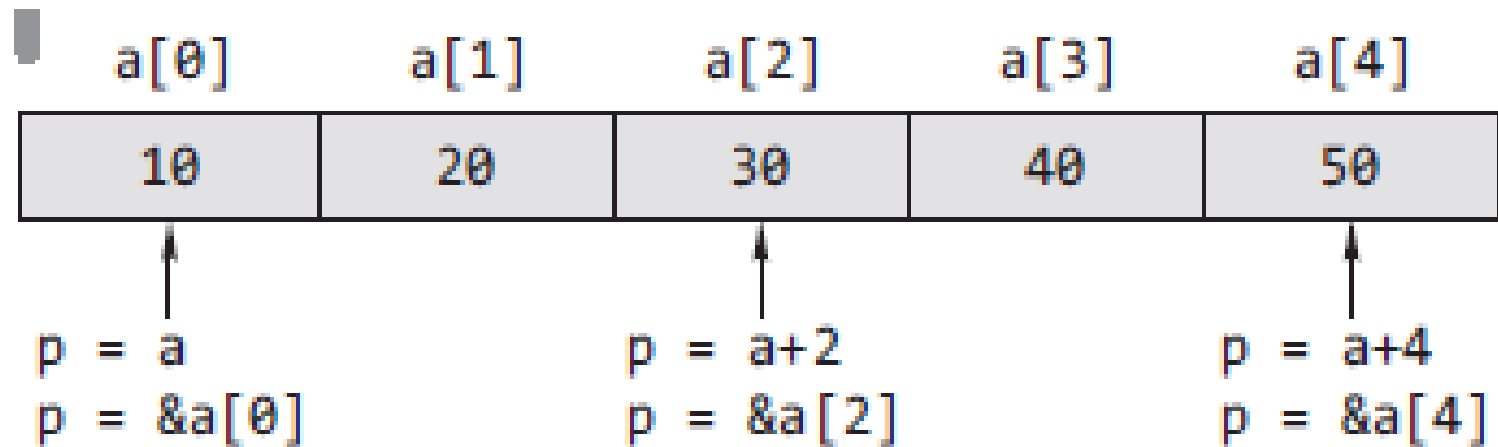
b = 7

p = 7

# Array and Pointers Basics-

- The expression  $a + i$  (with integer  $i$ ) means the address of the  $i$ th element beyond the one  $a$  points to.
- As indirection operator '\*' implies value at address,  $a[i]$  is equivalent to  $*(a+i)$ .
- The integer identifier  $i$  is added to the base address of the array.

## Pointer Notation of array elements-





# Example-

```
#include <stdio.h>
int main()
{
int a[]={10, 20, 30, 40, 50};
int i;
for(i=0;i<5;++i)
printf("\n%d", a[i]);
return 0;
}
```

## **Output:**

```
10
20
30
40
50
```

# Example 2-

```
#include <stdio.h>

int main()
{
 int a[]={10, 20, 30, 40, 50};
 int i;
 for(i=0;i<5;++i)
 printf("\n%d", *(a+i));
 return 0;
}
```

## **Output:**

```
10
20
30
40
50
```

# Array and Pointers Basics-

- All the following expressions are the same when their addresses are considered.
- $a[i]$
- $*(a + i)$
- $*(i + a)$

# Array and Pointers Basics-

- For any one-dimensional array `a` and integer `i`, the following relationships are always true.
- **`&a[0] == a`**
- `&a[i] == a + i`
- `a[i] == *(a + i)`

# Example-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a[]={10, 20, 30, 40, 50};
```

```
int i, *p;
```

```
p=a; /* it can also be written as p=&a[0]; */
```

```
for(i=0;i<5;++i)
```

```
printf("\n%d", p[i]);
```

```
OR
```

```
printf ("\n%d", *(p+i));
```

```
OR
```

```
printf ("\n%d", *(i+p));
```

```
OR
```

```
printf ("\n%d", i[p]);
```

```
return 0;
```

```
}
```

## Output:

10

20

30

40

50

# Pointers & Arrays

You can also use pointers to access arrays.

Consider the following array of integers:

## Example

```
int myNumbers[4] = {25, 50, 75, 100};
```

## Example

```
int myNumbers[4] = {25, 50, 75, 100};
int i;

for (i = 0; i < 4; i++) {
 printf("%d\n", myNumbers[i]);
}
```

Result:

```
25
50
75
100
```

Instead of printing the value of each array element, let's print the memory address of each array element:

## Example

```
int myNumbers[4] = {25, 50, 75, 100};
int i;

for (i = 0; i < 4; i++) {
 printf("%p\n", &myNumbers[i]);
}
```

Result:

```
0x7ffe70f9d8f0
0x7ffe70f9d8f4
0x7ffe70f9d8f8
0x7ffe70f9d8fc
```

# How Are Pointers Related to Arrays

Ok, so what's the relationship between pointers and arrays? Well, in C, the **name of an array**, is actually a **pointer** to the **first element** of the array.

Confused? Let's try to understand this better, and use our "memory address example" above again.

The **memory address** of the **first element** is the same as the **name of the array**:

## Example

```
int myNumbers[4] = {25, 50, 75, 100};

// Get the memory address of the myNumbers array
printf("%p\n", myNumbers);

// Get the memory address of the first array element
printf("%p\n", &myNumbers[0]);
```

Result:

```
0x7ffe70f9d8f0
0x7ffe70f9d8f0
```



This basically means that we can work with arrays through pointers!

How? Since myNumbers is a pointer to the first element in myNumbers, you can use the `*` operator to access it:

## Example

```
int myNumbers[4] = {25, 50, 75, 100};

// Get the value of the first element in myNumbers
printf("%d", *myNumbers);
```

Result:

25

To access the rest of the elements in myNumbers, you can increment the pointer/array (+1, +2, etc):

## Example

```
int myNumbers[4] = {25, 50, 75, 100};

// Get the value of the second element in myNumbers
printf("%d\n", *(myNumbers + 1));

// Get the value of the third element in myNumbers
printf("%d", *(myNumbers + 2));

// and so on..
```

Result:

```
50
75
```

Or loop through it:

## Example

```
int myNumbers[4] = {25, 50, 75, 100};
int *ptr = myNumbers;
int i;

for (i = 0; i < 4; i++) {
 printf("%d\n", *(ptr + i));
}
```

Result:

```
25
50
75
100
```

It is also possible to change the value of array elements with pointers:

## Example

```
int myNumbers[4] = {25, 50, 75, 100};

// Change the value of the first element to 13
*myNumbers = 13;

// Change the value of the second element to 17
*(myNumbers + 1) = 17;

// Get the value of the first element
printf("%d\n", *myNumbers);

// Get the value of the second element
printf("%d\n", *(myNumbers + 1));
```

Result:

13

17

```
#include <stdio.h>
int main() {
 int x[4];
 int i;

 for(i = 0; i < 4; ++i) {
 printf("&x[%d] = %p\n", i, &x[i]);
 }

 printf("Address of array x: %p", x);

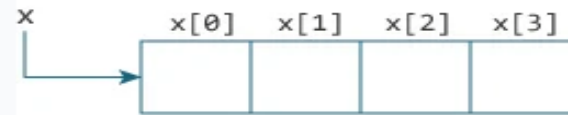
 return 0;
}
```

## Output

```
&x[0] = 1450734448
&x[1] = 1450734452
&x[2] = 1450734456
&x[3] = 1450734460
Address of array x: 1450734448
```

There is a difference of 4 bytes between two consecutive elements of array `x`. It is because the size of `int` is 4 bytes (on our compiler).

Notice that, the address of `&x[0]` and `x` is the same. It's because the variable name `x` points to the first element of the array.



Relation between Arrays and Pointers

From the above example, it is clear that `&x[0]` is equivalent to `x`. And, `x[0]` is equivalent to `*x`.

Similarly,

- `&x[1]` is equivalent to `x+1` and `x[1]` is equivalent to `*(x+1)`.
- `&x[2]` is equivalent to `x+2` and `x[2]` is equivalent to `*(x+2)`.
- ...
- Basically, `&x[i]` is equivalent to `x+i` and `x[i]` is equivalent to `*(x+i)`.

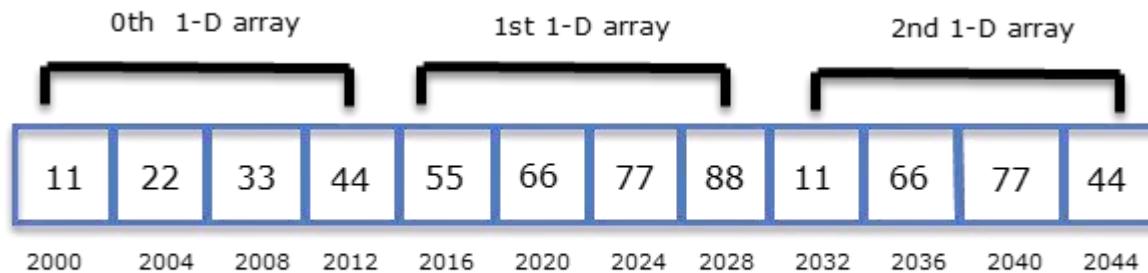
## Sum of array elements using pointer

```
void main()
{
 int i,x[6],sum=0;
 printf("enter 6 elements\n");
 for(i=0;i<6;i++)
 {
 //same as scanf("%d",&x[i]);
 scanf("%d",x+i);
 //same as sum=sum+x[i];
 sum=sum+*(x+i);
 }
 printf("sum is %d",sum);
 getch();
}
```

# Pointer and 2-d array

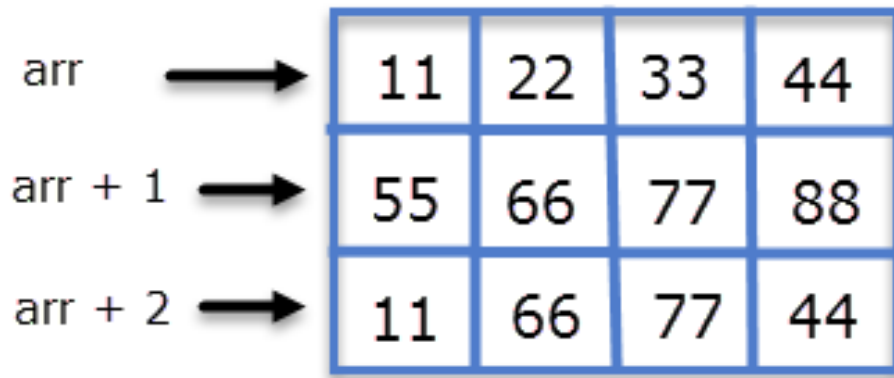
```
int arr[3][4] = {
 {11,22,33,44},
 {55,66,77,88},
 {11,66,77,44}
};
```

|       | Col 0 | Col 1 | Col 2 | Col 3 |
|-------|-------|-------|-------|-------|
| Row 0 | 11    | 22    | 33    | 44    |
| Row 1 | 55    | 66    | 77    | 88    |
| Row 2 | 11    | 66    | 77    | 44    |



`arr` points to 0th 1-D array.  
`(arr + 1)` points to 1st 1-D array.  
`(arr + 2)` points to 2nd 1-D array.

# Pointer and 2-d array



Let's see how we can do this:

`*(arr + i)` points to the address of the 0th element of the 1-D array. So,

`*(arr + i) + 1` points to the address of the 1st element of the 1-D array

`*(arr + i) + 2` points to the address of the 2nd element of the 1-D array

Hence we can conclude that:

`*(arr + i) + j` points to the base address of jth element of ith 1-D array.

On dereferencing `*(arr + i) + j` we will get the value of jth element of ith 1-D array.

```
*(*(arr + i) + j)
```



# Pointers and Strings-

- Strings are one-dimensional arrays of type char. By convention, a string in C is terminated by the end-of string sentinel `\0`, or null character.

# Pointers and Strings-

## Creating a Pointer for the String

When we create an array, the variable name points to the address of the first element of the array. The other way to put this is the variable name of the array points to its starting position in the memory.

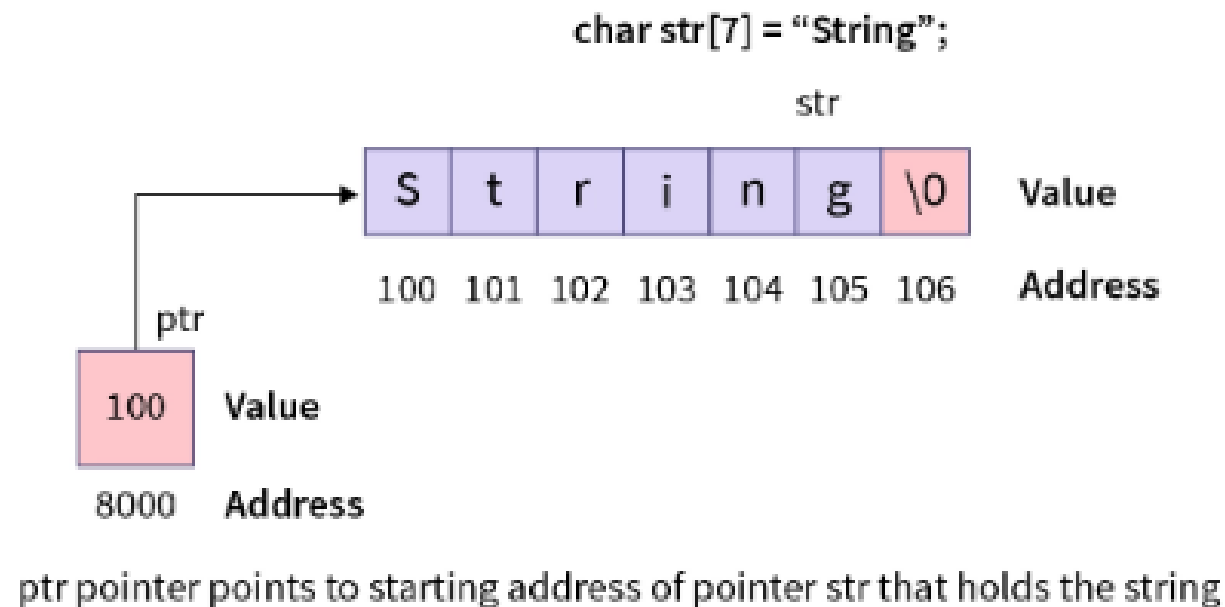
We can create a character pointer to string in C that points to the starting address of the character array. This pointer will point to the starting address of the string, that is **the first character of the string**, and we can dereference the pointer to access the value of the string.

```
// character array storing the string 'String'
char str[7] = "String";
// pointer storing the starting address of the
// character array str
char *ptr = str;
```



# Pointers and Strings-

**Note:** Pointer and array are not the same, and here pointer stores the starting address of the array, and it can be dereferenced to access the value stored in the address.



# Pointers and Strings-

## Accessing String via a Pointer

An array is a contiguous block of memory, and when pointers to string in C are used to point them, the pointer stores the starting address of the array. Similarly, when we point a `char` array to a pointer, we pass the base address of the array to the pointer. The pointer variable can be dereferenced using the asterisk `*` symbol in C to get the character stored in the address. For example,

```
char arr[] = "Hello";
// pointing pointer ptr to starting address
// of the array arr
char *ptr = arr;
```



# Pointers and Strings-

In this case, `ptr` points to the starting character in the array `arr` i.e. `H`. To get the value of the first character, we can use the `*` symbol, so the value of `*ptr` will be `H`. Similarly, to get the value of `ith` character, we can add `i` to the pointer `ptr` and dereference its value to get `ith` character, as shown below.

```
printf("%c ", *ptr); // H
printf("%c ", *(ptr + 1)); // e
printf("%c ", *(ptr + 2)); // l
printf("%c ", *(ptr + 3)); // l
printf("%c ", *(ptr + 4)); // o
```



Instead of incrementing the pointer manually to get the value of the string, we can use a simple fact that our string terminates with a null `\0` character and use a `while` loop to increment the pointer value and print each character till our pointer points to a null character.

# Pointers and Strings-

```
#include<stdio.h>
#include<conio.h>
void main()
{
 char str[100];
 char *ptr;
 clrscr();
 printf("enter string");
 gets(str);
 ptr=str;
 while(*ptr!='\0')
 {
 printf("%c",*ptr);
 ptr++;
 }
 getch();
}
```

Write a program in C to find string length using pointer.

```
#include <stdio.h>

int stringLength(char*);

int main(){
 char str[100];
 int len;
 printf("Enter any String:");
 gets(str);
 len=stringLength(str);
 printf("The length of string %s is %d",str,len);
 return 0;
}

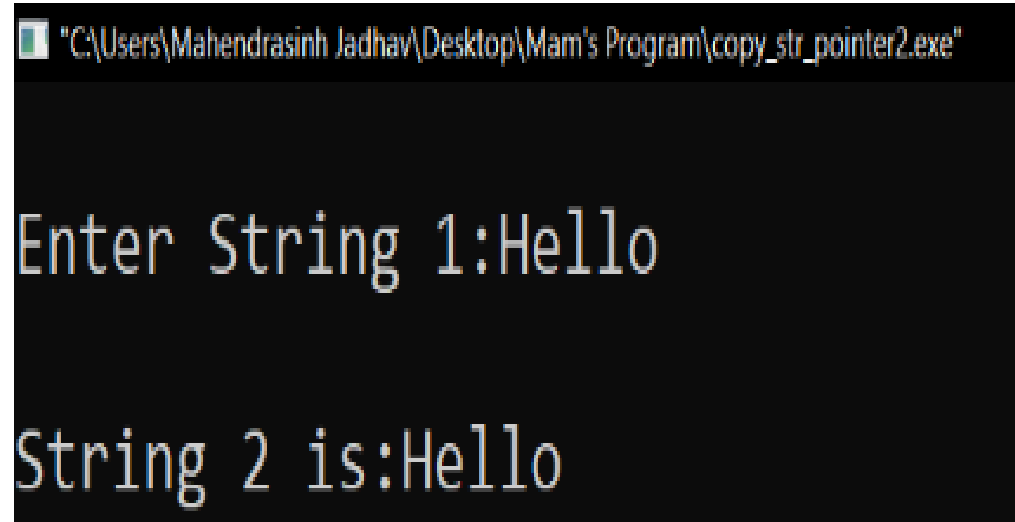
int stringLength(char *p){
 int count=0;
 while(*p!='\0') {
 count++;
 p++;
 }
 return count;
}
```

program to copy character array to the character array using pointer.

```
#include <stdio.h>
#include <conio.h>
void main()
{
 char s1[10],s2[10],*p1,*p2;
 p1 = s1;
 p2 = s2;
 printf("\nEnter String 1:");
 gets(s1);
 while(*p1!='\0')
 {
 *p2 = * p1;
 p1++;
 p2++;
 }
 *p2 = '\0';
 printf("\nString 2 is:");
 puts(s2);
}
```



# Output-



```
"C:\Users\Mahendrasinh Jadhav\Desktop\Mam's Program\copy_str_pointer2.exe"

Enter String 1:Hello

String 2 is:Hello
```

A screenshot of a Windows command prompt window. The title bar at the top shows the file path: "C:\Users\Mahendrasinh Jadhav\Desktop\Mam's Program\copy\_str\_pointer2.exe". The command prompt has a black background with white text. It displays the prompt "Enter String 1:" followed by the input "Hello". Below that, it displays "String 2 is:" followed by the output "Hello".

# program concating two Strings using Pointer.

```
#include <stdio.h>
#include <conio.h>
void main()
{
 char str1[10],str2[10],str3[10],*p1,*p2,*p3;
 p1 = str1;
 p2 = str2;
 p3 = str3;
 printf("Enter String 1:");
 gets(str1);
 printf("\nEnter String2:");
 gets(str2);
```

```
while(*p1!='\0')
{
 *p3 = *p1;
 p3++;
 p1++;
}
while(*p2!='\0')
{
 *p3 = *p2;
 p3++;
 p2++;
}
*p3 = '\0';
printf("\nCombine string is:");
puts(str3);
getch();
}
```

# Output-

"C:\Users\Mahendrasinh Jadhav\Desktop\Mam's Program\concat\_str\_pointer.exe"

Enter String 1:He

Enter String2:llo

Combine string is:Hello

—

Program to read and print a string and also count the number of characters, words and lines in the line.

```
#include <stdio.h>
#include <conio.h>
void main()
{
 char str[100],*pstr;
 int chars = 1,lines = 1,words = 1;
 pstr = str;
 printf("Enter the String:");
 scanf("%[^~]",&str);
 while(*pstr != '\0')
 {
 if(*pstr != '\n')
 {
 lines++;
 }
 }
```

```
if(*pstr == ' ' && *(pstr+1)!=' ')
 {
 words++;
 }
chars++;
pstr++;
}
printf("\nThe String is:");
puts(str);
printf("\nThe number of characters:%d",chars);
printf("\nThe number of lines:%d",lines);
printf("\nThe number of words:%d",words);
getch();
}
```

# Output-

Enter the String: How are you

The String is: How are you

The number of characters:11

The number of lines:1

The number of words:3

program which reads a string and that scans each character to count the number of uppercase and lowercase characters entered.

```
#include <stdio.h>
#include <conio.h>
void main()
{
 char str[100],*pstr;
 int upper=0,lower=0;
 printf("\nEnter the String:");
 gets(str);
 pstr = str;
 while(*pstr != '\0')
 {
 if(*pstr>= 'A' && *pstr<='Z')
 {
 upper++;
 }
 }
```



```
else if(*pstr>= 'a' && *pstr<='z')
{
 lower++;
}
pstr++;
}
printf("\nThe total number of uppercase
characters:%d",upper);
printf("\nThe total number of lowercase
characters:%d",lower);
getch();
}
```

# Output-

```
"C:\Users\Mahendrasinh Jadhav\Desktop\Mam's Program\count_upper_lower_point.exe"

Enter the String:Hello How Are You

The total number of uppercase characters:4
The total number of lowercase characters:10_
```

## C - Pointer arithmetic

A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -

To understand pointer arithmetic, let us consider that **ptr** is an integer pointer which points to the address 1000. Assuming 32-bit integers, let us perform the following arithmetic operation on the pointer –

```
ptr++
```

After the above operation, the **ptr** will point to the location 1004 because each time ptr is incremented, it will point to the next integer location which is 4 bytes next to the current location. This operation will move the pointer to the next memory location without impacting the actual value at the memory location. If **ptr** points to a character whose address is 1000, then the above operation will point to the location 1001 because the next character will be available at 1001.

## Incrementing a Pointer

We prefer using a pointer in our program instead of an array because the variable pointer can be incremented, unlike the array name which cannot be incremented because it is a constant pointer. The following program increments the variable pointer to access each succeeding element of the array –

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int var[]={10,20,30};
 int i, *ptr;
 clrscr();
 ptr=var;
 for(i=0;i<3;i++)
 {
 printf("address of var[%d]=%p\n",i,ptr);
 printf("value of var[%d]=%d\n",i,*ptr);
 //move ptr to next location
 ptr++;
 }
 getch();
}
```

```
address of var[0]=FFF0
value of var[0]=10
address of var[1]=FFF2
value of var[1]=20
address of var[2]=FFF4
value of var[2]=30
```

## Decrementing a Pointer

The same considerations apply to decrementing a pointer, which decreases its value by the number of bytes of its data type as shown below –

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int var[]={10,20,30};
 int i, *ptr;
 clrscr();
 ptr=&var[2];
 for(i=3;i>0;i--)
 {
 printf("address of var[%d]=%p\n",i-1,ptr);
 printf("value of var[%d]=%d\n",i-1,*ptr);
 //move ptr to next location
 ptr--;
 }
 getch();
}
```

```
address of var[2]=FFF4
value of var[2]=30
address of var[1]=FFF2
value of var[1]=20
address of var[0]=FFF0
value of var[0]=10
```

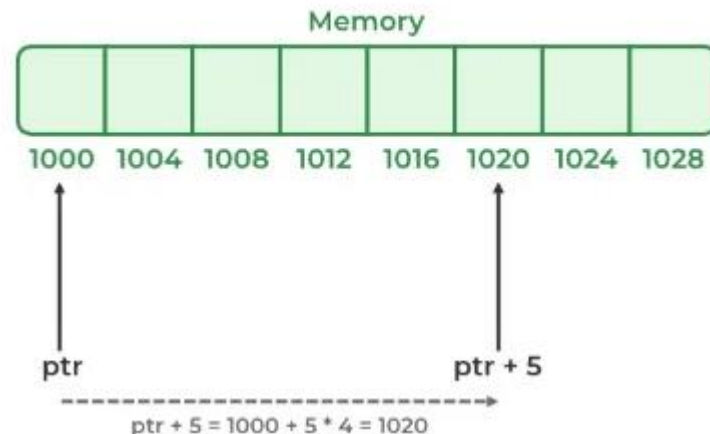
## 2. Addition of Integer to Pointer

When a pointer is added with an integer value, the value is first multiplied by the size of the data type and then added to the pointer.

**For Example:**

Consider the same example as above where the `ptr` is an **integer pointer** that stores **1000** as an address. If we add integer 5 to it using the expression, `ptr = ptr + 5`, then, the final address stored in the `ptr` will be `ptr = 1000 + sizeof(int) * 5 = 1020`.

### Pointer Addition



## C - Pointer arithmetic

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int n=4;
 int *ptr2;
 clrscr();
 ptr2=&n;
 printf("pointer ptr2 before addition:%u\n",ptr2);
 ptr2=ptr2+3;
 printf("pointer ptr2 after addition:%u\n",ptr2);
 getch();
}
```

```
pointer ptr2 before addition:65524
pointer ptr2 after addition:65530
```

# C - Pointer arithmetic

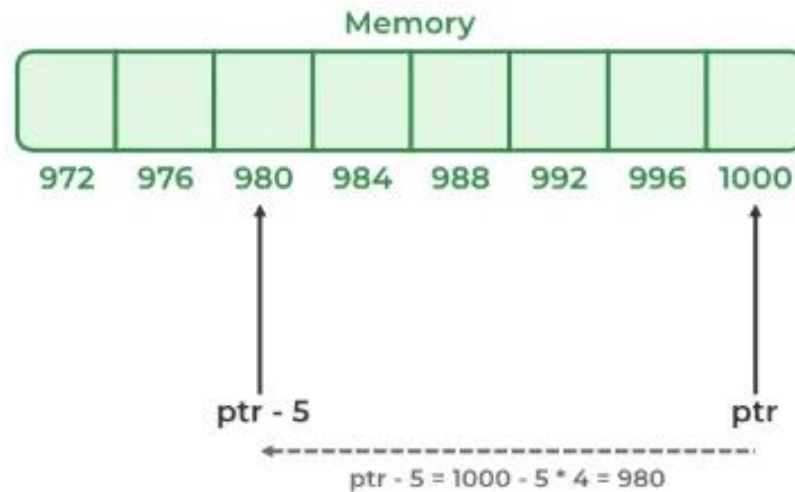
## 3. Subtraction of Integer to Pointer

When a pointer is subtracted with an integer value, the value is first multiplied by the size of the data type and then subtracted from the pointer similar to addition.

**For Example:**

Consider the same example as above where the **ptr** is an **integer pointer** that stores **1000** as an address. If we subtract integer 5 from it using the expression, **ptr = ptr - 5**, then, the final address stored in the ptr will be **ptr = 1000 - sizeof(int) \* 5 = 980**.

## Pointer Subtraction





## C - Pointer arithmetic

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int n=4;
 int *ptr2;
 clrscr();
 ptr2=&n;
 printf("pointer ptr2 before addition:%u\n",ptr2);
 ptr2=ptr2-3;
 printf("pointer ptr2 after addition:%u\n",ptr2);
 getch();
}
```

```
pointer ptr2 before addition:65524
pointer ptr2 after addition:65518
```

### 4. Subtraction of Two Pointers

The subtraction of two pointers is possible only when they have the same data type. The result is generated by calculating the difference between the addresses of the two pointers and calculating how many bits of data it is according to the pointer data type. The subtraction of two pointers gives the increments between the two pointers.

#### For Example:

Two integer pointers say **ptr1(address:1000)** and **ptr2(address:1004)** are subtracted. The difference between addresses is 4 bytes. Since the size of int is 4 bytes, therefore the **increment between ptr1 and ptr2 is given by  $(4/4) = 1$ .**

### 5. Comparison of Pointers

We can compare the two pointers by using the comparison operators in C. We can implement this by using all operators in C `>`, `>=`, `<`, `<=`, `==`, `!=`. It returns true for the valid condition and returns false for the unsatisfied condition.

1. **Step 1:** Initialize the integer values and point these integer values to the pointer.
2. **Step 2:** Now, check the condition by using comparison or relational operators on pointer variables.
3. **Step 3:** Display the output.

## C - Pointer arithmetic

```
#include<stdio.h>
#include<conio.h>
void main()
{
 int arr[5];
 int *ptr1,*ptr2;
 clrscr();
 ptr1= arr;
 ptr2=&arr[0];
 if(ptr1==ptr2)
 printf("pointer to array name and first element are equal");
 else
 printf("pointer to array name and first element are not equal");
 getch();
}
```

pointer to array name and first element are equal\_

# Pointers to Pointers

- C allows the use of pointers that point to pointers, and these, in turn, point to data. For pointers to do that, we only need to add an asterisk (\*) for each level of reference. Consider the following declaration.

```
int a=5;
```

```
int *p <-pointer to an integer
```

```
int **q <-pointer to a pointer to an integer
```

```
p=&a;
```

```
q=&p;
```

# Example-

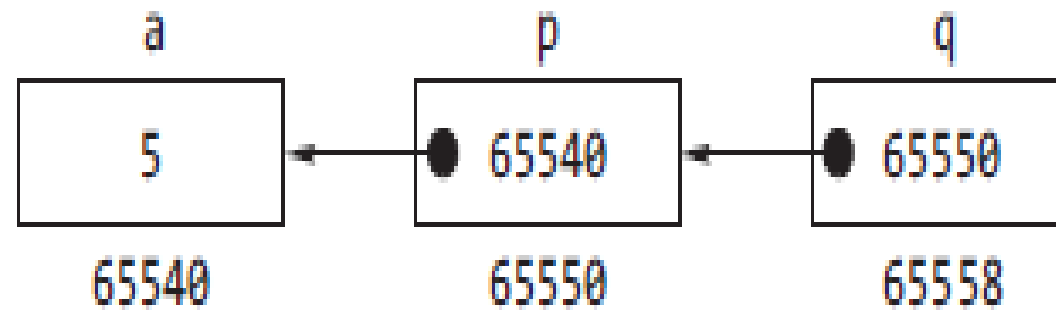
```
#include <stdio.h>
int main()
{
 int a=5;
 int *p,**q;
 p=&a;
 q=&p;
 printf("\n *p=%d",*p);
 printf("\n **q=%d",**q);
 return 0;
}
```

**Output:**

\*p=5

\*\*q=5

Continue..



# Dynamic Memory Allocation-

- The process of allocating memory to the variable during execution of the program or at runtime is known as dynamic memory allocation.
- C language has four library routines for dynamic memory allocation – malloc, calloc, free and realloc.



# *Static memory allocation-*

- ***The compiler allocates the required*** memory space for a declared variable. By using the address of operator, the reserved address is obtained that may be assigned to a pointer variable.
- Since most declared variables have static memory, this way of assigning pointer value to a pointer variable is known as static memory allocation.

# *Dynamic Memory Allocation(DMA)-*

- A dynamic memory allocation uses functions such as malloc() or calloc() to get memory dynamically.
- If these functions are used to get memory dynamically and the values returned by these functions are assigned to pointer variables, such assignments are known as dynamic memory allocation.
- Memory is assigned during run-time.

## Memory allocation and de-allocation function-

| Function  | Task                                                                                                                                |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| malloc()  | Allocates memory and returns a pointer to the first byte of allocated space                                                         |
| calloc()  | Allocates space for an array of elements and initialize them to zero. Like malloc(), calloc() also returns a pointer to the memory. |
| free()    | Frees previously allocated memory                                                                                                   |
| realloc() | Alters the size of previously allocated memory                                                                                      |

# Points to remember-

- In dynamic memory allocation, memory is allocated at runtime from heap.
- According to ANSI compiler, the block pointer returned by allocating function is a void pointer.
- If sufficient memory is not available, the malloc() and calloc() returns a NULL.
- calloc() initializes all the bits in the allocated space set to zero where as malloc() does not do this.
- When dynamically allocated, arrays are no longer needed, it is recommended to free them immediately.

# Allocating a block of memory-malloc()

The malloc() function reserves a block of memory of specified size and returns a pointer of type void.

Syntax-

```
ptr= (cast-type*)malloc(byte-size);
```

where ptr is a pointer of type cast-type. The malloc() returns a pointer (of cast type) to an area of memory with size byte-size. For e.g.

```
arr= (int*)malloc(10*sizeof(int));
```

# Continue..

- This statement is equivalent to 10 times the area of int bytes.
- On successful execution of the statement the space is reserved and the address of the first byte of memory allocated is assigned to pointer arr of type int.

# Example -1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void main()
```

```
{
```

```
int N,*a,i;
```

```
printf("\n enter no. of elements of the array:");
```

```
scanf("%d",&N);
```

```
a=(int *)malloc(N*sizeof(int));
```

```
if(a==NULL)
```

```
{
```

```
printf("\n memory allocation unsuccessful...");
```

```
exit(0);
```

```
}
```

# Continue..

```
printf("\n enter the array elements one by one");
for(i=0; i<N;++i)
{
scanf("%d",&a[i])); /* equivalent statement scanf("%d",(a+i));*/
}
printf("\n array elements are");
for(i=0; i<N;++i)
{
printf("%d",a[i])); //equivalent statement printf("%d",*(a+i));
}
}
```



## Example -2

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void main()
```

```
{
```

```
int N,*a,i,s=0;
```

```
printf("\n enter no. of elements of the array:");
```

```
scanf("%d",&N);
```

```
a=(int *)malloc(N*sizeof(int));
```

```
if(a==NULL)
```

```
{
```

```
printf("\n memory allocation unsuccessful...");
```

```
exit(0);
```

```
}
```

# Continue..

```
printf("\n enter the array elements one by one");
for(i=0; i<N;++i)
{
scanf("%d",&a[i])); /*equivalent statement scanf("%d",(a+i));
s+=a[i];
}
printf("\n sum is %d ",s);
}
```

# Allocating a block of memory-calloc()

- The function `calloc ()` is another function that reserve memory at the runtime. It is normally used to request multiple blocks of storage each of same size and then sets all bytes to zero.
- It is used to allocate memory of an array.

# Continue..

Syntax-

```
ptr= (cast-type*)calloc(n,elem-size);
```

The above statement allocates contiguous space for n blocks each size of elem-size bytes.

# Example -

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void main()
```

```
{
```

```
int N,*a,i;
```

```
printf("\n enter no. of elements of the array:");
```

```
scanf("%d",&N);
```

```
a=(int *)calloc(N,sizeof(int));
```

```
if(a==NULL)
```

```
{
```

```
printf("\n memory allocation unsuccessful...");
```

```
exit(0);
```

```
}
```

# Continue..

```
printf("\n enter the array elements one by one");
for(i=0; i<N;++i)
{
scanf("%d",&a[i])); /* equivalent statement scanf("%d",(a+i));*/
}
printf("\n array elements are");
for(i=0; i<N;++i)
{
printf("%d",a[i])); //equivalent statement printf("%d",*(a+i));
}
free(a);
}
```

# Releasing the used space- free()

- When a variable is allocated space during the compile time, then the memory used by the variable is automatically released by the system in accordance with its storage class.
- But when we dynamically allocate memory then it is our responsibility to release the space when it is not required.

# Continue..

- The general syntax of the free is-

`free(ptr);`

- When memory is de allocated using free , it is returned back to the free list within the heap.



To alter the size of allocated memory-`realloc()`

- At times the memory allocated by using `calloc()` or `malloc()` might be insufficient or in excess. In both the situations we can always use `realloc()` to change the memory size already allocated by `calloc()` or `malloc()`.
- The process is called *reallocation* of memory.

Syntax for `realloc()`-

```
ptr= realloc(ptr,newsiz)
```

# Example -

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void main()
```

```
{
```

```
int N,*a,i;
```

```
printf("\n enter no. of elements of the array:");
```

```
scanf("%d",&N);
```

```
a=(int *)malloc(N*sizeof(int));
```

```
if(a==NULL)
```

```
{
```

```
printf("\n memory allocation unsuccessful...");
```

```
exit(0);
```

```
}
```

# Continue..

```
printf("\n enter the array elements one by one");
for(i=0; i<N;++i)
{
scanf("%d",&a[i])); /* equivalent statement scanf("%d",(a+i));*/
}
printf("\n array elements are");
for(i=0; i<N;++i)
{
printf("%d",a[i])); //equivalent statement printf("%d",*(a+i));
}
```

Continue..

```
printf("enter new size of an array);
scanf("%d",&N);
a=(int *)realloc(a,N*sizeof(int))
if(a==NULL)
{
printf("\n memory allocation unsuccessful...");
exit(0);
}
```

# Continue..

```
printf("\n enter the array elements one by one");
for(i=0; i<N;++i)
{
scanf("%d",&a[i])); /* equivalent statement scanf("%d",(a+i));*/
}
printf("\n array elements are");
for(i=0; i<N;++i)
{
printf("%d",a[i])); //equivalent statement printf("%d",*(a+i));
}
}
```

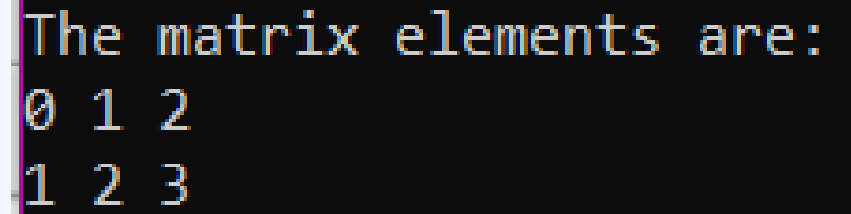
# Dynamic Allocation of 2D Array

We'll look at a few different approaches to creating a 2D array on the **heap** or dynamically allocate a 2D array.

## Using Single Pointer

A single pointer can be used to dynamically allocate a 2D array in C. This means that a memory block of size  $\text{row} \times \text{column} \times \text{dataTypeSize}$  is allocated using malloc, and the matrix elements are accessed using pointer arithmetic.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
 int row = 2, col = 3; //number of rows=2 and number of columns=3
 int *arr = (int *)malloc(row * col * sizeof(int));
 int i, j;
 for (i = 0; i < row; i++)
 for (j = 0; j < col; j++)
 *(arr + i*col + j) = i + j;
 printf("The matrix elements are:\n");
 for (i = 0; i < row; i++) {
 for (j = 0; j < col; j++) {
 printf("%d ", *(arr + i*col + j));
 }
 printf("\n");
 }
 free(arr);
 return 0;
}
```

A terminal window with a black background and yellow text. It displays the output of the C program, showing the matrix elements in a 2x3 grid.

```
The matrix elements are:
0 1 2
1 2 3
```

# Drawbacks of pointers-

Wild Pointer- Un-initialized pointer is pointing to an unknown memory location is known as wild pointer.

e.g.

## Example of Wild Pointers

```
#include <stdio.h>
int main()
{
 int *ptr;
 /* wild pointer */
 /* Random unknown memory location is being allocated. This should never be done. */
 *ptr = 12;
}
```

# Drawbacks of pointers-

**Note:** If a pointer `ptr` points to any known variable then it is no longer a wild pointer in C. We can understand this better with the below example,

```
#include <stdio.h>
int main()
{
 int *ptr;
 /* Ptr is a wild pointer in C */
 int a = 20;
 ptr = &a;
 /* ptr is no longer a wild pointer */
 *ptr = 12;
 /* The value of a is modified */
 printf("%d", a);
}
```

**Output**

12

In the above code, we can see that initially “ptr” was a wild pointer because it was declared and not initialized but as soon as we assigned ptr to a known address, it was no longer a wild pointer.



- Memory Leak- A *memory leak occurs when a dynamically allocated area* of memory is not released or when no longer needed.
- For example, if a function dynamically allocates memory for 100 double values and forgets to free the memory and in worst case if that function called several times within the code then ultimately the system may crash.

- Dangling pointer-

In C, a pointer may be used to hold the address of dynamically allocated memory. After this memory is freed with free() function, the pointer itself will still contain the address of the released block. This is referred as dangling pointer.

```
char *ptr1;
```

```
char *ptr2=(char *)malloc(sizeof (char));
```

```
ptr1=ptr2;
```

```
free(ptr2);
```

Now ptr1 becomes a dangling pointer.

- Memory Corruption-

Memory corruption often occurs when due to programming errors, the content of a memory location gets modified unintentionally. When program uses the contents of the corrupted memory, it either results in program crash or in storage.

# POINTER AND CONST QUALIFIER

- A declaration involving a pointer and const has several possible orderings

## **Pointer to Constant**

- The const keyword can be used in the declaration of the pointer when a pointer is declared to indicate that the value pointed to must not be changed. If a pointer is declared as follows

```
int n = 10;
```

```
const int *ptr=&n;
```

```
ptr = 100; / ERROR */
```

As the declaration asserted that what ptr points to must not be changed. But the following assignment is valid.

```
n = 50;
```

The value pointed to has changed but here it was not tried to use the pointer to make the change. Of course, the pointer itself is not constant, so it is always legal to change what it points to:

```
int v = 100;
```

```
ptr = &v; /* OK - changing the address in ptr */
```

This will change the address stored in ptr to point to the variable v.

# Constant Pointers

Constant pointers ensure that the address stored in a pointer cannot be changed.

```
int n = 10;
```

```
int *const ptr = &n; /* Defines a constant */
```

the second statement declares and initializes ptr and indicates that the address stored must not be changed.

# Continue..

Any attempt to change what the pointer points to elsewhere in the program will result in an error message

when you compile:

```
int v = 5;
```

```
ptr = &v; /* Error - attempt to change a constant pointer */
```

# C struct

In C programming, a struct (or structure) is a collection of variables (can be of different types) under a single name.

---

## Define Structures

Before you can create structure variables, you need to define its data type. To define a struct, the `struct` keyword is used.

## Define Structures

Before you can create structure variables, you need to define its data type. To define a struct, the `struct` keyword is used.

## Syntax of struct

```
struct structureName {
 dataType member1;
 dataType member2;
 ...
};
```



For example,

```
struct Person {
 char name[50];
 int citNo;
 float salary;
};
```

Here, a derived type `struct Person` is defined. Now, you can create variables of this type.

## Create struct Variables

When a `struct` type is declared, no storage or memory is allocated. To allocate memory of a given structure type and work with it, we need to create variables.

Here's how we create structure variables:

```
struct Person {
 // code
};

int main() {
 struct Person person1, person2, p[20];
 return 0;
}
```

Another way of creating a `struct` variable is:

```
struct Person {
 // code
} person1, person2, p[20];
```

In both cases,

- `person1` and `person2` are `struct Person` variables
- `p[]` is a `struct Person` array of size 20.

## Access Members of a Structure

There are two types of operators used for accessing members of a structure.

1. `.` - Member operator
2. `->` - Structure pointer operator

Suppose, you want to access the `salary` of `person2`. Here's how you can do it.

```
person2.salary
```

## Accessing the members of a structure-

- Access structure variables by the dot (.) operator

- Generic form:

`<structure_var>.<member_name>`

- For example:

`first.x = 50 ;`

`second.y = 100;`

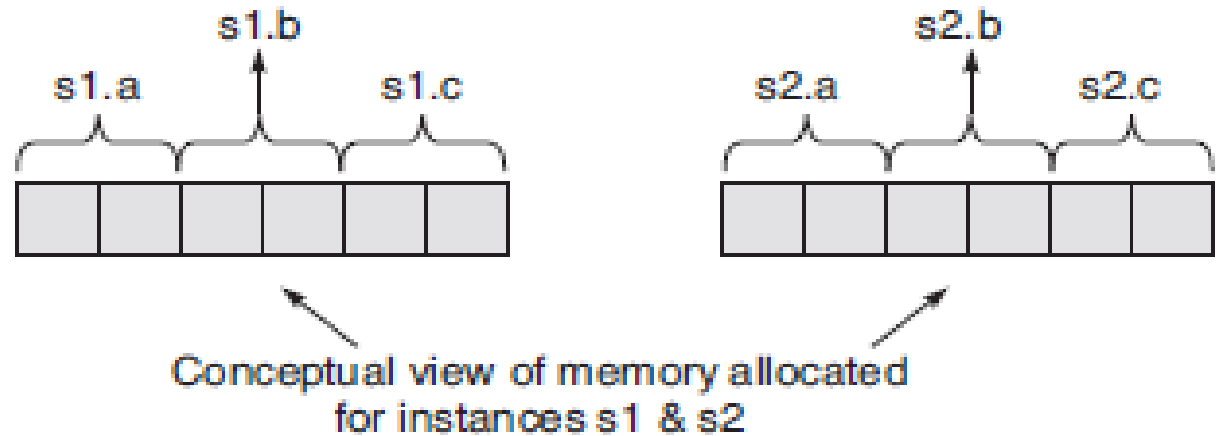
- `struct_var.member_name` can be used anywhere a variable can be used:

- `printf ("%d , %d", second.x , second.y );`
- `scanf("%d, %d", &first.x, &first.y);`

# Continue..

The . (dot) operator selects a particular member from a structure.

```
struct myStruct
{
 int a;
 int b;
 int c;
} s1, s2;
```



the first member can be accessed by the construct

s1.a

For second member-

s2.b = 12;

# Initialization of Structure-

- A structure can be initialized in much the same way as any other data type.
- This consists of assigning some constants to the members of the structure.
- Structures that are not explicitly initialized by the programmer are, by default, initialized by the system.

# Example-

```
#include <stdio.h>
struct tablets
{
int count;
float average_weight;
int m_date, m_month, m_year;
int ex_date, ex_month, ex_year;
}batch1={2000,25.3,07,11,2004};
```

# Continue..

```
void main()
{
printf("\n count=%d, av_wt=%f",batch1.count,
batch1.average_weight);

printf("\n mfg-date=%d/%d/%d", batch1.m_date,
batch1.m_month batch1.m_year);

printf("\n exp-date=%d/%d/%d", batch1.ex_date,
batch1.ex_month, batch1.ex_year);
}
```

## **Output:**

```
count=2000, av_wt=25.299999
mfg-date=7/11/2004
exp-date= 0/0/0
```

```
#include<stdio.h>
#include<conio.h>
#include<string.h>_
struct person{
 char name[50];
 int empno;
 float salary;
}person1;
void main()
{
 clrscr();
 //assign values to structure variables
 strcpy(person1.name,"akash");
 person1.empno=1934;
 person1.salary=25000;
 //print values
 printf("Name is:%s\n",person1.name);
 printf("Employee no is:%d\n",person1.empno);
 printf("salary is:%f",person1.salary);
 getch(); _
}
```



## Copying and Comparing Structures-

- You can assign structures as a unit with =  
first = second;

instead of writing:

```
first.x = second.x ;
first.y = second.y ;
```

- Although the saving here is not great
  - It will reduce the likelihood of errors and
  - Is more convenient with large structures

# Example-

```
#include <stdio.h>
```

```
struct employee
```

```
{
```

```
char grade;
```

```
int basic;
```

```
float allowance;
```

```
};
```

```
void main()
```

```
{
```

```
struct employee ramesh={'b', 6500, 812.5}; /* member of employee */
```

```
struct employee vivek; /* member of employee */
```

```
vivek = ramesh; /* copy respective members of
ramesh to vivek */
```

# Continue..

```
printf("\n vivek's grade is %c, basic is Rs %d,
allowance is Rs %f", vivek.grade,vivek.
basic, vivek.allowance);
```

```
}
```

**Output:**

vivek's grade is b, basic is Rs 6500, allowance  
is Rs 812.500000

# Keyword typedef

We use the `typedef` keyword to create an alias name for data types. It is commonly used with structures to simplify the syntax of declaring variables.

For example, let us look at the following code:

```
struct Distance{
 int feet;
 float inch;
};

int main() {
 struct Distance d1, d2;
}
```

We can use `typedef` to write an equivalent code with a simplified syntax:

```
typedef struct Distance {
 int feet;
 float inch;
} distances;

int main() {
 distances d1, d2;
}
```

## Example 2: C typedef

```
#include <stdio.h>
#include <string.h>

// struct with typedef person
typedef struct Person {
 char name[50];
 int citNo;
 float salary;
} person;

int main() {

 // create Person variable
 person p1;

 // assign value to name of p1
 strcpy(p1.name, "George Orwell");

 // assign values to other p1 variables
 p1.citNo = 1984;
 p1.salary = 2500;

 // print struct variables
 printf("Name: %s\n", p1.name);
 printf("Citizenship No.: %d\n", p1.citNo);
 printf("Salary: %.2f", p1.salary);

 return 0;
}
```

## Output

```
Name: George Orwell
Citizenship No.: 1984
Salary: 2500.00
```

# Nesting of Structures-

- Any “type” of thing can be a member of a structure.
- We can define a structure as a member of another structure this is known as nesting of structure.

# Example-

```
#include<stdio.h>
```

```
struct address
```

```
{
```

```
 char city[20];
```

```
 int pin;
```

```
 char phone[14];
```

```
};
```

```
struct employee
```

```
{
```

```
 char name[20];
```

```
 struct address add;
```

```
};
```

# Continue..

```
void main ()
```

```
{
```

```
 struct employee emp;
```

```
 printf("Enter employee information?\n");
```

```
 scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add
.pin, emp.add.phone);
```

```
 printf("Printing the employee information....\n");
```

```
 printf("name: %s\nCity: %s\nPincode: %d\nPhone: %s",e
mp.name,emp.add.city,emp.add.pin,emp.add.phone);
```

```
}
```

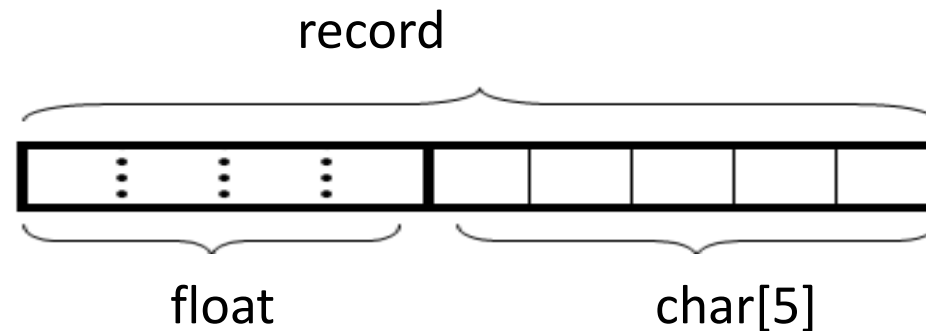


# Structures Containing Arrays

- Arrays within structures are the same as any other member element.
- For example:

```
struct record {
 float x;
 char y [5];
};
```

- Logical organization:



# An Example

```
#include <stdio.h>

struct data {
 float amount;
 char fname[30];
 char lname[30];
} rec;

int main () {
 struct data rec;
 printf ("Enter the donor's first and last names, \n");
 printf ("separated by a space: ");
 scanf ("%s %s", rec.fname, rec.lname);
 printf ("\nEnter the donation amount: ");
 scanf ("%f", &rec.amount);
 printf ("\nDonor %s %s gave $%.2f.\n",
 rec.fname, rec.lname, rec.amount);
}
```

# Arrays of Structures

- The converse of a structure with arrays:

- Example:

```
struct entry {
 char fname [10] ;
 char lname [12] ;
 char phone [8] ;
} ;
struct entry list [1000];
```

- This creates a list of 1000 identical entry(s).

- Assignments:

```
list [1] = list [6];
strcpy (list[1].phone, list[6].phone);
list[6].phone[1] = list[3].phone[4] ;
```

# An Example

```
#include <stdio.h>

struct entry {
 char fname [20];
 char lname [20];
 char phone [10];
};
```

```
int main() {
 struct entry list[4];
 int i;
 for (i=0; i < 4; i++) {
 printf ("\nEnter first name: ");
 scanf ("%s", list[i].fname);
 printf ("Enter last name: ");
 scanf ("%s", list[i].lname);
 printf ("Enter phone in 123-4567 format: ");
 scanf ("%s", list[i].phone);
 }
 printf ("\n\n");
 for (i=0; i < 4; i++) {
 printf ("Name: %s %s", list[i].fname, list[i].lname);
 printf ("\t\tPhone: %s\n", list[i].phone);
 }
}
```

# Problem: float is not scanned in structure

while running this code in Turbo c and Borland compiler, it throws the error i.e **floating point formats not linked**

```
#include<stdio.h>
#include<conio.h>
struct emp
{
 char name[20];
 int no;
 float marks;
}e1[5];
static void force_fpf()
{
 float x,*y;
 y=&x;
 x=*y;
}
void main()
{
 int i;
 for(i=0;i<2;i++)
 {
 scanf("%s",e1[i].name);
 scanf("%d",&e1[i].no);
```

This is because **Turbo and Borland** C/ C++ compilers sometimes leave out floating point support and use non-floating-point version of **printf()** and **scanf()** to save space on smaller systems. When the compiler encounters a reference to the address of float, it sets a flag to have the linker link in the floating point emulator. There are some cases in which the reference to a float is a bit obscure and the compiler does not detect the need for emulator.

## Solution:

Just add the following function in your program. It will force the compiler to include required libraries for handling floating point linkages.

```
static void force_fpf() /* A dummy function */
{
 float x, *y; /* Just declares two variables */
 y = &x; /* Forces linkage of FP formats */
 x = *y; /* Suppress warning message about x */
}
```

*The dummy call to a floating-point function will force the compiler to load the floating-point support and solve the original problem.*

## C Pointers to struct

Here's how you can create pointers to structs.

```
struct name {
 member1;
 member2;
 .
 .
};

int main()
{
 struct name *ptr, Harry;
}
```

Here, `ptr` is a pointer to `struct`.

# Pointers to Structures

To access members of a structure using pointers, we use the `->` operator.

```
#include<stdio.h>
#include<conio.h>
struct person
{
 int age;
 int weight;
};

void main()
{
 struct person *ptr, person1;
 clrscr();
 ptr=&person1;
 printf("enter age");
 scanf("%d",&ptr->age);
 printf("enter weight");
 scanf("%d",&ptr->weight);
 printf("displaying\n");
 printf("age is:%d\n",ptr->age);
 printf("weight is:%d\n",ptr->weight);
 getch();
}
```

```
enter age 30
enter weight 50
displaying
age is:30
weight is:50
```

By the way,

- `personPtr->age` is equivalent to `(*personPtr).age`
- `personPtr->weight` is equivalent to `(*personPtr).weight`

### Example: Dynamic memory allocation of structs

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
struct person{
 int age;
 float weight;
 char name[20];
};
void main()
{
 struct person *ptr;
 int i,n;
 printf("enter number of persons");
 scanf("%d",&n);
 //allocating memory for n numbers of struct person
 ptr=(struct person*)malloc(n*sizeof(struct person));
 for(i=0;i<n;i++)
 {
 printf("enter name and age");
 scanf("%s%d",(ptr+i)->name,&(ptr+i)->age);
 }
 printf("displaying info\n");
 for(i=0;i<n;i++)
 {
 printf("Name is: %s\tAge is:%d\n",(ptr+i)->name,(ptr+i)->age);
 }
 getch();
}
```

```
enter number of persons 2
enter name and age Seema 20
enter name and age Neema 21
displaying info
Name is: Seema Age is:20
Name is: Neema Age is:21
```



# Passing Structures to Functions

- Structures are passed by value to functions
  - The parameter variable is a local variable, which will be assigned by the value of the argument passed.
- This means that the structure is copied if it is passed as a parameter.
  - This can be inefficient if the structure is big.
    - In this case it may be more efficient to pass a pointer to the `struct`.
- A `struct` can also be returned from a function.

# Example-

```
#include<stdio.h>
#include<conio.h>
struct student
{
 char name[50];
 int age;
};
void display (struct student s);
void main()
{
 struct student s1;
 clrscr();
 printf("enter first name");
 scanf("%s",s1.name);
 printf("enter age");
 scanf("%d",&s1.age);
 display(s1);
 getch();
}
void display(struct student s)
{
 printf("displaying");
 printf("name is %s\n",s.name);
 printf("age is %d",s.age);
}
```

# Output-

Enter Name:Joe

Enter age:20

Displaying information:

Name:Joe

Age:20

# C Unions

A union is a user-defined type similar to [structs in C](#) except for one key difference.

Structures allocate enough space to store all their members, whereas **unions can only hold one member value at a time.**

## How to define a union?

We use the `union` keyword to define unions. Here's an example:

```
union car
{
 char name[50];
 int price;
};
```

The above code defines a derived type `union car`.

## Create union variables

When a union is defined, it creates a user-defined type. However, no memory is allocated. To allocate memory for a given union type and work with it, we need to create variables.

Here's how we create union variables.

```
union car
{
 char name[50];
 int price;
};

int main()
{
 union car car1, car2, *car3;
 return 0;
}
```

Another way of creating union variables is:

```
union car
{
 char name[50];
 int price;
} car1, car2, *car3;
```

In both cases, union variables `car1`, `car2`, and a union pointer `car3` of `union car` type are created.

### Access members of a union

We use the `.` operator to access members of a union. And to access pointer variables, we use the `->` operator.

In the above example,

- To access `price` for `car1`, `car1.price` is used.
- To access `price` using `car3`, either `(*car3).price` or `car3->price` can be used.

# Example-

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
union employee{
 char name[20];
}emp;
void main()
{
 clrscr();
 strcpy(emp.name,"Joe");
 printf("employee name id: %s",emp.name);
 getch();
}
```

# Example-

```
#include <stdio.h>
#include <string.h>
union employee
{
 int id;
 char name[50];
}e1; //declaring e1 variable for union
int main()
{
 //store first employee information
 e1.id=101;
```



# Continue..

```
strcpy(e1.name, "Joe");//copying string into char array
//printing first employee information
```

```
printf("employee 1 id : %d\n", e1.id);
printf("employee 1 name : %s\n", e1.name);
return 0;
}
```

output-

employee 1 id :101

employee 1 name :Joe

# Structure vs Union-

## Structure

```
struct Employee{
char x; // size 1 byte
int y; //size 2 byte
float z; //size 4 byte
}e1; //size of e1 = 7 byte
```

**size of e1 = 1 + 2 + 4 = 7**

## Union

```
union Employee{
char x; // size 1 byte
int y; //size 2 byte
float z; //size 4 byte
}e1; //size of e1 = 4 byte
```

**size of e1 = 4 (maximum size of 1 element)**

## Difference between unions and structures

Let's take an example to demonstrate the difference between unions and structures:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
union uemp{
 char name[32];
 float salary;
 int wno;
}u1;
struct semp{
 char name[32];
 float salary;
 int wno;
}s1;

void main()
{
 clrscr();
 printf("size of union=%d bytes\n", sizeof(u1));
 printf("size of structure=%d bytes",sizeof(s1));
 getch();
}
```

```
size of union=32 bytes
size of structure=38 bytes_
```

# Enumeration-

- Enumeration is a user defined datatype in C language. It is used to assign names to the integral constants which makes a program easy to read and maintain.
- The keyword “enum” is used to declare an enumeration.
- Here is the syntax of enum in C language,
- `enum enumname`

# Enumeration-

In C programming, an enumeration type (also called enum) is a data type that consists of integral constants. To define enums, the `enum` keyword is used.

```
enum flag {const1, const2, ..., constN};
```

By default, `const1` is 0, `const2` is 1 and so on. You can change default values of enum elements during declaration (if necessary).

```
// Changing default values of enum constants
enum suit {
 club = 0,
 diamonds = 10,
 hearts = 20,
 spades = 3,
};
```

# Enumeration-

## Enumerated Type Declaration

When you define an enum type, the blueprint for the variable is created. Here's how you can create variables of enum types.

```
enum boolean {false, true};
enum boolean check; // declaring an enum variable
```

Here, a variable `check` of the type `enum boolean` is created.

You can also declare enum variables like this.

```
enum boolean {false, true} check;
```

Here, the value of `false` is equal to 0 and the value of `true` is equal to 1.

## Example-

```
#include<stdio.h>
#include<conio.h>
enum week{sunday,monday,tuesday,wednesday,thursday,friday,saturday};
void main()
{
 //creating today variable of enum week type
 enum week today;
 clrscr();_
 today=wednesday;
 printf("Day %d",today+1);
 getch();
}
```

Day 4\_

# Example-

```
#include<stdio.h>
```

```
enum week{Mon=10, Tue, Wed, Thur, Fri=10, Sat=16, Sun};
```

```
enum day{Mond, Tues, Wedn, Thurs, Frid=18, Satu=11, Sund};
```

```
int main()
```

```
{
```

```
 printf("The value of enum week:
%d\t%d\t%d\t%d\t%d\t%d\t%d\n\n",Mon , Tue, Wed, Thur, Fri, Sat,
Sun);
```

```
 printf("The default value of enum day:
%d\t%d\t%d\t%d\t%d\t%d\t%d",Mond , Tues, Wedn, Thurs, Frid, Satu,
Sund);
```

```
 return 0;}
```



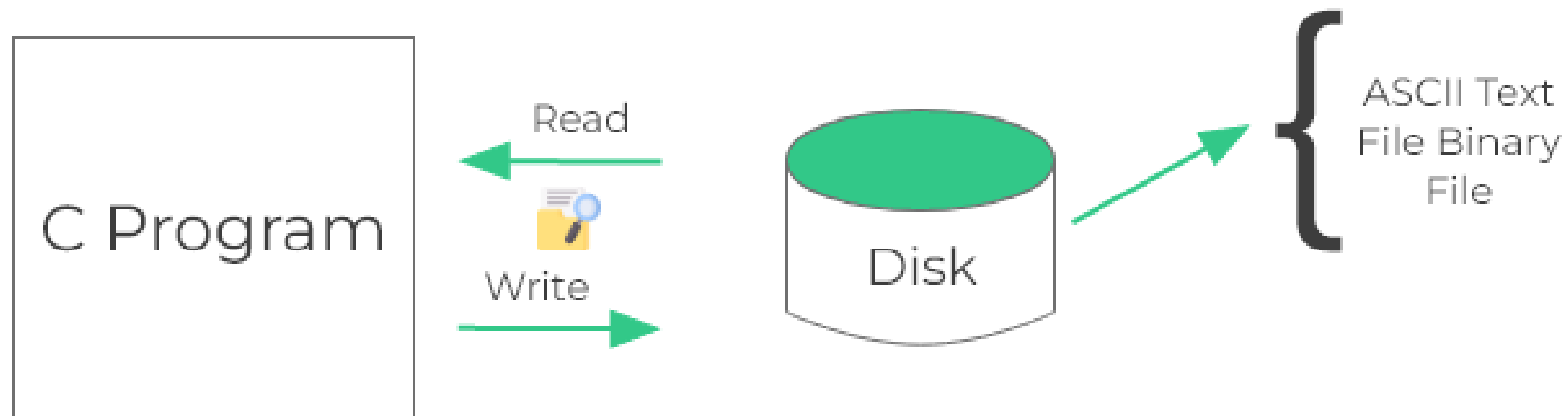
# Output-

The value of enum week: 10 11 12 13 10 16 17

The default value of enum day: 0 1 2 3 18 11 12

# Files in C

## File Handling in C



# Files in C

- A file is a repository of data that is stored in a permanent storage media, mainly in secondary memory.
- So far, data was entered into the programs through the computer's keyboard. This is somewhat laborious if there is a lot of data to process.
- The solution is to combine all the input data into a file and let the C program read the information from the file when it is required.
- Frequently files are used for storing information that can be processed by the programs.

# Files in C

**ANSI C abstracts all I/O as stream of bytes moving into and out of a C program.** This stream of bytes is called STREAM. A C program is concerned only with creating the correct stream of bytes and interpreting the stream of bytes coming into the program as input. **Mostly all I/O streams are fully buffered. This means that reading and writing data is actually copying data out of and into an area of memory called buffer.** Because reading from and writing to memory is fast, this enhances the performance. Buffer for output stream is flushed, meaning physically written to device or file when it's full. Writing full buffer is more efficient than writing data in small bits or pieces as program produces them. Similarly, input buffers are re-filled when they become empty by reading next large chunk of input from the device or file into the buffer.

# USING FILES IN C

- To use a file four essential actions should to be carried out. These are
- Declare a file pointer variable.
- Open a file using the `fopen()` function.
- Process the file using suitable functions.
- Close the file using the `fclose()` function.

# Declaration of File Pointer

- `FILE *file_pointer_name,...;`

For example,

- `FILE *ifp;`
- `FILE *ofp;`
- declares ifp and ofp to be FILE pointers. Or, the two FILE pointers can be declared in just one declaration statement as shown below.
- `FILE *ifp, *ofp;`
- The \* must be repeated for each variable.
- FILE is a structure declared in `stdio.h`.

# Opening a File

- To open a file and associate it with a stream, the `fopen()` function is used. Its prototype is as follows.
- `FILE *fopen(const char *fname, const char *mode);`
- The first parameter is the name of the file whereas the second parameter is opening mode of the file.
- The following statements are used to create a text file with the name `data.dat` under current directory. It is opened in `w` mode as data are to be written into the file `data.dat`.

```
FILE *fp;
```

```
fp = fopen("data.dat","r");
```

# File opening modes

| Mode | Meaning                                     |
|------|---------------------------------------------|
| r    | Open a text file for reading                |
| w    | Create a text file for writing              |
| a    | Append to a text file                       |
| rb   | Open a binary file for reading              |
| wb   | Open a binary file for writing              |
| ab   | Append to a binary file                     |
| r+   | Open a text file for read/write             |
| w+   | Create a text file for read/write           |
| a+   | Append or create a text file for read/write |
| r+b  | Open a binary file for read/write           |
| w+b  | Create a binary file for read/write         |
| a+b  | Append a binary file for read/write         |



# File opening modes

| Mode | File Must Exist?                                                                       | Creates File if Not Exist?                                                              | Truncates File?                                                                           | Read                                                                                      | Write                                                                                     | Description                   |
|------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------|
| "r"  |  Yes  |  No    |  No    |  Yes   |  No    | Read-only access              |
| "r+" |  Yes  |  No    |  No    |  Yes   |  Yes   | Read/write without truncation |
| "w"  |  No   |  Yes   |  Yes   |  No    |  Yes   | Write-only with truncation    |
| "w+" |  No |  Yes |  Yes |  Yes |  Yes | Read/write with truncation    |

# *Checking the Result of Fopen()*

- The fopen() function returns a FILE \*, which is a pointer to structure FILE, that can then be used to access the file. When the file cannot be opened due to reasons described below, fopen() will return NULL.

The reasons include the following.

- Use of an invalid filename
- Attempt to open a file on a disk that is not ready
- Attempt to open a file in a non-existent directory
- Attempt to open a non-existent file in mode r

# Continue..

One may check to see whether fopen() succeeds or fails by writing the following set of statements.

```
fp = fopen("data.dat","r");//attempts to open the file named
"data.dat" in read mode
```

```
if(fp == NULL)
{
printf("Can not open data.dat\n");
exit(1);
}
```

# Closing a file

- After completing the processing on the file, the file must be closed using the `fclose()` function. Its prototype is
- `int fclose(FILE *fp);`
- The argument `fp` is the FILE pointer associated with the stream;  
`fclose()` returns 0 on success or -1 on error.
- You can clear a buffer without closing it  
`int fflush (FILE * fp) ;`

# Detecting End of File

- Text mode files:

```
while ((c = fgetc (fp)) != EOF)
```

- Reads characters until it encounters the EOF
- The problem is that the byte of data read may actually be indistinguishable from EOF.

- Binary mode files:

```
int feof (FILE * fp) ;
```

- Note: the feof function realizes the end of file only after a reading failed (fread, fscanf, fgetc ... )

```
fseek(fp,0,SEEK_END);
```

```
printf("%d\n", feof(fp)); /* zero value */
```

```
fgetc(fp); /* fgetc returns -1 */
```

```
printf("%d\n",feof(fp)); /* nonzero value */
```

# WORKING WITH TEXT FILES

- C provides four functions that can be used to read text files from the disk. These are
  - fscanf()
  - fgets()
  - fgetc()
  - fread()
- C provides four functions that can be used to write text files into the disk. These are
  - fprintf()
  - fputs()
  - fputc()
  - fwrite()

# Four Ways to Read and Write Files

- Formatted file I/O

`fscanf()`, `fprintf()`

- Get and put a character

`fgetc()`, `fputc()`

- Get and put a line

`fgets()`, `fputs()`

- Block read and write

`fread()`, `fwrite()`

# Example formatted File I/O

- Formatted File input is done through fscanf:
  - `int fscanf (FILE * fp, const char * fmt, ...) ;`
- Formatted File output is done through fprintf:
  - `int fprintf(FILE *fp, const char *fmt, ...);`
  - Output of following code1: fprintf and 10 will be written in file1.txt output of code2: read integer from file1 and write to file2

```
#include<stdio.h>
void main()
{
 FILE *fp1;
 fp1=fopen("file1.txt","w");
 fprintf(fp1,"%s%d","fprintf",10);
 fclose(fp1);
}
```

```
#include<stdio.h>
void main()
{
 FILE *fp1,*fp2;
 int n;
 fp1=fopen("file1.txt","r");
 fp2=fopen("file2.txt","w");
 fscanf(fp1,"%d",&n);
 fprintf(fp2,"%d",n);
 fclose(fp1);
 fclose(fp2);
}
```



# Get and Put a Character

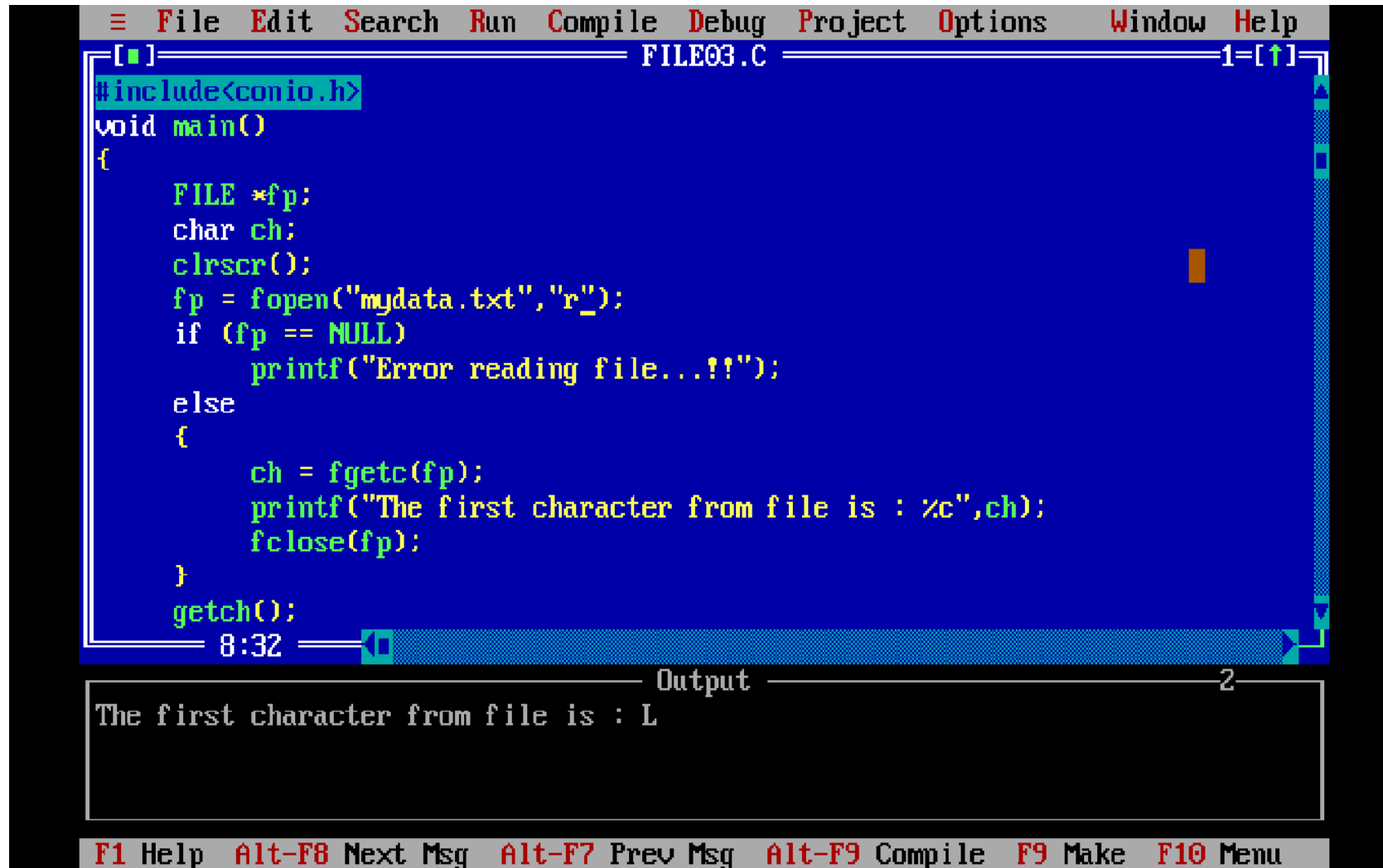
```
#include <stdio.h>
```

```
int fgetc(FILE * fp);
```

```
int fputc(int c, FILE * fp);
```

- These two functions read or write a single byte from or to a file.
- fgetc returns the character that was read, converted to an integer.
- fputc returns the same value of parameter c if it succeeds, otherwise, return EOF.

# Example- fgetc()



```
File Edit Search Run Compile Debug Project Options Window Help
FILE03.C
#include<conio.h>
void main()
{
 FILE *fp;
 char ch;
 clrscr();
 fp = fopen("mydata.txt","r");
 if (fp == NULL)
 printf("Error reading file...!!");
 else
 {
 ch = fgetc(fp);
 printf("The first character from file is : %c",ch);
 fclose(fp);
 }
 getch();
}
```

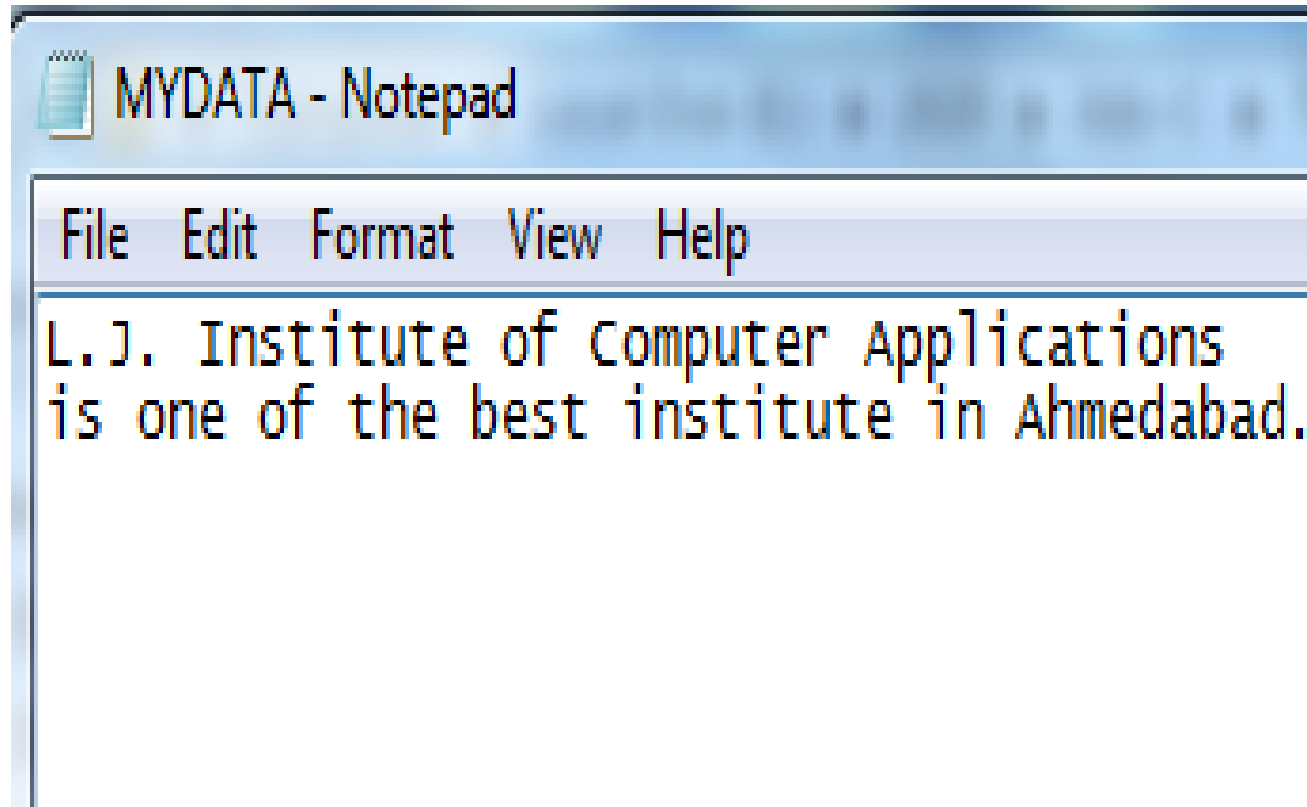
8:32

Output

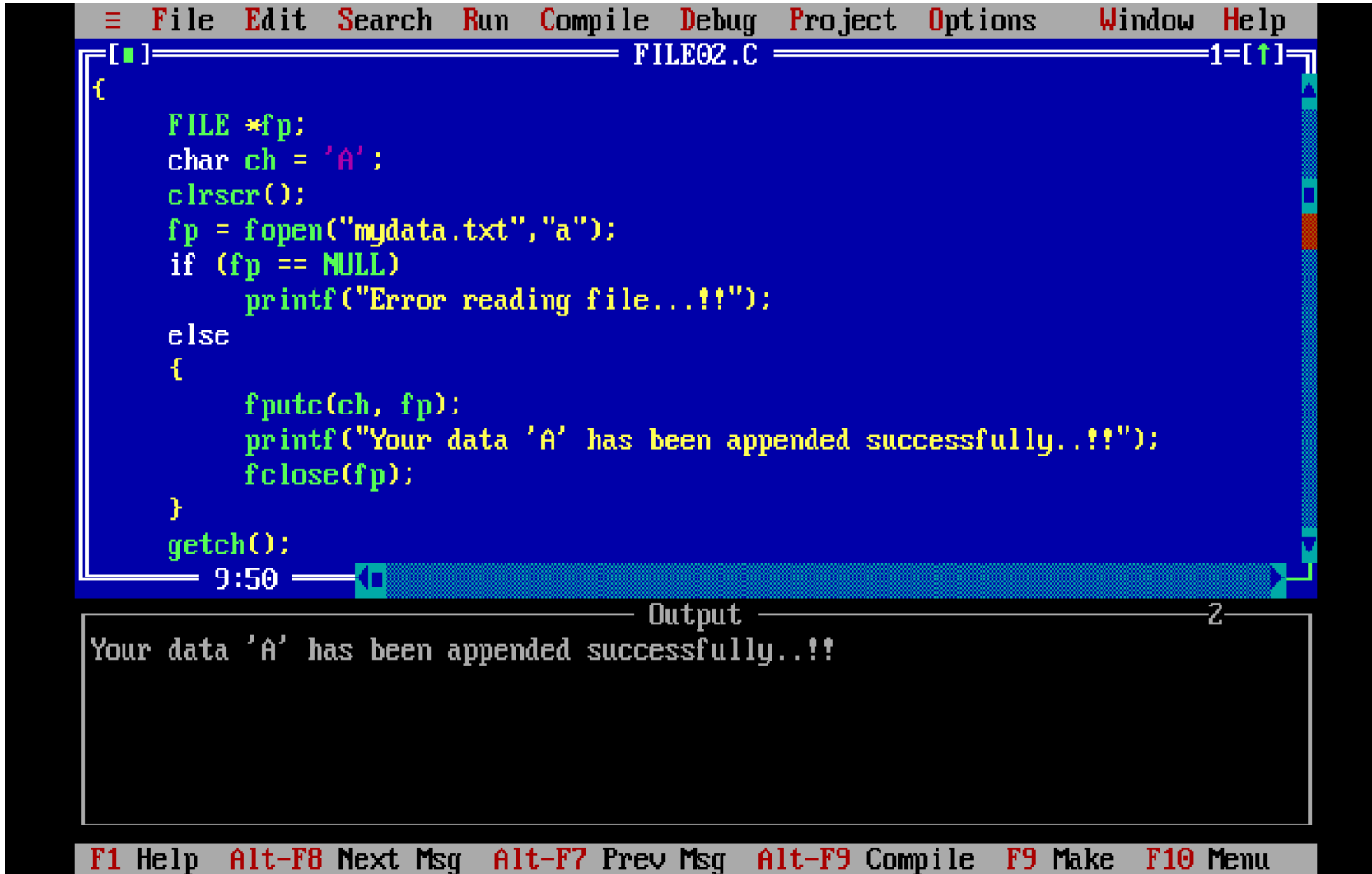
The first character from file is : L

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

# Continue..



# Example- fputc()



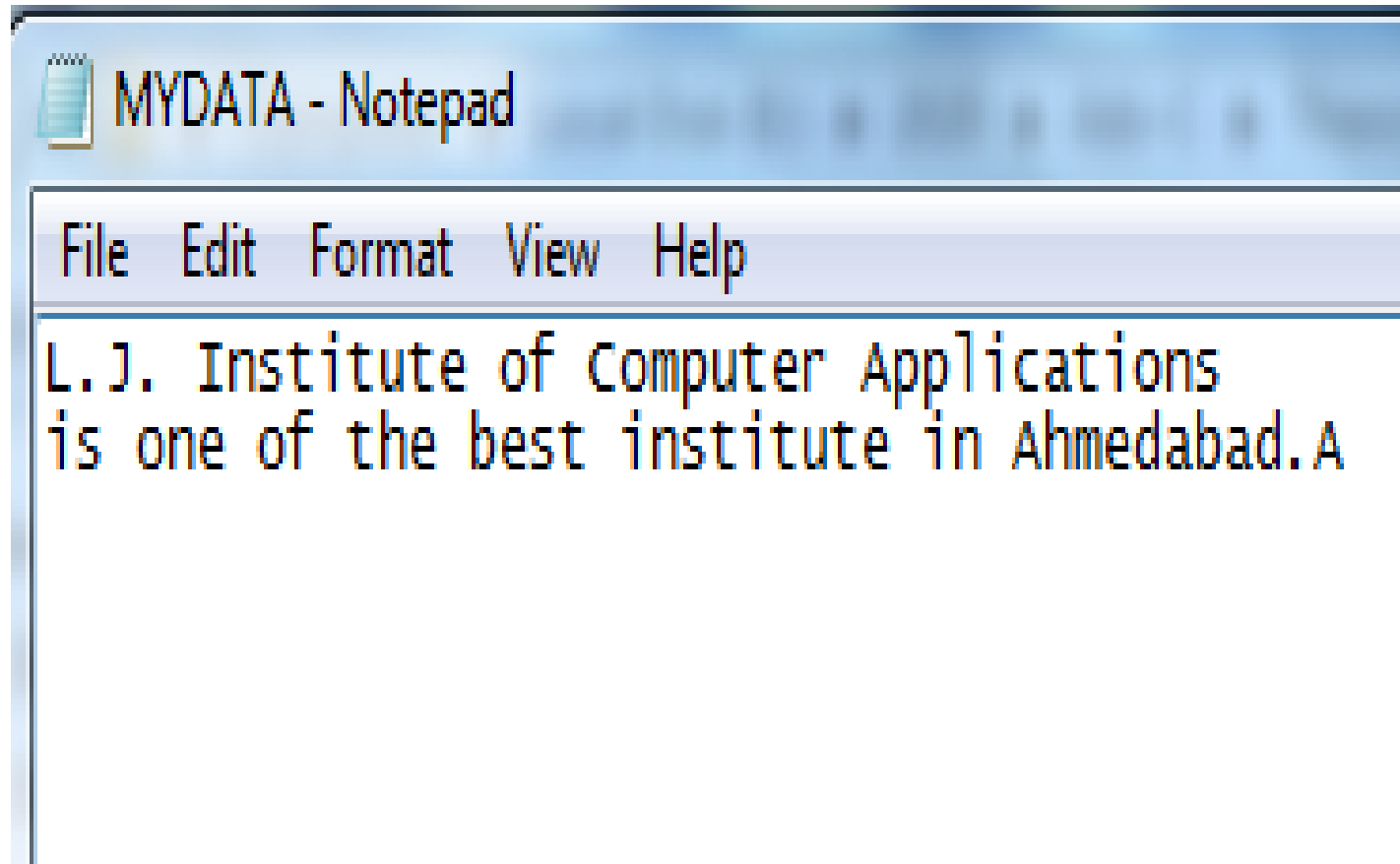
```
File Edit Search Run Compile Debug Project Options Window Help
FILE02.C
{
 FILE *fp;
 char ch = 'A';
 clrscr();
 fp = fopen("mydata.txt","a");
 if (fp == NULL)
 printf("Error reading file...!!");
 else
 {
 fputc(ch, fp);
 printf("Your data 'A' has been appended successfully...!!");
 fclose(fp);
 }
 getch();
}
9:50
```

Output

Your data 'A' has been appended successfully...!!

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

Continue..



# Get and Put a Line

```
#include <stdio.h>
```

```
char *fgets(char *str, int n, FILE *stream)
```

```
int fputs(const char *str, FILE *stream)
```

- These two functions read or write a string from or to a file.

# fgets()-

Syntax-

```
char *fgets(char *str, int n, FILE *stream)
```

- str – This is the pointer to an array of chars where the string read is stored.
- n – This is the maximum number of characters to be read (including the final null-character).  
Usually, the length of the array passed as str is used.
- stream – This is the pointer to a FILE object that identifies the stream where characters are read from.

# Continue..

## Return Value-

- On success, the function returns the same str parameter. If the End-of-File is encountered and no characters have been read, the contents of str remain unchanged and a null pointer is returned.
- If an error occurs, a null pointer is returned.



# Example-

```
#include <stdio.h>

int main ()
{
 FILE *fp;
 char str[60];
 fp = fopen("file.txt" , "r"); /* opening file for reading */
 if(fp == NULL)
 {
 printf("Error opening file");
 exit(1);
 }
 if(fgets (str, 60, fp)!=NULL)
 {
 puts(str); /* writing content to stdout */
 }
 fclose(fp);
 return(0);
}
```

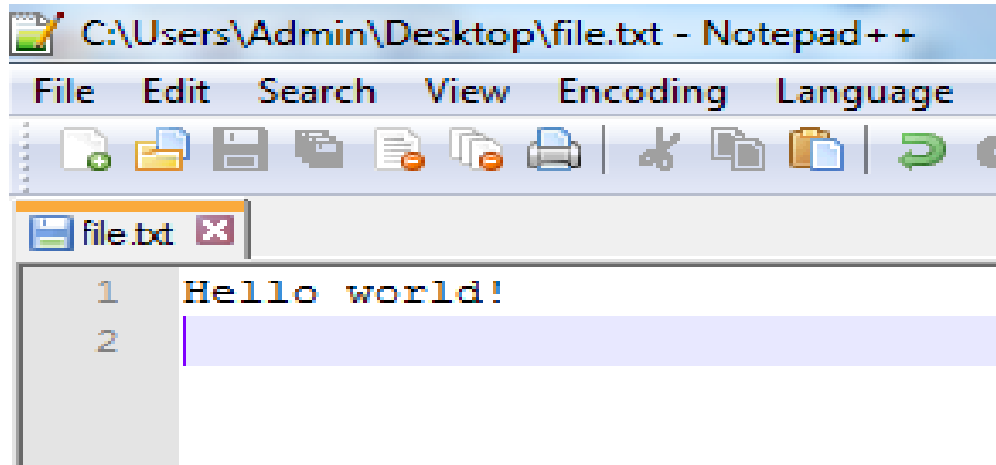
- **Output-**  
**Hello world!**

```
#include<stdio.h>
#include<conio.h>
```

```
int main()
```

```
FILE *fp;
char t[100];
clrscr();
fp=fopen("file1.txt","r");
fgets(t,100,fp);
printf("%s",t);
fclose(fp);
getch();
```

Continue..file to be read-



A screenshot of the Notepad++ application window. The title bar reads 'C:\Users\Admin\Desktop\file.txt - Notepad++'. The menu bar includes 'File', 'Edit', 'Search', 'View', 'Encoding', and 'Language'. The toolbar contains icons for opening, saving, printing, and other file operations. The editor window shows a single file named 'file.txt' with two lines of text: '1 Hello world!' and '2'. The second line is currently selected.

```
1 Hello world!
2
```

# fputs()

Syntax-

```
int fputs(const char *str, FILE *stream)
```

- **str** – This is an array containing the null-terminated sequence of characters to be written.
- **stream** – This is the pointer to a FILE object that identifies the stream where the string is to be written.

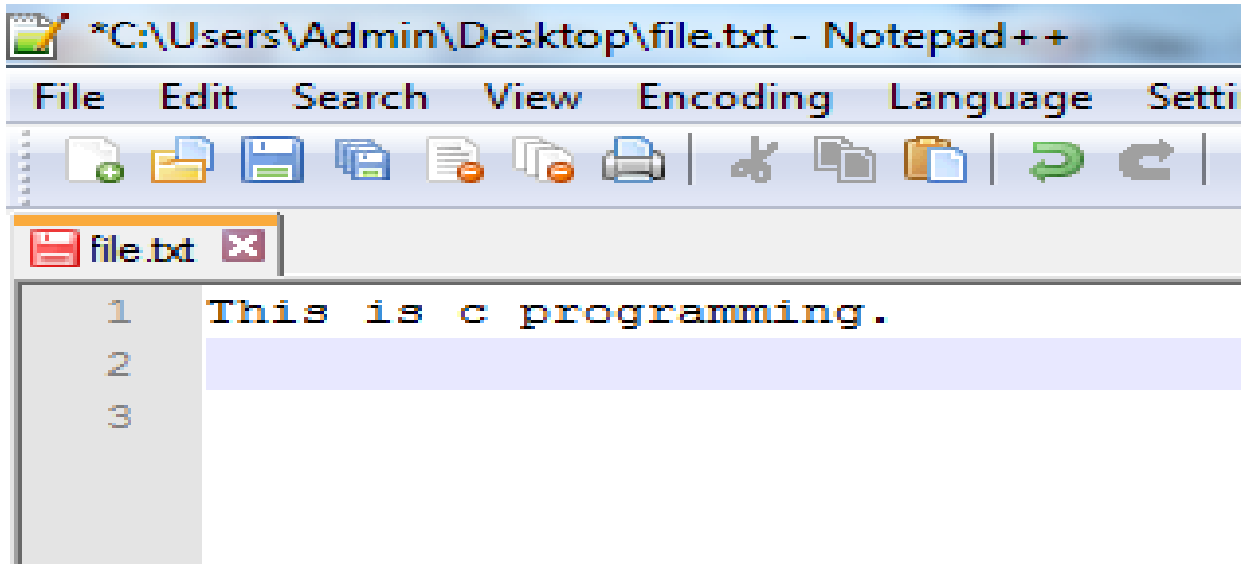
Return Value

- This function returns a non-negative value, or else on error it returns EOF.

# Example-

```
#include <stdio.h>
int main ()
{
FILE *fp;
fp = fopen("file.txt", "w+");
if(fp == NULL)
{
printf("Error opening file");
exit(1);
}
fputs("This is c programming.", fp);
fclose(fp);
return(0);
}
```

# Output-



A screenshot of the Notepad++ application window. The title bar reads '\*C:\Users\Admin\Desktop\file.txt - Notepad++'. The menu bar includes File, Edit, Search, View, Encoding, Language, and Settings. The toolbar contains icons for opening, saving, printing, and other file operations. A single tab labeled 'file.txt' is open. The text area shows the output of a C program: '1 This is c programming.' on the first line, and lines 2 and 3 are empty. The text is in a monospaced font.

```
1 This is c programming.
2
3
```

# putw(),getw()

```
#include<stdio.h>
#include<conio.h>
void main()
{
 FILE *fp1;
 int n=5,m;
 clrscr();
 fp1=fopen("mydata.txt","w");
 if(fp1==NULL)
 {
 printf("error");
 }
 else
 {
 putw(n,fp1);
 fclose(fp1);
 fp1=fopen("mydata.txt","r");
 m=getw(fp1);
 printf("%d",m);
 fclose(fp1);
 }
 getch();
}
```

**putw( )** function is used for writing a number into file.

**getw( )** function is used for reading a number from a file.

Output: 5

# fwrite and fread

The function **fwrite()** is a standard library function in C language, present in the **stdio.h** header file, which allows us to write data into a file, generally, the file here is a binary file, but we can use it with text files as well.

```
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream);
```

## Parameters of fwrite() Function in C

The function **fwrite** in C receives **4** parameters

- **ptr** : pointer to the block of memory or the array from where the elements are to be copied
- **size** : size of each element in **bytes** which are to be written
- **count** : specifies the number of elements that are to be written
- **stream** : pointer to a **FILE** object, where the data read will be written

- For example a 100 element array of integers
  - `fwrite( buf, sizeof(int), 100, fp);`
- The `fwrite` (`fread`) returns the number of items actually written (read).



# Example- fwrite()

```
#include <stdio.h>
int main ()
{
FILE *fp;
char str[]="welcome";
fp = fopen("file.txt" , "w");
if(fp == NULL)
{
printf("Error opening file");
exit(1);
}
fwrite(str,1,strlen(str),fp);
fclose(fp);
}
```

## Output-

In file file.txt

**welcome**

```
#include<stdio.h>
#include<conio.h>
void main()
{
 FILE *fp1;
 char str[]="hello world";
 clrscr();
 fp1=fopen("mydata.txt","w");
 if(fp1==NULL)
 {
 printf("error");
 }
 else
 {
 fwrite(str,1,sizeof(str)-1,fp1);
 fclose(fp1);
 }
 getch();
}
```

# Example- fread()

```
#include<stdio.h>
#include<conio.h>
void main()
{
 FILE *fp1;
 char str[30];
 int count;
 clrscr();
 fp1=fopen("mydata.txt","r");
 if(fp1==NULL)
 {
 printf("error");
 }
 else
 {
 fread(str,1,10,fp1);
 printf("%s",str);
 fclose(fp1);
 }
 getch();
}
```

# Binary file and text file

## Text File

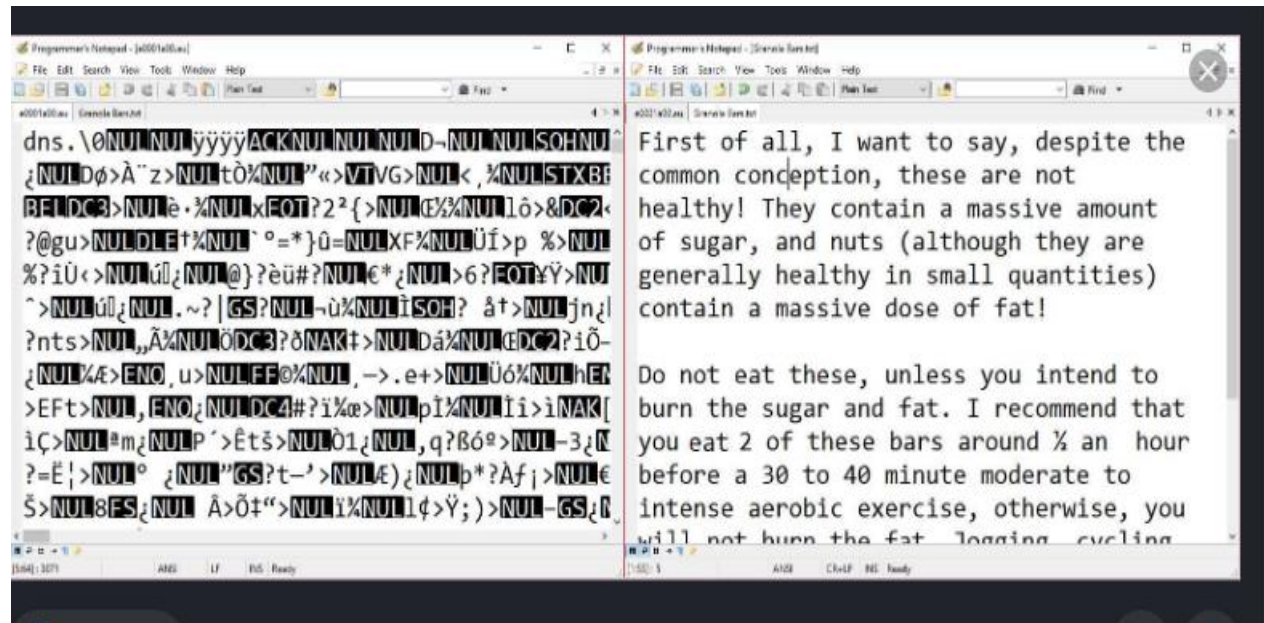
- ▣ It contains alphabets and numbers which are easily understood by human beings.
- ▣ An error in a text file can be eliminated when seen.
- ▣ In text file, the text and characters will store one char per byte.
- ▣ For example, the integer value 4567 will occupy 2 bytes in memory, but, it will occupy 5 bytes in text file.
- ▣ The data format is usually line-oriented. Here, each line is a separate command.

# Binary file and text file

## Binary file

- ▣ It contains 1's and 0's, which are easily understood by computers.
- ▣ The error in a binary file corrupts the file and is not easy to detect.
- ▣ In binary file, the integer value 1245 will occupy 2 bytes in memory and in file.
- ▣ A binary file always needs a matching software to read or write it.
- ▣ For example, an MP3 file can be produced by a sound recorder or audio editor, and it can be played in a music player.
- ▣ MP3 file will not play in an image viewer or a database software.

Continue..



# Sequential and Random Access

- ▣ **Sequential files** – Here, data is stored and retained in a sequential manner.
- ▣ **Random access Files** – Here, data is stored and retrieved in a random way.
- In the FILE structure, there is a long type to indicate the position of your next reading or writing.
- When you read/write, the position move forward.
- You can “rewind” and start reading from the beginning of the file again:  

```
void rewind (FILE * fp) ;
```
- To determine where the position indicator is use:  

```
long ftell (FILE * fp) ;
```

  - Returns a long giving the current position in bytes.
  - The first byte of the file is byte 0.
  - If an error occurs, ftell () returns -1.

# rewind()-

- The rewind() function sets the file pointer at the beginning of the stream. It is useful if you have to use stream many times.
- **Syntax:**
- **void** rewind(**FILE** \*stream)

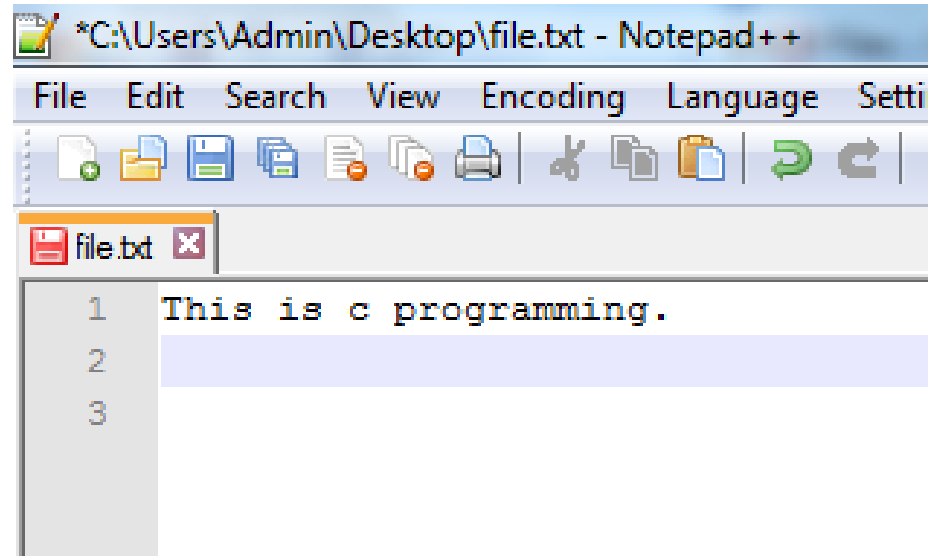
Example- rewind puts file pointer at first position

```
#include<stdio.h>
#include<conio.h>
void main()
{
 FILE *fp;
 char ch;
 clrscr();
 fp=fopen("n1.txt","r");
 printf("file content\n");
 do
 {
 ch=fgetc(fp);
 printf("%c",ch);
 }while(ch!=EOF);
 rewind(fp);
 do
 {
 ch=fgetc(fp);
 printf("%c",ch);
 }while(ch!=EOF);
 fclose(fp);
 getch();
}
```

```
l=ftell(fp);
printf("length is %d",l);
```



# file.txt



# Output-

- **This is c programming.This is c programming.**
- rewind() function moves the file pointer at beginning of the file that is why "**This is c programming.**" is printed 2 times.

# ftell()-

- The ftell() function returns the current file position of the specified stream. We can use ftell() function to get the total size of a file after moving file pointer at the end of file.
- We can use SEEK\_END constant to move the file pointer at the end of file.

## **Syntax:**

- **long int** ftell(**FILE** \*stream)

# Example-

```
#include <stdio.h>
#include <conio.h>
void main (){
 FILE *fp;
 int length;
 clrscr();
 fp = fopen("file.txt", "r");
 fseek(fp, 0, SEEK_END);

 length = ftell(fp);

 fclose(fp);
 printf("Size of file: %d bytes", length);
 getch();
}
```

# Output-

- Size of file: 22 bytes

# Random Access

- One additional operation gives slightly better control:  
`int fseek (FILE * fp, long offset, int origin) ;`
  - offset is the number of bytes to move the position indicator
  - origin says where to move from
- Three options/constants are defined for origin
  - `SEEK_SET`
    - move the indicator offset bytes from the beginning
  - `SEEK_CUR`
    - move the indicator offset bytes from its current position
  - `SEEK_END`
    - move the indicator offset bytes from the end

# Example-

```
#include<stdio.h>
#include<conio.h>
void main()
{
 FILE *fp;
 fp=fopen("n11.txt","w+");
 fputs("This is example",fp);
 fseek(fp,0,SEEK_SET);
 fputs("programming",fp);
 fclose(fp);
 getch();
}
```

Output-in myfile.txt example is overwrite by programming

This is programming

# File Management Functions

- Erasing a file:

`int remove (const char * filename);`

- This is a character string naming the file.
- Returns 0 if deleted; otherwise -1.

- Renaming a file:

`int rename (const char * oldname, const char * newname);`

- Returns 0 if successful or -1 if an error occurs.
- error: file oldname does not exist
- error: file newname already exists
- error: try to rename to another disk



# Example-remove()

```
#include <stdio.h>
#include <string.h>

void main () {
 int rem;
 FILE *fp;
 char filename[] = "file.txt";
 fp = fopen(filename, "w");

 fprintf(fp, "%s", "This is an example");
 fclose(fp);

 rem = remove(filename);

 if(rem == 0) {
 printf("File deleted successfully");
 } else {
 printf("Error: unable to delete the file");
 }
}
```

# Example- rename()

```
#include <stdio.h>
```

```
void main ()
```

```
{
```

```
int ren;
```

```
char oldname[] = "file.txt";
```

```
char newname[] = "newfile.txt";
```

```
ren = rename(oldname, newname);
```

```
if(ren == 0)
```

```
{
```

```
printf("File renamed successfully");
```

```
}
```

```
else
```

```
{
```

```
printf("Error: unable to rename the file");
```

```
}
```

```
}
```

# fgetpos and fsetpos

```
void main()
{
 FILE *fp;
 fpos_t pos;
 clrscr();
 fp=fopen("n1.txt","w");
 fgetpos(fp,&pos);
 fputs("hello world",fp);
 fsetpos(fp,&pos);
 fputs("this is going to override previous content",fp);
 fclose(fp);
 getch();
}
```

Let us compile and run the above program to create a file **file.txt** which will have the following content. First of all we get the initial position of the file using **fgetpos()** function, and then we write Hello, World! in the file but later we used **fsetpos()** function to reset the write pointer at the beginning of the file and then over-write the file with the following content -

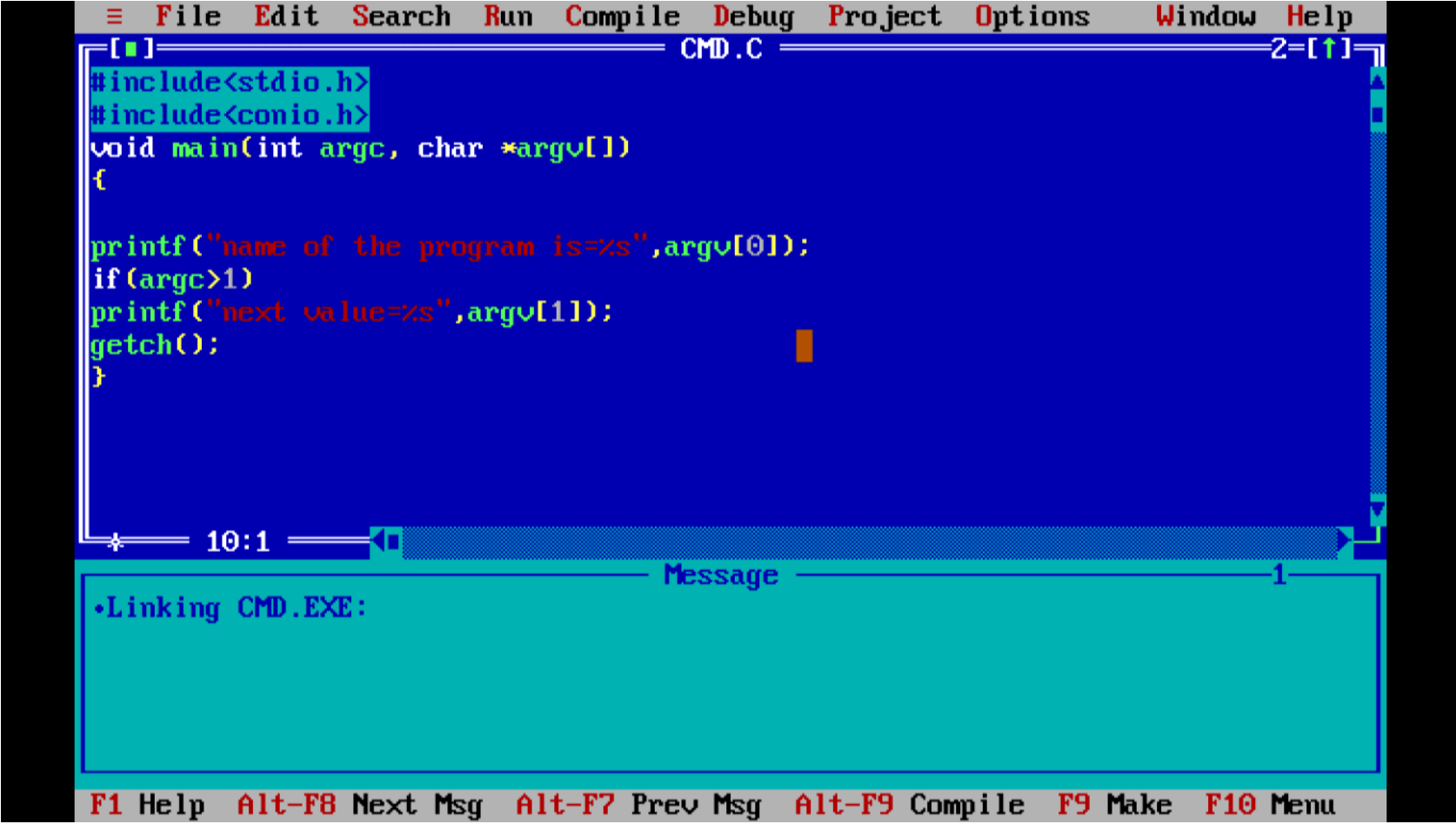
This is going to override previous content

# Command Line Arguments in C

- The arguments passed from command line are called command line arguments. These arguments are handled by `main()` function.
- To support command line argument, you need to change the structure of `main()` function as given below.
- **`int main(int argc, char *argv[] )`**
- Here, **`argc`** counts the number of arguments. It counts the file name as the first argument.
- The **`argv[]`** contains the total number of arguments. The first argument is the file name always.

# How to execute program?

## Step-1



The screenshot shows the Turbo C++ IDE interface. The main window displays a C program named `CMD.C`. The code includes `<stdio.h>` and `<conio.h>`, and defines a `main` function that prints the program name and any command-line arguments. The status bar at the bottom indicates the file size is 10:1. A message box titled "Message" is open at the bottom, displaying the text "•Linking CMD.EXE:". The bottom status bar also shows function key shortcuts: F1 Help, Alt-F8 Next Msg, Alt-F7 Prev Msg, Alt-F9 Compile, F9 Make, and F10 Menu.

```
File Edit Search Run Compile Debug Project Options Window Help
CMD.C
#include<stdio.h>
#include<conio.h>
void main(int argc, char *argv[])
{
 printf("name of the program is=%s",argv[0]);
 if(argc>1)
 printf("next value=%s",argv[1]);
 getch();
}
```

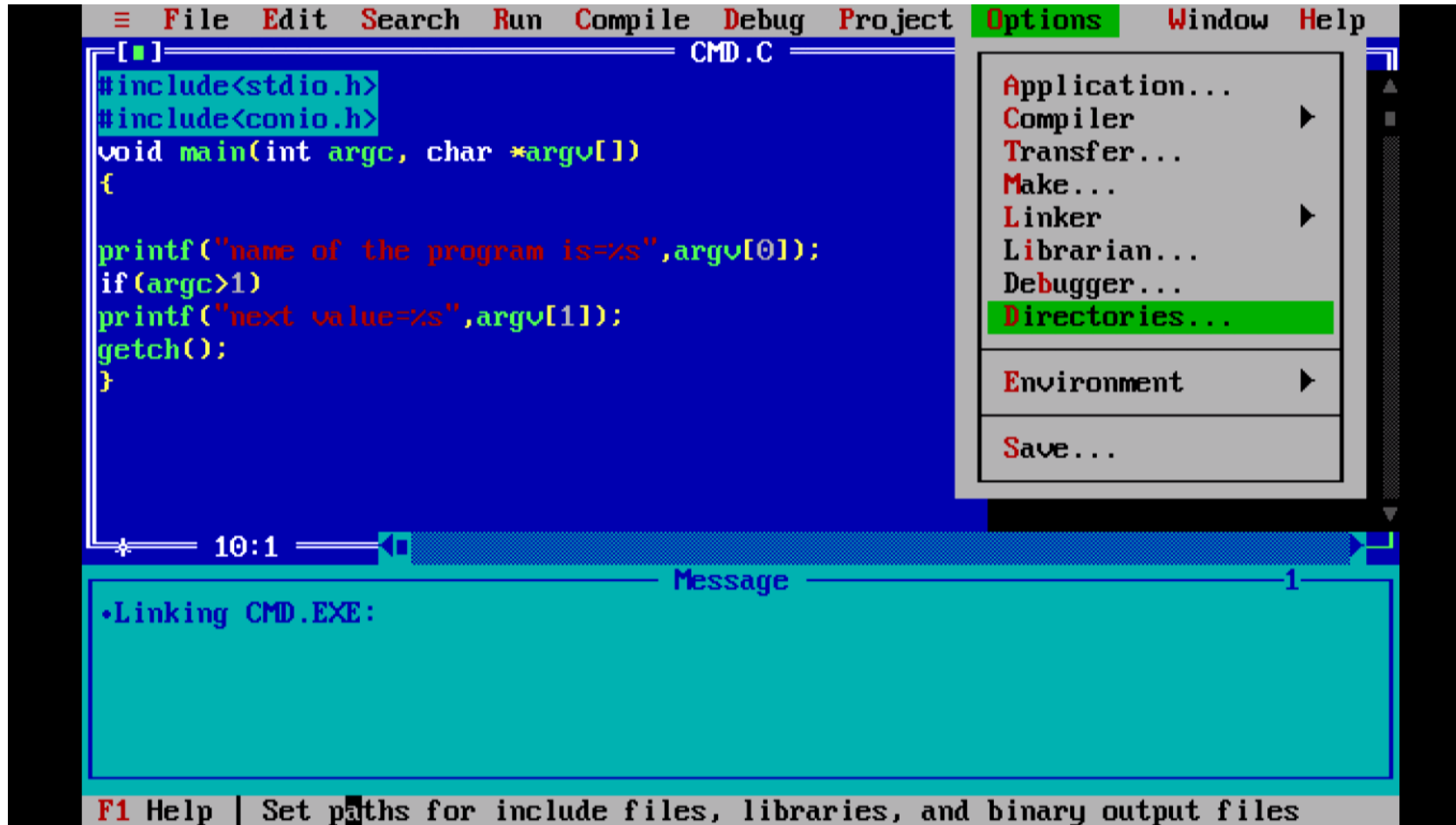
10:1

Message

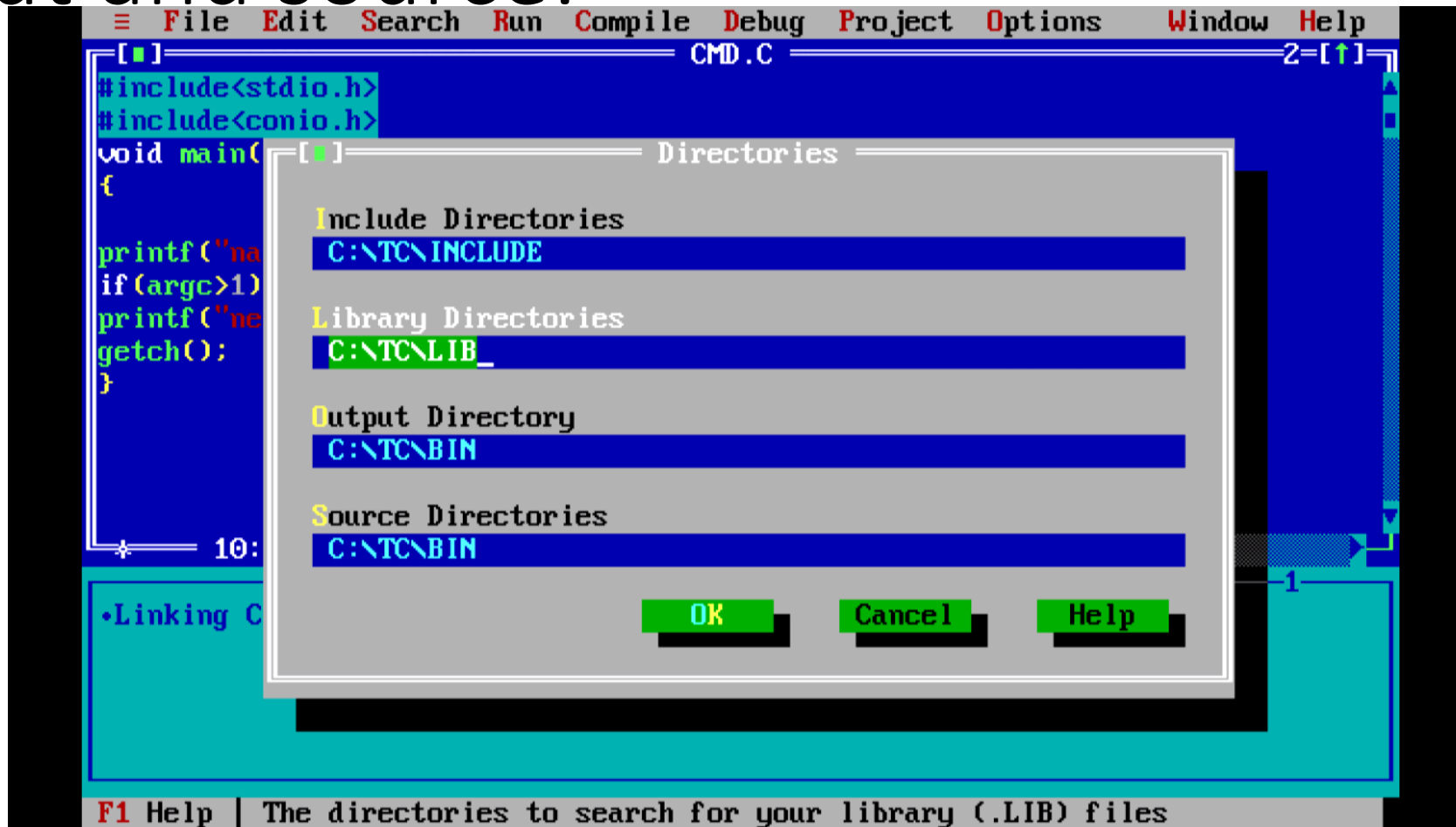
•Linking CMD.EXE:

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

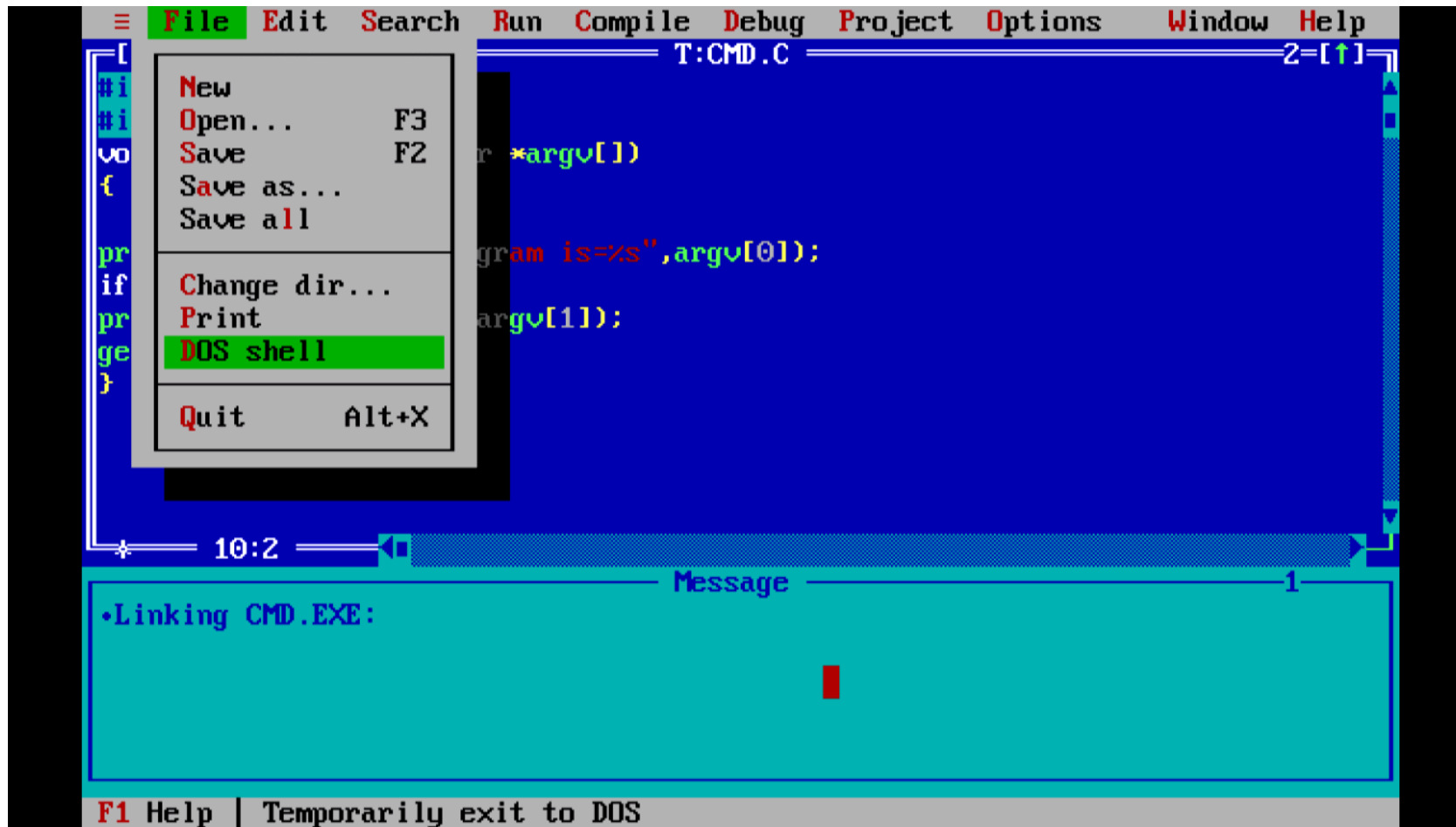
## Step-2 Set directories under option tab-



Step-2 Set directories under option tab-set output and source:



Step-3 go in dos shell under file tab-





Step-4 in dos write file name(ex-cmd.exe) and other arguments-

```
name of the program is=C:\TC\BIN\CMD.EXE
next value=67
C:\TC\BIN>exit

Type EXIT to return to Turbo C++. . .

Welcome to DOSBox v0.74

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

C:\TC\BIN>cmd.exe 55
```

# Step-5 Output

```
name of the program is=C:\TC\BIN\CMD.EXE
next value=55_
```

# Example-

```
#include <stdio.h>

int main(int argc, char *argv [])
{
 printf(" \n Name of my Program %s \t", argv[0]);
 if(argc == 2)
 {
 printf("\n Value given by user is: %s \t", argv[1]);
 }
 else if(argc > 2)
 {
 printf("\n Many values given by users.\n");
 } else
 {
 printf(" \n Single value expected.\n");
 }
}
```

# Output-

 Command Prompt

```
F:\>program.exe hello
```

```
 Name of my Program program.exe
```

```
 Value given by user is: hello
```

```
F:\>
```

# Example-

```
#include <stdio.h>

void main(int argc, char *argv[])
{

 printf("Program name is: %s\n", argv[0]);

 if(argc < 2){
 printf("No argument passed through command line.\n");
 }
 else{
 printf("First argument is: %s\n", argv[1]);
 }
}
```

# Continue..

- Run this program as follows in Windows from command line:
- program.exe hello
- **Output:**

Program name is: program

First argument is: hello

# Continue..

- If you pass many arguments, it will print only one.
- `program.exe hello c how r u`
- **Output:**

Program name is: program

First argument is: hello

- But if you pass many arguments within double quote, all arguments will be treated as a single argument only.
- `program.exe "hello c how r u"`
- **Output-**

Program name is: program

First argument is: hello c how r u

# Continue..

- We can write program to print all the arguments. In this program, we are printing only `argv[1]`, that is why it is printing only one argument.



# Preprocessor

The **C Preprocessor** is not a part of the compiler, but is a separate step in the compilation process. In simple terms, a C Preprocessor is just a text substitution tool and it instructs the compiler to do required pre-processing before the actual compilation. We'll refer to the C Preprocessor as CPP.

All preprocessor commands begin with a hash symbol (#). It must be the first nonblank character, and for readability, a preprocessor directive should begin in the first column. The following section lists down all the important preprocessor directives –

# Preprocessor

| Sr.No. | Directive & Description                                           |
|--------|-------------------------------------------------------------------|
| 1      | <b>#define</b><br>Substitutes a preprocessor macro.               |
| 2      | <b>#include</b><br>Inserts a particular header from another file. |
| 3      | <b>#undef</b><br>Undefines a preprocessor macro.                  |
| 4      | <b>#ifdef</b><br>Returns true if this macro is defined.           |
| 5      | <b>#ifndef</b><br>Returns true if this macro is not defined.      |
| 6      | <b>#if</b><br>Tests if a compile time condition is true.          |

|    |                                                                                         |
|----|-----------------------------------------------------------------------------------------|
| 7  | <b>#else</b><br>The alternative for #if.                                                |
| 8  | <b>#elif</b><br>#else and #if in one statement.                                         |
| 9  | <b>#endif</b><br>Ends preprocessor conditional.                                         |
| 10 | <b>#error</b><br>Prints error message on stderr.                                        |
| 11 | <b>#pragma</b><br>Issues special commands to the compiler, using a standardized method. |

```
#include <stdio.h>
```

```
#define DEBUG 1
```

```
int main() {
 #if DEBUG
 printf("Debugging is enabled\n");
 #else
 printf("Debugging is disabled\n");
 #endif
 return 0;
}
```

```
#include <stdio.h>
```

```
#define MAX 100
```

```
#undef MAX // The macro MAX is now undefined
```

```
int main() {
 // Now, trying to use MAX will result in a compilation error
 // printf("Max value is: %d\n", MAX); // Error: MAX is undefined
 return 0;
}
```

```
#include <stdio.h>
```

```
#define DEBUG
```

```
int main() {
 #ifdef DEBUG
 printf("Debug mode is enabled\n");
 #endif

 #ifndef RELEASE
 printf("Release mode is not enabled\n");
 #endif

 return 0;
}
```

# Preprocessors Examples

Analyze the following examples to understand various directives.

```
#define MAX_ARRAY_LENGTH 20
```

This directive tells the CPP to replace instances of MAX\_ARRAY\_LENGTH with 20. Use #define for constants to increase readability.

```
#include <stdio.h>
#include "myheader.h"
```

These directives tell the CPP to get stdio.h from **System Libraries** and add the text to the current source file. The next line tells CPP to get **myheader.h** from the local **directory** and add the content to the current source file.

```
#undef FILE_SIZE
#define FILE_SIZE 42
```

It tells the CPP to undefine existing FILE\_SIZE and define it as 42.

```
#ifndef MESSAGE
 #define MESSAGE "You wish!"
#endif
```

It tells the CPP to define MESSAGE only if MESSAGE isn't already defined.

# Predefined Macros

ANSI C defines a number of macros. Although each one is available for use in programming, the predefined macros should not be directly modified.

| Sr.No. | Macro & Description                                                                 |
|--------|-------------------------------------------------------------------------------------|
| 1      | <b>__DATE__</b><br>The current date as a character literal in "MMM DD YYYY" format. |
| 2      | <b>__TIME__</b><br>The current time as a character literal in "HH:MM:SS" format.    |
| 3      | <b>__FILE__</b><br>This contains the current filename as a string literal.          |
| 4      | <b>__LINE__</b><br>This contains the current line number as a decimal constant.     |
| 5      | <b>__STDC__</b><br>Defined as 1 when the compiler complies with the ANSI standard.  |

```
#include <stdio.h>
```

```
int main() {
```

```
 printf("File :%s\n", __FILE__);
```

```
 printf("Date :%s\n", __DATE__);
```

```
 printf("Time :%s\n", __TIME__);
```

```
 printf("Line :%d\n", __LINE__);
```

```
 printf("ANSI :%d\n", __STDC__);
```

```
}
```

When the above code in a file **test.c** is compiled and executed, it produces the following result -

```
File :test.c
```

```
Date :Jun 2 2012
```

```
Time :03:36:24
```

```
Line :8
```

```
ANSI :1
```

```
// Open a text file to write the output
FILE *file = fopen("student_report.txt", "w");
if (file == NULL) {
 printf("Error opening file for writing\n");
 return 1;
}

// Write the details of the students to the file
fprintf(file, "Roll No\tName\t\tMarks1\tMarks2\tMarks3\tTotal\tPercentage\n");
fprintf(file, "-----\n");

for (int i = 0; i < 3; i++) {
 fprintf(file, "%d\t%s\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f%%\n",
 students[i].rollno, students[i].name, students[i].m1,
 students[i].m2, students[i].m3, students[i].total,
 students[i].percentage);
}

// Close the file after writing
fclose(file);

printf("Student report has been written to 'student_report.txt'.\n");
```